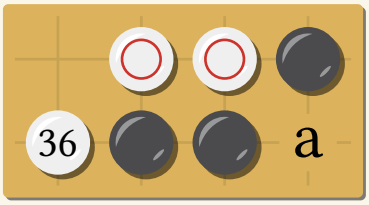
gobanist



Tableros, partidas y problemas de go para Typst

Manual de la versión 0.2.0

Jonatán Perren Hug
jonatan.perren@gmail.com

Índice

1 Presentación ... 4
- 1.1 Qué es gobanist ... 4
- 1.2 Primeros pasos ... 4
- 1.3 Cómo leer este manual ... 4
- 1.4 Para quien viene de pgf-go ... 5

2 Los tableros de Kāru ... 7
- 2.1 La primera partida ... 7
- 2.2 Una posición a mano ... 8
- 2.3 Jugadas ... 10
- 2.4 Tablero parcial y marcas ... 11
- 2.5 Explicar una pelea: flechas, letras y vistas ... 12
- 2.6 Un ko: capturas y notas ... 14
- 2.7 Una partida entera ... 16
- 2.8 Variantes: seguir desde un tablero ... 18
- 2.9 El final del juego: territorio ... 19
- 2.10 Un cuaderno de problemas ... 20
- 2.11 Un tablero de madera: un perfil con imágenes ... 22
- 2.12 Diapositivas ... 25
- 2.13 Un diccionario de josekis ... 26
- 2.14 Otros idiomas ... 27
- 2.15 Escribir menos: la sintaxis de coordenadas ... 28

3 Configuración ... 30
- 3.1 Las opciones ... 30
- 3.2 El kernel ... 31
- 3.3 Idiomas ... 31

4 Sintaxis de coordenadas ... 33
- 4.1 Formas sueltas ... 33
- 4.2 Caminos ... 35
- 4.3 El orden de las intersecciones ... 37
- 4.4 Quitar intersecciones ... 38

5 El tablero ... 40
- 5.1 La cuadrícula ... 42
- 5.2 Puntos de estrella ... 44
- 5.3 El fondo ... 46
- 5.4 Tablero parcial ... 50
- 5.5 Las coordenadas del borde ... 51

6 Piedras, jugadas y prisioneros ... 54
- 6.1 Piedras ... 54
- 6.2 Jugadas ... 60
- 6.3 Prisioneros ... 62

7 Marcas, flechas y líneas ... 64
- 7.1 Las formas ... 64

7.2 Las opciones de las marcas · · · 66
7.3 Formas propias · · · 68
7.4 Flechas y líneas · · · 69

8 El juego: capturas, notas y territorio · · · 72
8.1 Capturas · · · 72
8.2 Jugadas en puntos ya jugados · · · 73
8.3 Territorio · · · 74
8.4 Puntaje · · · 78

9 Tableros como valores y vistas · · · 80
9.1 Seguir desde un tablero · · · 80
9.2 Vistas en el texto · · · 81
9.3 Una piedra en el texto · · · 83

10 Partidas en SGF · · · 85
10.1 Leer una partida · · · 85
10.2 Diagramas de Sensei's Library · · · 87
10.3 Una partida en un tablero · · · 87
10.4 Datos y comentarios · · · 91
10.5 Diagramas como en los libros · · · 92
10.6 Problemas · · · 99

11 Disposición de los diagramas · · · 103

12 Jugadores y perfiles · · · 107
12.1 Jugadores · · · 107
12.2 Perfiles · · · 108

13 Catálogo de perfiles · · · 109
13.1 default · · · 109
13.2 pgfgo · · · 109
13.3 pgfgosmooth · · · 110
13.4 wain · · · 110
13.5 simple · · · 111
13.6 draft · · · 111
13.7 shell · · · 112
13.8 yunzi · · · 113
13.9 baduktv · · · 113

14 Novedades · · · 115

Índice de funciones y opciones · · · 116

1 Presentación

1 Qué es gobanist

gobanist es un paquete de Typst para dibujar tableros, posiciones, partidas y problemas de go. Es un port de pgf-go, un paquete de LaTeX publicado hace dos años por el mismo autor, con sus mismas ideas y casi todas sus opciones, escritas con los valores de Typst:

- Partidas en SGF: una partida en un tablero, series de diagramas como en los libros y problemas con sus soluciones.
- Las reglas del juego, en un kernel de WebAssembly: capturas, notas «N en M» para los kos, territorio y puntaje.
- Perfiles para el tablero y los jugadores, que pueden usar imágenes: madera en el fondo, piedras de pizarra y de concha.
- Tableros como valores, para seguir una variante desde una posición o mostrar una parte del tablero en medio de un párrafo.
- Marcas, flechas y líneas, y formas de marca propias.
- Una disposición en grilla para acomodar diagramas del mismo tamaño.
- Los textos del paquete en diez idiomas, y coordenadas del borde al estilo occidental, japonés o chino.
- Una sintaxis para escribir muchas coordenadas a la vez: filas, columnas, rectángulos, caminos y escaleras.

El dibujo, las coordenadas y las opciones están escritos en Typst. Lo que necesita jugar la partida (las capturas, el territorio, las regiones, el puntaje y la lectura de los archivos SGF) lo hace un plugin de WebAssembly compilado de código Rust (pág. 31), que viene con el paquete.

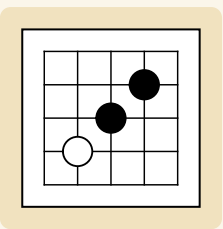
2 Primeros pasos

gobanist se importa como cualquier paquete. Necesita Typst 0.15 o posterior.

```
#import "@preview/gobanist:0.2.0": *

#goban(stones(black: "C3, D4", white: "E5"))
```

Todo lo que se dibuja va en un `goban`: las piedras, las jugadas, las marcas, las flechas, el territorio o una partida leída de un archivo. Las opciones del tablero van en el mismo `goban`, con nombre (`size: 13`), y las de todo el documento, en `gobanist-set` (pág. 30).



```
#goban(size: 5, stones(black: "C3, D4", white: "B2"))
```

3 Cómo leer este manual

El manual tiene tres partes: esta presentación; los tutoriales (pág. 7), en los que Kāru prepara un artículo, un cuaderno de problemas y unas clases, y aprende el paquete en el camino; y la referencia, con cada función

y cada opción, ordenada por objeto. Al final están el catálogo de perfiles y un índice de las funciones y las opciones.

Conviene leer primero los tutoriales, de corrido, para tener un pantallazo de todo, y después la referencia, para ver todo lo que se puede configurar. Para algo puntual:

- **Dibujar una partida guardada en SGF**: el tutorial de la pág. 7 y, para la partida entera en diagramas, el de la pág. 16. La referencia está en la pág. 85.
- **Dibujar una posición a mano**: el tutorial de la pág. 8, y las piedras en la pág. 54. Para escribir menos coordenadas, el tutorial de la pág. 28 y la sintaxis completa en la pág. 33.
- **Numerar jugadas**: el tutorial de la pág. 10, y `moves` en la pág. 60.
- **Mostrar solo una parte del tablero o marcar puntos**: el tutorial de la pág. 11 `partial` en la pág. 50, y las marcas en la pág. 64.
- **Flechas, líneas y letras**: el tutorial de la pág. 12, y la referencia en la pág. 69.
- **Mostrar una piedra o una parte del tablero dentro del texto**: el final del tutorial de la pág. 12, y `goban-view` en la pág. 81.
- **Capturas, kos y notas «N en M»**: el tutorial de la pág. 14, y la referencia en la pág. 72.
- **Marcar el territorio y contar los puntos**: el tutorial de la pág. 19, y `territory` y `goban-score` en la pág. 74.
- **Seguir desde una posición, por ejemplo para mostrar variantes**: el tutorial de la pág. 18, y `board` y `resume` en la pág. 80.
- **Josekis de un diccionario en SGF**: el tutorial de la pág. 26, y `variation` y `variations` en la pág. 85.
- **Problemas (tsumego) con sus soluciones**: el tutorial de la pág. 20, y `sgf-problem` en la pág. 99.
- **Acomodar varios diagramas en una grilla**: los tutoriales de las páginas 16 y 20, y `layout` y `goban-layout` en la pág. 103.
- **Cambiar el estilo de todos los tableros**: el final del tutorial de la pág. 8 y el catálogo (pág. 109). Para cambiar un detalle, `gobanist-set` (pág. 30) y las opciones de cada objeto; para guardar un estilo propio, los perfiles (pág. 108).
- **Usar imágenes en el tablero y en las piedras**: el tutorial de la pág. 22 el fondo y la cuadrícula en la pág. 40, y las piedras en la pág. 58.
- **Diapositivas**: el tutorial de la pág. 25.
- **Escribir en otro idioma**: el tutorial de la pág. 27, y la referencia en la pág. 31.

Una regla general: las ramas que prenden o apagan algo (la cuadrícula, el fondo, las sombras…) tienen la clave `on`; `true` y `false` las prenden y las apagan, y un diccionario cambia sus opciones sin prenderlas (pág. 30).

4 Para quien viene de pgf-go

Las opciones de gobanist son las de pgf-go, con la sintaxis de Typst: un árbol de diccionarios en lugar de un árbol de *keys*, y valores de Typst en lugar de los de TikZ. Los cambios más visibles:

pgf-go	gobanist
`\begin{goban}[13] ... \end{goban}`	`#goban(size: 13, ...)`
`\pgfgoset{goban = {...}}`	`#gobanist-set(...)`
`\stones[b]{C3, D4}`	`stones(black: "C3, D4")`
`\stone{C3}`, `\stones{C3}`	`stones("C3")`
`\moves[w]{D2, B3}`	`moves(white: "D2, B3")`
`\mark[triangle]{C3}`	`marks(triangle: "C3")`
`\mark[custom = {!}]{D3}`	`marks(custom: "D3", content: [!])`
`\arrow{B3}{F3}`, `\line{B3}{F3}`	`arrows(("B3", "F3"))`, `lines(("B3", "F3"))`
`\territory[b]{auto}`	`territory(black: auto)`
`shine = {true, angle = 45}`	`shine: (on: true, angle: 45deg)`
`format = Alph`	`format: "A"` (un patrón de `numbering`)

pgf-go	gobanist
`label = {at = {north, west}}`	`label: (at: (top, left))`
`name = base`, `resume = base`	`#let base = board(...)`, `goban(base)`, `resume: base`
`\pgfgoview{base}{C3}`	`goban-view(base, "C3")`
`\pgfgodefinesgf{partida}{partida.sgf}`	`#let partida = sgf-read(path("partida.sgf"))`
`\pgfgodefinesgf[origin = top right]`	`sgf-read(..., origin: top + right)`
`\begin{pgfgosgf}{corta} ... \end{pgfgosgf}`	`#let corta = sgf-read("(;SZ[9]...)")`
`\sgf[to = 7]{partida}`	`sgf(partida, to: 7)`
`\sgfdiagrams`, `\sgfproblem`	`sgf-diagrams`, `sgf-problem`
`\pgfgomovenotes`, `\pgfgoscore{b}`	`goban-notes()`, `goban-score("black")`
`pgfgolayout`	`goban-layout(...)`
`\pgfgodefinemark`	la opción `shapes`
`\pgfgostone[b, number = 5]`	`goban-stone("black", number: 5)`
`label = {letter case = lower}`	`label: (columns: "a")`
`layout = {halign = left}`	`layout: (halign: left)`
`\pgfgodefineprofile{madera}{...}`	`#let madera = (...)`, `profile: madera`

Algunas cosas cambian de comportamiento:

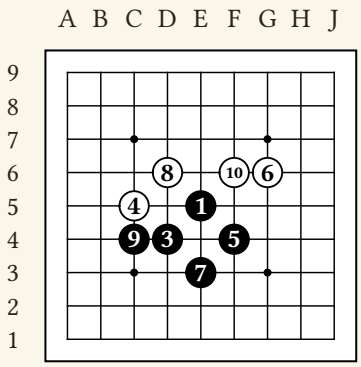
- Una marca sin jugador sobre una piedra toma el jugador de la piedra (en pgf-go era neutral).
- Typst no tiene avisos: lo que en pgf-go avisaba (una coordenada que no se entiende o que está fuera del tablero, una partida de otro tamaño) es un error. Una jugada en un punto ocupado, con capturas, también es error, salvo en `sgf-diagrams` y `sgf-problem`.
- Los grises son los de xcolor: el gris de gobanist es `luma(50%)`, no el `gray` de Typst.
- El tablero tiene al menos 2×2 líneas.
- Las imágenes son `path` o `bytes`, no nombres de archivo.

Y algunas no se portan: `\sgfanimation` (Typst no hace PDF animados; para diapositivas están Touying y Polylux, pág. 25), `export` y `\pgfgoimport` (un tablero es un valor: se guarda en un módulo `.typ` y se importa), el dibujo con TikZ dentro del tablero, el sistema de coordenadas `go cs`, `\pgfgodefinegoban`, `\pgfgodefineplayer` y los perfiles parciales (`goban profile`, `players profile`): un perfil es un diccionario, y se toman sus ramas.

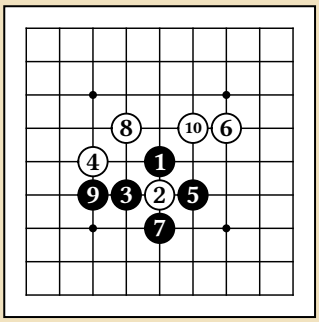
2 Los tableros de Kāru

1 La primera partida

Kāru está escribiendo un artículo para sus estudiantes y decide probar gobanist. Lo primero que quiere mostrar es una partida corta que jugó en clase. La tiene guardada en un archivo SGF, `partida.sgf`: el formato en el que casi todos los programas y servidores de go guardan las partidas.



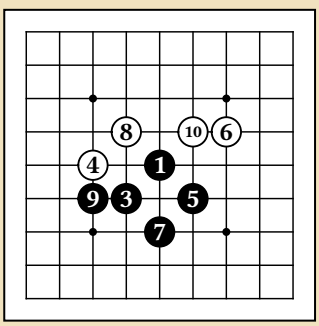
`sgf-read` lee el archivo y devuelve la partida como un valor, que Kāru guarda con `let` para usarlo muchas veces. Todo lo que se dibuja va en un `goban`, y la partida también: dentro de él, dibuja su línea principal. El tablero toma el tamaño del archivo.



```
#let partida = sgf-read(path("partida.sgf"))

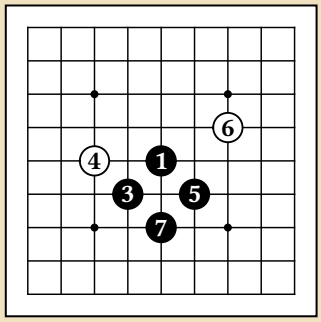
#goban(partida)
```

Las jugadas aparecen numeradas, como en los libros. Pero con la jugada 7 las negras capturaron la piedra blanca 2, que sigue en el tablero: gobanist dibuja lo que se le pide, sin aplicar las reglas. Con la opción `captures`, el `goban` retira las piedras capturadas (pág. 72). Las opciones del tablero van con nombre, junto a lo que se dibuja.



```
#goban(captures: "remove", partida)
```

Para comentar la partida en el texto, Kāru la detiene en la jugada 7 con `to`: `sgf(partida, ...)` es la misma partida con otras opciones. El archivo trae un comentario para esa jugada, y `sgf-comment` lo escribe.



Jugada 7: Tesuji. Negras capturan.

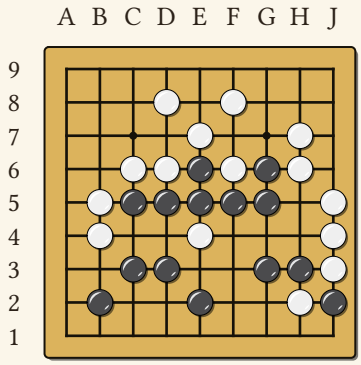
```
#goban(captures: "remove", sgf(partida, to: 7))

Jugada 7: #sgf-comment(partida, 7)
```

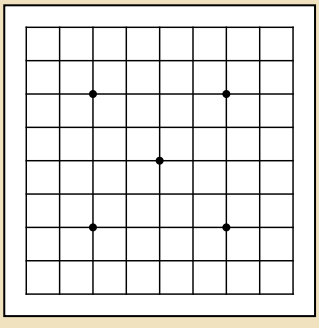
Esta partida es de 9×9; una de 19×19 se lee igual, y el tablero toma su tamaño. Así lo hará Kāru con la partida de la pág. 16, donde también verá cómo dibujar una partida larga en una serie de diagramas.

2 Una posición a mano

Para la introducción del artículo, Kāru quiere una posición propia, que no está en ningún archivo. Tiene que dibujarla piedra por piedra.

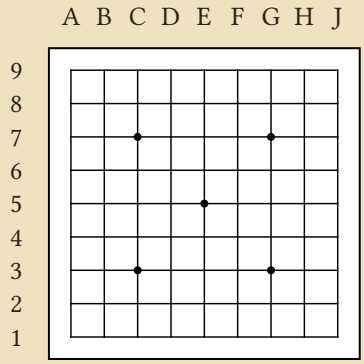


Empieza por el tablero vacío. `goban` se encarga del tamaño del tablero, su color, sus coordenadas y todo lo que va encima.



```
#goban()
```

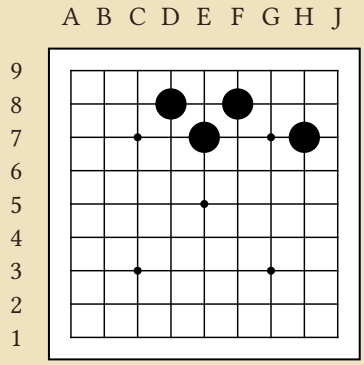
El tablero es de 9×9, justo lo que necesita, pero le faltan las coordenadas en los bordes. Las opciones de cada parte del tablero se agrupan en ramas: `grid`, la cuadrícula; `background`, el borde y el relleno, y `label`, las coordenadas (pág. 40). Dentro de `label`, `at` dice en qué lados van: `top`, `bottom`, `left` y `right`, las alineaciones de Typst. Kāru las quiere arriba y a la izquierda.




```
#goban(label: (at: (top, left)))
```

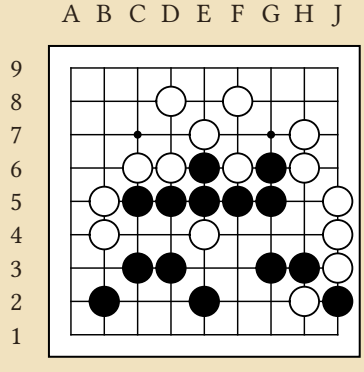
Las columnas pasan de la H a la J: como es costumbre en el go, gobanist omite la I, en las coordenadas y en las etiquetas. Si alguna vez la necesita, la opción `use-i` (pág. 41) la vuelve a poner.

Ahora sí, las piedras. `stones` recibe las intersecciones, como `"D8"`, y el jugador es la clave: `black` o `white`. Sin jugador, la piedra es neutral.



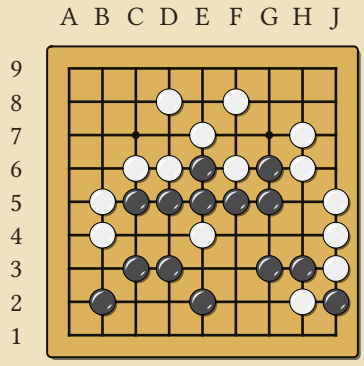

```
#goban(label: (at: (top, left)),
  stones(black: "D8, F8, E7, H7"))
```

Una sola cadena lleva todas las piedras de un jugador, separadas por comas, y un mismo `stones` puede llevar las de los dos. En rigor, `black` y `white` no son colores sino jugadores: cada piedra toma el estilo del suyo (pág. 107).




```
#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

Falta el estilo. En el catálogo (pág. 109), Kāru encuentra uno que le gusta: el perfil `pgfgo`. Lo aplica con `gobanist-set(profile: "pgfgo")` (pág. 108), y desde ahí vale para todos los tableros que siguen.



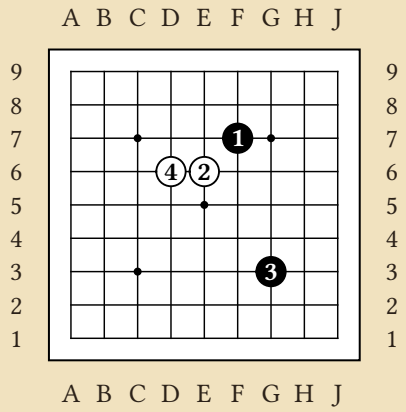
```
#gobanist-set(profile: "pgfgo")

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

Las listas de este tutorial son largas, y la paciencia de Kāru, corta. En el último tutorial (pág. 28) aprende a escribirlas más cortas.

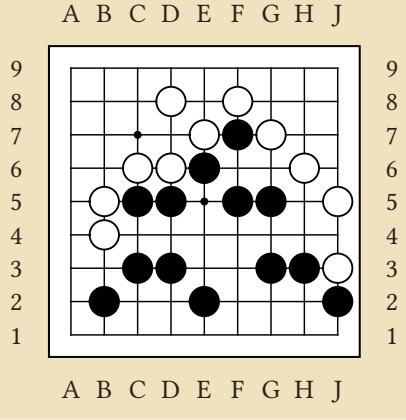
3 Jugadas

Para explicar cómo empezó una pelea, Kāru quiere mostrar las piedras en el orden en que se jugaron. Con `stones` tendría que separar las de cada jugador y perdería el orden. Para eso está `moves` (pág. 60): juega las intersecciones en el orden en que se escriben, alterna los colores y numera las jugadas.



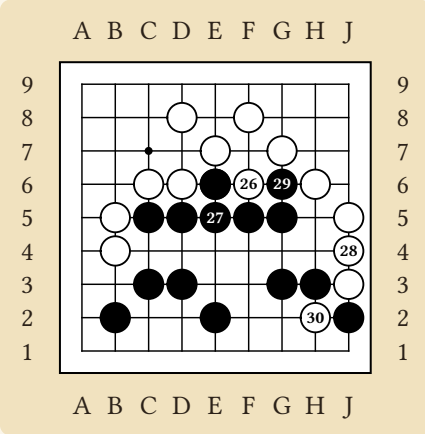
```
#goban(label: (at: (top, bottom, left, right)),
  moves("F7, E6, G3, D6"))
```

La pelea empezó con una negra en F7, que Kāru pone con `stones`. Las jugadas que siguen no las va a comentar: con `numbers: false`, se dibujan sin números. Con `white` como clave, la secuencia empieza con blancas: en `moves`, el jugador solo dice quién juega primero.



```
#goban(label: (at: (top, bottom, left, right)),
  stones(black: "F7"),
  moves(numbers: false, white: "E6, G3, D6, F5,
    E7, E2, F8, E6, D8, D3, G7, C5, C6, D5,
    B5, G5, B4, B2, H6, C3, J3, J2, J5, H3"))
```

Las últimas jugadas sí las comenta, así que van con número. La numeración y el turno siguen de un `moves` al siguiente, aunque las jugadas anteriores no lleven números, y con `from` se elige desde qué número se sigue.

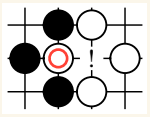



```
#goban(label: (at: (top, bottom, left, right)),
  moves(numbers: false, white: "E6, G3, D6, F5,
    E7, E2, F8, E6, D8, D3, G7, C5, C6, D5,
    B5, G5, B4, B2, H6, C3, J3, J2, J5, H3"),
  moves("F6, E5, J4, G6", from: 26),
  moves("H2"))
```

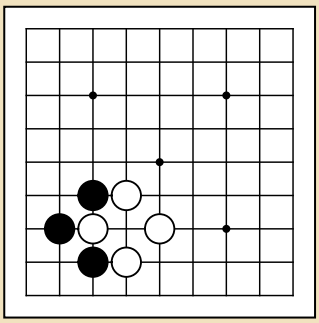
`H2` va en otro `moves` y, sin decirlo, es blanca y lleva el 30: la jugada anterior, `G6`, fue la 29, de negras.

4 Tablero parcial y marcas

Para un ejercicio del artículo, Kāru quiere mostrar solo una parte del tablero, con dos señales: un círculo en la piedra que está en atari y un «!» en el punto clave.

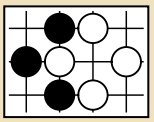


Empieza, como siempre, por las piedras.



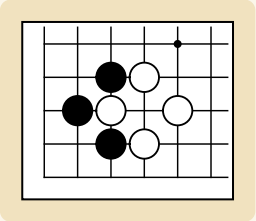
```
#goban(stones(black: "C2, B3, C4",
  white: "D4, E3, D2, C3"))
```

Con la opción `partial`, se indica qué parte del tablero mostrar: dos esquinas opuestas del rectángulo, separadas por dos puntos, como `"B2:E4"` (pág. 50).



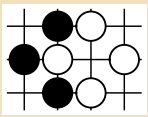
```
#goban(partial: "B2:E4",
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2, C3"))
```

Si Kāru no quiere buscar las esquinas, con `partial: auto` el tablero elige la parte: la que usa lo que se dibuja, con una línea más alrededor. Un lado que queda a dos líneas o menos del borde llega hasta el borde, como aquí el de abajo y el de la izquierda.



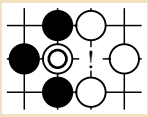
```
#goban(partial: auto,
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2, C3"))
```

Kāru prefiere el recorte justo, así que sigue con `"B2:E4"`. En un tablero parcial, el borde del tablero sobra: lo quita con `line: false` en el `background`.



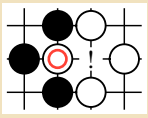
```
#goban(partial: "B2:E4",
  background: (line: false),
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2, C3"))
```

Las marcas se ponen con `marks`, con la forma como clave; hay muchas (pág. 64). A Kāru le alcanzan dos: `circle`, el círculo, y `custom`, que pone cualquier contenido (`content`), como «!». Una marca también puede ir en las opciones de la piedra, con `mark`.



```
#goban(partial: "B2:E4",
  background: (line: false),
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2"),
  stones(white: "C3", mark: "circle"),
  marks(custom: "D3", content: [!]))
```

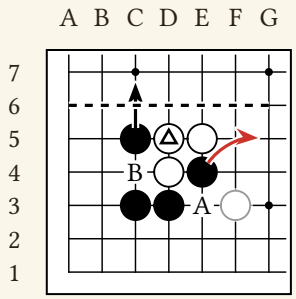
Para que el círculo se distinga, Kāru lo pinta de rojo, con `line.color` de la marca. Con opciones, la marca es un diccionario, y la forma va en `shape`.



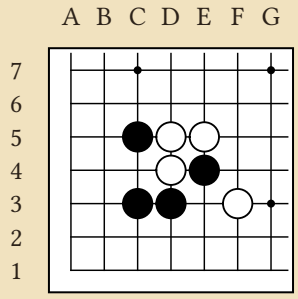
```
#goban(partial: "B2:E4",
  background: (line: false),
  stones(black: "C2, B3, C4",
    white: "D4, E3, D2"),
  stones(white: "C3", mark: (shape: "circle",
    line: (color: red))),
  marks(custom: "D3", content: [!]))
```

5 Explicar una pelea: flechas, letras y vistas

En la segunda parte del artículo, Kāru quiere explicar una pelea. Necesita señalar las jugadas posibles con letras, mostrar hacia dónde corre cada grupo y, en el texto, nombrar las piedras sin obligar al lector a buscarlas en el diagrama.

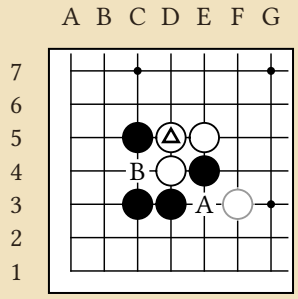


Empieza, como siempre, por las piedras. Para no ocupar espacio, muestra solo la esquina con `partial`.



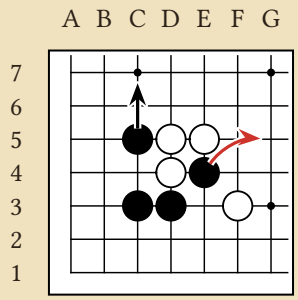

```
#goban(partial: "A1:G7",
  label: (at: (top, left)),
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"))
```

Las jugadas candidatas se marcan con letras, con `custom`, que acepta cualquier contenido. La marca `triangle` destaca la piedra de la que habla el texto, y `dim` apaga la piedra blanca de F3, que ya no importa. Un mismo `marks` puede llevar varias formas; dos `custom` con distinto contenido van en dos `marks`.



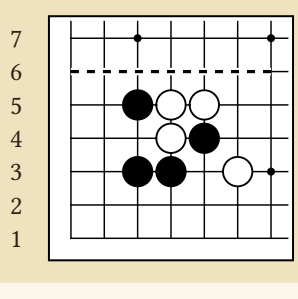

```
#goban(partial: "A1:G7",
  label: (at: (top, left)),
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"),
  marks(dim: "F3", triangle: "D5"),
  marks(custom: "E3", content: [A]),
  marks(custom: "C4", content: [B]))
```

Para mostrar hacia dónde corren los grupos, Kāru usa `arrows`, que une dos intersecciones (pág. 69). La flecha se dibuja con un halo del color del tablero, para que no se confunda con la cuadrícula. Con `bend`, es un arco; con `shorten`, no tapa las piedras de sus extremos.




```
#goban(partial: "A1:G7",
  label: (at: (top, left)),
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"),
  arrows(("C5", "C7"), shorten: 4pt),
  arrows(("E4", "G5"), color: red.darken(20%),
    bend: 30deg, shorten: 4pt))
```

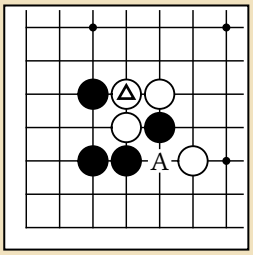
`lines` es como `arrows`, pero sin punta, y acepta las mismas opciones de trazo, como `dash`.




```
#goban(partial: "A1:G7",
  label: (at: (top, left)),
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"),
  lines(("A6", "G6"), dash: "dashed"))
```

Ahora, el texto. Kāru quiere escribir «la piedra negra de E4» y mostrarla ahí mismo. Para eso guarda el tablero como un valor, con `board`, que recibe lo mismo que `goban` pero no dibuja nada. Para dibujarlo, el tablero va

primero en un `goban`, que le agrega sus opciones, como la parte que se ve. Y `goban-view` muestra una parte de ese tablero en medio del párrafo, como quedó al final (pág. 81): un punto o un rectángulo.




```
#let pelea = board(
  stones(black: "C3, D3, E4, C5",
    white: "D4, D5, E5, F3"),
  marks(triangle: "D5"),
  marks(custom: "E3", content: [A]))

#goban(pelea, partial: "A1:G7")
```

```
La negra #goban-view(pelea, "E4") está separada de sus compañeras.
Si juega en A, conecta: #goban-view(pelea, "C3:E5").
Sin marcas: #goban-view(pelea, "C3:E5", marks: false).
```

La negra  está separada de sus compañeras. Si juega en A, conecta: 

. Sin marcas:  .

La vista tiene el estilo del tablero, pero `view.goban` puede cambiarlo: Kāru prefiere quitar el fondo en todas las vistas del artículo.

```
#gobanist-set(view: (goban: (background: false)))

La negra #goban-view(pelea, "E4") está separada de sus compañeras.
Si juega en A, conecta: #goban-view(pelea, "C3:E5").
```

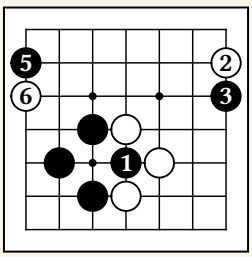
La negra  está separada de sus compañeras. Si juega en A, conecta: 

.

Como el tablero es un valor, la vista puede ir antes que el diagrama que lo dibuja: `pelea` se conoce desde el `let`, no desde el `goban`.

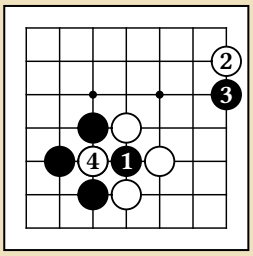
6 Un ko: capturas y notas

En el primer tutorial, `captures` retiró una piedra capturada de una partida en SGF. Para explicar un ko, Kāru necesita lo mismo con jugadas escritas a mano: ponerlas en orden y que gobanist retire las piedras capturadas.



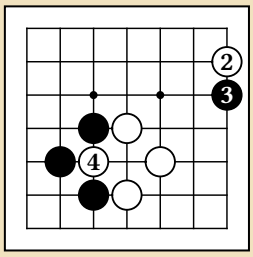
7 en 1

Negras toman el ko en D3 (1), blancas amenazan en otro lado (2), negras responden (3) y blancas retoman (4). Sin más, las jugadas se dibujan unas sobre otras y ninguna piedra se va del tablero.




```
#goban(size: 7,
  stones(black: "B3, C2, C4",
    white: "C3, D2, D4, E3"),
  moves("D3, G6, G5, C3"))
```

Con `captures: "remove"` (pág. 72), el `goban` lleva la cuenta de las libertades y retira cada grupo que se queda sin ellas. La blanca de C3 desaparece con la jugada 1, y la negra 1, con la 4. `goban-prisoners` dice cuántas piedras capturó cada jugador.

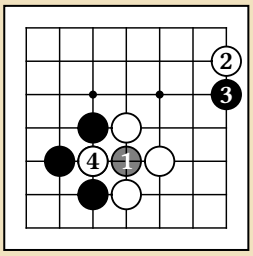


Negras capturaron 1, blancas 1.

```
#goban(size: 7, captures: "remove",
  stones(black: "B3, C2, C4",
    white: "C3, D2, D4, E3"),
  moves("D3, G6, G5, C3"))

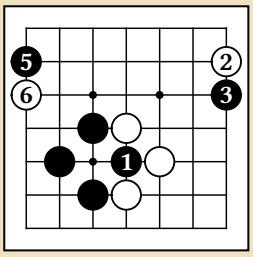
Negras capturaron
#goban-prisoners("black"), blancas
#goban-prisoners("white").
```

Si las piedras capturadas deben seguir viéndose, `captures: "fade"` las deja como prisioneras.




```
#goban(size: 7, captures: "fade",
  stones(black: "B3, C2, C4",
    white: "C3, D2, D4, E3"),
  moves("D3, G6, G5, C3"))
```

El ko sigue: negras amenazan (5), blancas responden (6) y negras retoman en D3 (7), donde ya se había jugado la 1. En los libros, esa jugada no se dibuja: debajo del diagrama se escribe «7 en 1». Eso hace la opción `notes` (pág. 73), y `goban-notes` escribe las notas.



7 en 1.

```
#goban(size: 7, captures: "remove",
  notes: true,
  stones(black: "B3, C2, C4",
    white: "C3, D2, D4, E3"),
  moves("D3, G6, G5, C3, A6, A5, D3"))

#goban-notes().
```

7 Una partida entera

El artículo también comenta una partida larga: la primera entre Lee Sedol y AlphaGo, de 2016, guardada en `lee.sgf`. Como en el primer tutorial (pág. 7), no hace falta escribir sus 186 jugadas: gobanist las lee del archivo (pág. 85).

Lo primero son los datos de la partida, que `sgf-info` toma de la raíz del archivo.

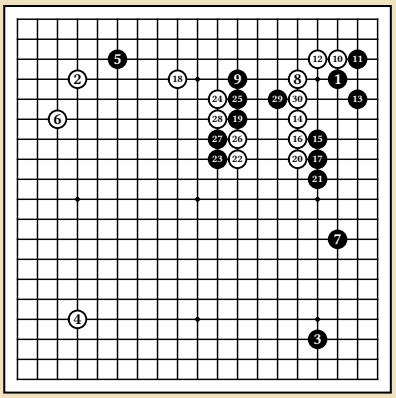
```
#let lee = sgf-read(path("lee.sgf"))

#sgf-info(lee, "PB") (negras) contra #sgf-info(lee, "PW") (blancas),
#sgf-info(lee, "EV"), #sgf-info(lee, "RO"), #sgf-info(lee, "DT").
Resultado: #sgf-info(lee, "RE").
```

Lee Sedol (negras) contra AlphaGo (blancas), Google DeepMind Challenge Match, Game 1, 2016-03-09. Resultado: W+Resign.

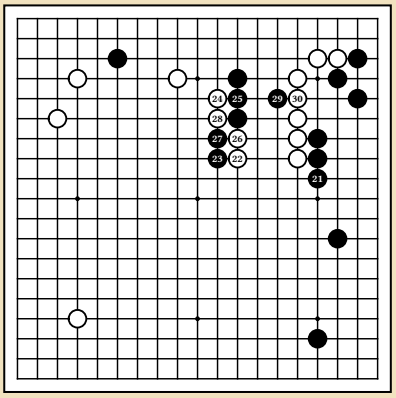
`sgf-info` escribe el resultado tal como está en el archivo; más adelante, `sgf-diagrams` lo escribe en palabras.

La partida es de 19×19, y el tablero toma ese tamaño. Con `to`, Kāru la muestra hasta la jugada 30, y con `scale`, achica el tablero.



```
#goban(captures: "remove", scale: 0.6,
  sgf(lee, to: 30))
```

Para comentar un tramo, Kāru numera solo las jugadas desde la 21 con `from`: las anteriores se dibujan sin número.



```
#goban(captures: "remove", scale: 0.6,
  sgf(lee, from: 21, to: 30))
```

Después, Kāru quiere la partida entera, como en los libros: un diagrama cada 50 jugadas, cada uno con su epígrafe y sus notas «N en M», y el resultado al final. Eso hace `sgf-diagrams`, que va fuera de `goban`. Con `layout`, los diagramas van en una grilla de dos columnas, y con `fit` se escalan para llenarlas (pág. 103). Aquí se muestran solo los dos últimos, con `from`.

```
#sgf-diagrams(lee, step: 50, from: 101,
  layout: (columns: 2, fit: true))
```

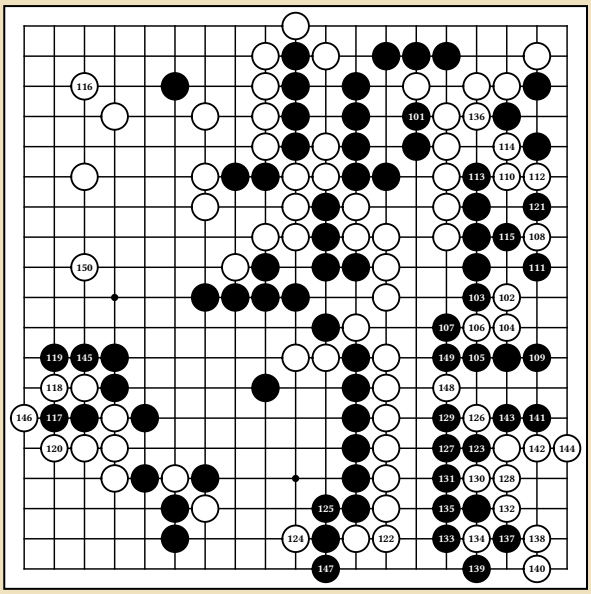


Diagrama 3 (101–150)

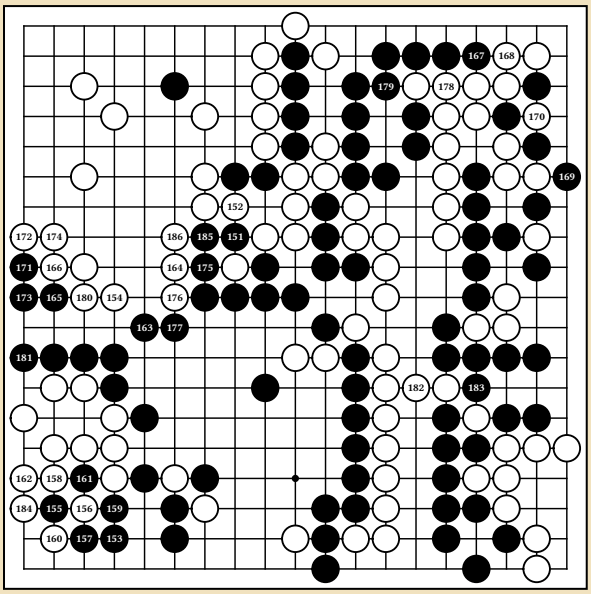


Diagrama 4 (151–186)
Resultado: Gana Blanco por abandono

El epígrafe y el resultado están en castellano porque el documento está en castellano (`text.lang`, pág. 31). Algunos libros reinician la numeración en cada diagrama; con `restart`, cada diagrama, desde el segundo, numera desde 1.

```
#sgf-diagrams(lee, step: 50, from: 51, to: 150,
  restart: true, goban: (label: (at: (top, left))),
  layout: (columns: 2, fit: true))
```

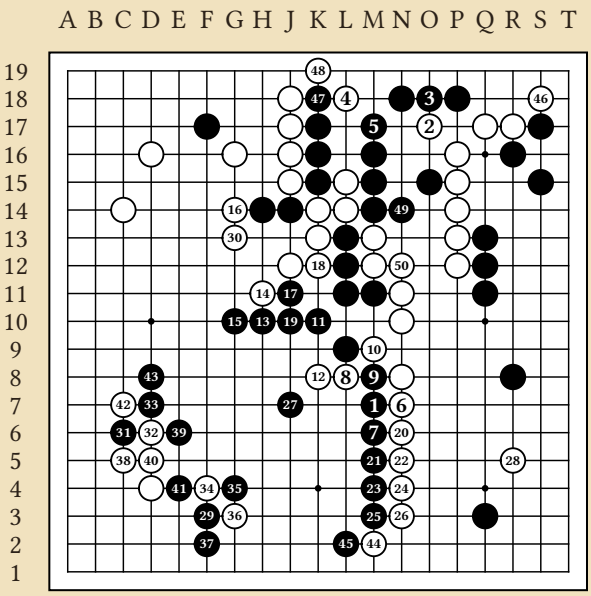


Diagrama 2 (51–100)

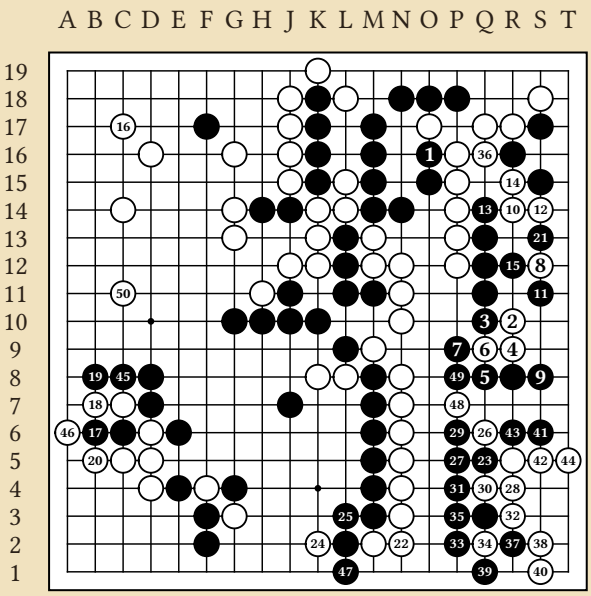
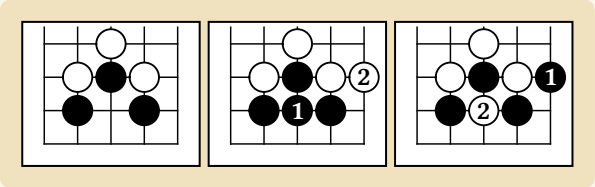


Diagrama 3 (101–150)

8 Variantes: seguir desde un tablero

En los comentarios, Kāru suele mostrar qué habría pasado con otra jugada. Copiar la posición en cada diagrama es tedioso y propenso a errores: si cambia una piedra, hay que cambiarla en todos. En la pelea, Kāru guardó un tablero como un valor, con `board`. Con `resume`, otro tablero empieza con ese, como quedó: la numeración y el turno siguen desde ahí (pág. 80).

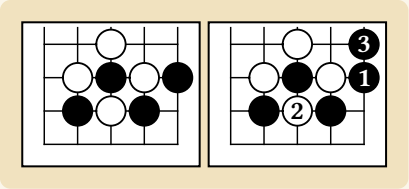



```
#gobanist-set(size: 5, partial: "A1:E4")

#let base = board(
  stones(black: "B2, C3, D2",
    white: "B3, C4, D3"))

#goban(base)
#goban(resume: base, moves(black: "C2, E3"))
#goban(resume: base, moves(black: "E3, C2"))
```

Una variante también puede ser un valor, y seguir desde ella: un `board` con `resume` tiene lo que trajo más lo suyo. Por defecto, `resume` trae solo las piedras; `numbers: true` trae también los números.




```
#gobanist-set(size: 5, partial: "A1:E4")

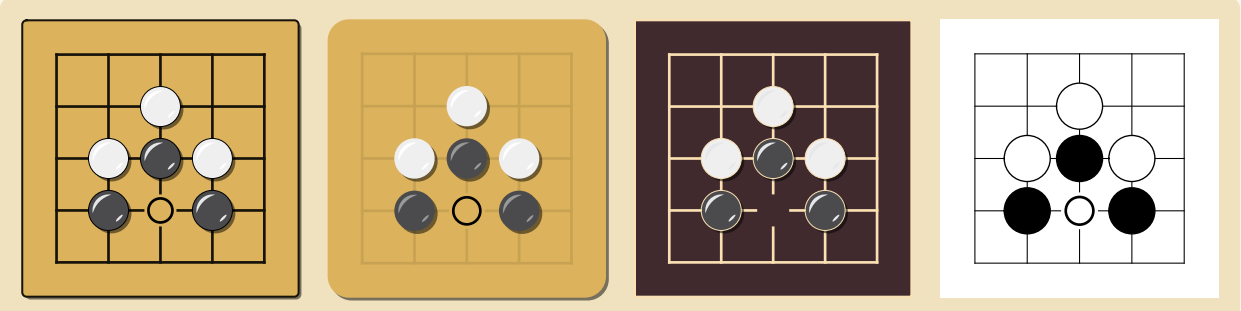
#let base = board(
  stones(black: "B2, C3, D2",
    white: "B3, C4, D3"))
#let variante = board(resume: base,
  moves(black: "E3, C2"))

#goban(resume: variante)
#goban(resume: (board: variante,
  numbers: true), moves("E4"))
```

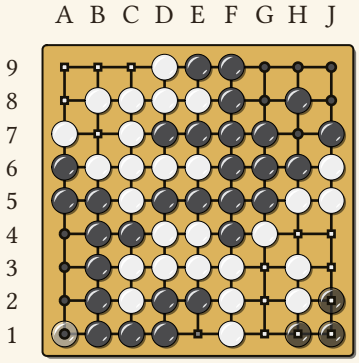
Un tablero guardado también sirve para comparar estilos. Antes de elegir el perfil de su artículo, Kāru dibuja el mismo tablero con varios perfiles del catálogo (pág. 109). Para ponerlos uno al lado del otro, del mismo ancho, usa `goban-layout` (pág. 103), que dispone tableros hechos con `board`. Cada uno es `estilos` con otro perfil: un tablero que va primero en `board` es ese mismo tablero, con lo que se le agrega.

```
#let estilos = board(size: 5,
  stones(black: "B2, C3, D2", white: "B3, C4, D3"),
  marks(circle: "C2"))
#let con(perfil) = board(estilos,
  profile: perfil)

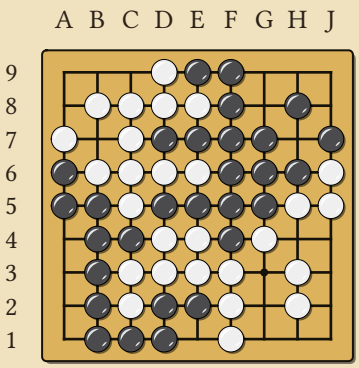
#goban-layout(columns: 4, fit: true,
  con("pgfgo"), con("pgfgosmooth"), con("wain"), con("simple"))
```



9 El final del juego: territorio



Para terminar el artículo, Kāru quiere mostrar cómo se cuenta una partida. Parte de la posición final. Como tiene muchas piedras y va a dibujarla varias veces, la guarda con `board`, como las variantes del tutorial anterior, y los tableros siguientes la llevan primero, sin repetir las coordenadas.

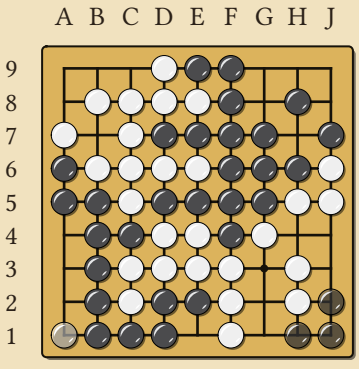



```
#gobanist-set(profile: "pgfgo")

#let final = board(stones(
  black: "A5, A6, B1, B2, B3, B4, B5, C1, C4,
    D1, D2, D5, D7, E2, E5, E7, E9, F4, F5,
    F6, F7, F8, F9, G5, G6, G7, H6, H8, J7",
  white: "A7, B6, B8, C2, C3, C5, C6, C7, C8,
    D3, D4, D6, D8, D9, E3, E4, E6, E8, F1,
    F2, F3, G4, H2, H3, H5, J5, J6"))

#goban(final, label: (at: (top, left)))
```

Lo siguiente son los prisioneros: las piedras muertas, que se retiran al contar. Se dibujan con `prisoners`, que recibe lo mismo que `stones` (pág. 62). Kāru guarda también este tablero, con otro nombre.

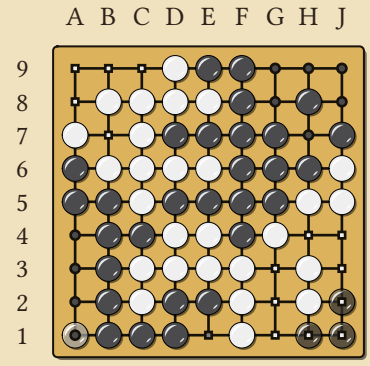



```
#gobanist-set(profile: "pgfgo")

#let final2 = board(final,
  prisoners(black: "H1, J1, J2",
    white: "A1"))

#goban(final2, label: (at: (top, left)))
```

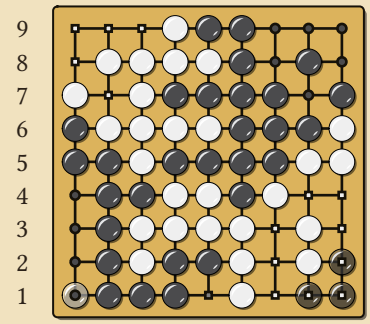
Ahora, el territorio. Una forma es marcar cada punto con `t-square` o `t-circle`. Estas marcas toman como relleno el de las piedras de su jugador, y como borde, el de la cuadrícula. Sobre un punto vacío, una marca es neutral salvo que diga su jugador, con `player`.



```
#gobanist-set(profile: "pgfgo")

#goban(final2, label: (at: (top, left)),
  marks(player: "black",
    t-circle: "A1, A2, A3, A4, G8, G9, H9,
      J9, J8, H7"),
  marks(player: "white",
    t-square: "A9, B9, C9, A8, B7, H4, J4,
      J3, J2, J1, H1, G1, G2, G3"),
  marks(t-square: "E1"))
```

Otra forma es `territory` (pág. 74). Con un polígono, marca los puntos de esa región que no tienen una piedra viva; con `auto`, busca solo el territorio de cada jugador. `goban-score` escribe el puntaje.



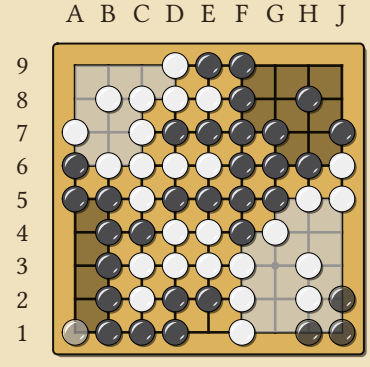
Negras: 11. Blancas: 23.5.

```
#gobanist-set(profile: "pgfgo")

#goban(final2, label: (at: (top, left)),
  territory(auto, dame: true))

Negras: #goban-score("black").
Blancas: #goban-score("white", komi: 6.5).
```

Para el artículo, Kāru prefiere que el territorio se vea como una mancha de color y no punto por punto. Con `draw: "region"`, el territorio de cada color es un solo polígono, cuyos lados van por las líneas del tablero.



```
#gobanist-set(profile: "pgfgo")

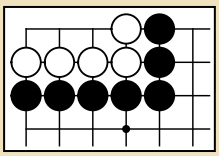
#goban(final2, label: (at: (top, left)),
  territory(auto, draw: "region"))
```

10 Un cuaderno de problemas

Para sus estudiantes, Kāru prepara un cuaderno de problemas de vida y muerte. Los problemas suelen guardarse también en SGF: la posición inicial, quién juega y un árbol con cada variante de la solución, correcta o no, con sus comentarios. `sgf-problem` dibuja todo eso: primero el enunciado y después un diagrama por variante. Si el archivo no dice qué parte del tablero mostrar, se muestra la que usa el problema, como con `partial: auto`.

```
#let problema = sgf-read(path("problema.sgf"))

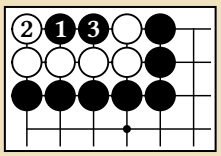
#sgf-problem(problema, title: "Problema 1")
```



Problema 1:

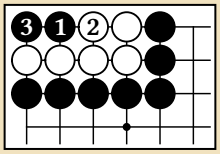
Juega Negro

Negras matan.



Variante 1

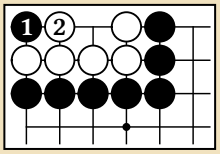
1: Tesuji. Correcto: el punto vital.



Variante 2

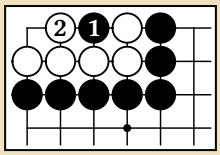
1: Tesuji. Correcto: el punto vital.

3: Blancas tienen un solo ojo.



Variante 3

1: Mala jugada. Incorrecto: Blancas viven.

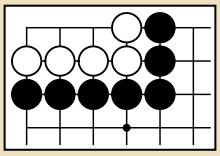


Variante 4

1: Incorrecto.

En un cuaderno, las soluciones no van junto al enunciado. Kāru pone los enunciados en la primera parte, con `solutions: false`, y las soluciones al final, con `statement: false`.

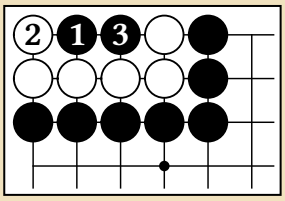
```
#sgf-problem(problema, title: "Problema 1",
  solutions: false)
// ... al final del cuaderno:
#sgf-problem(problema, statement: false,
  layout: (columns: 4, fit: true))
```



Problema 1:

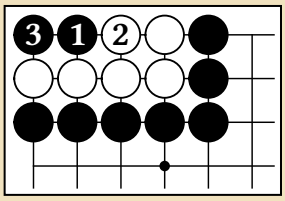
Juega Negro

Negras matan.



Variante 1

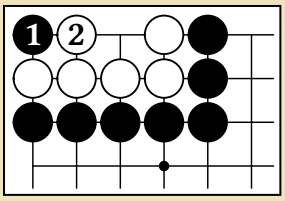
1: Tesuji. Correcto: el punto vital.



Variante 2

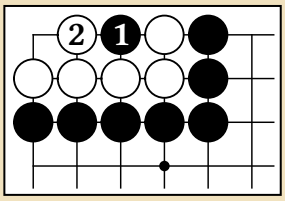
1: Tesuji. Correcto: el punto vital.

3: Blancas tienen un solo ojo.



Variante 3

1: Mala jugada. Incorrecto: Blancas viven.

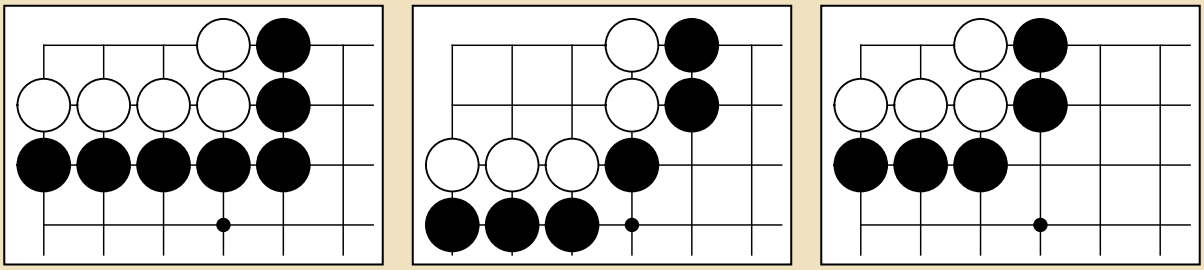


Variante 4

1: Incorrecto.

Algunos problemas no están en ningún archivo, y Kāru los dibuja a mano, con la misma parte del tablero que muestra `sgf-problem`. Aquí escribe la parte, `"A16:F19"`: con `partial: auto`, cada tablero mostraría solo la suya, y no coincidirían. Para que tengan el mismo tamaño que los demás, los pone junto con los de `sgf-problem` en un mismo `goban-layout`, que acomoda en una grilla todos los tableros que recibe.

```
#goban-layout(columns: 3, fit: true,
  sgf-problem(problema, title: "Problema 1",
    solutions: false),
  board(size: 19, partial: "A16:F19",
    stones(black: "A16, B16, C16, D17, E18, E19",
      white: "A17, B17, C17, D18, D19")),
  board(size: 19, partial: "A16:F19",
    stones(black: "A17, B17, C17, D18, D19",
      white: "A18, B18, C18, C19")))
```

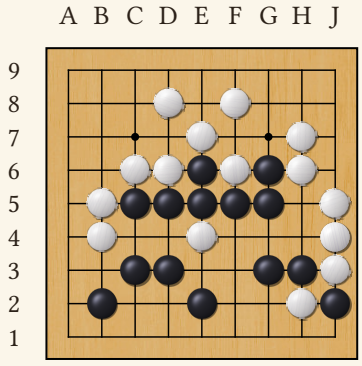


Problema 1: Juega Negro

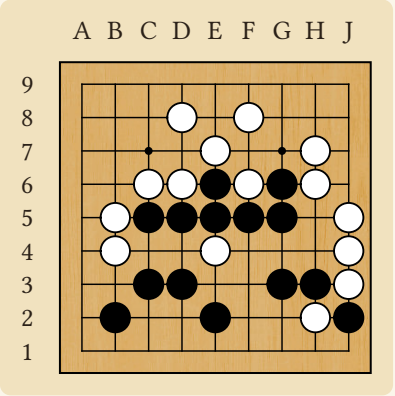
Negras matan.

11 Un tablero de madera: un perfil con imágenes

Para la portada del cuaderno, Kāru quiere un tablero que se parezca a uno de verdad, ya que el presupuesto no da para uno de kaya: madera, piedras con volumen y blancas con vetas. Los perfiles del catálogo dibujan las piedras como círculos; para esto hacen falta imágenes (PNG, JPEG, GIF, WebP, SVG o PDF). Kāru copia a su proyecto, en la carpeta `img`, las que usa el perfil `shell`, que están en la carpeta `images` del paquete: una foto de madera, `pgfgo-board.jpg`, una piedra negra de pizarra, `pgfgo-shell-black.png`, y cinco blancas de concha, `pgfgo-shell-white-1.png` a `pgfgo-shell-white-5.png`.



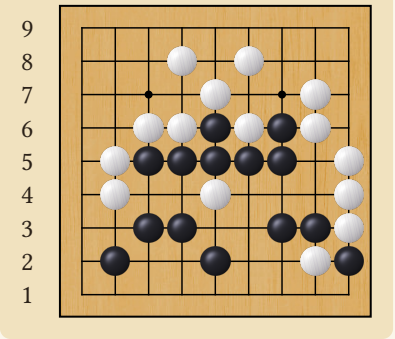
Parte del primer tablero del artículo. La madera va en el fondo del tablero, con `background.fill.image` (pág. 40). Una imagen es un archivo: un `path`, relativo al documento que lo nombra (o sus `bytes`). Por defecto, la imagen cubre todo el tablero, recortada si hace falta; con `fit`, puede estirarse o repetirse en baldosas.




```
#gobanist-set(background: (fill: (
  image: path("img/pgfgo-board.jpg"))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

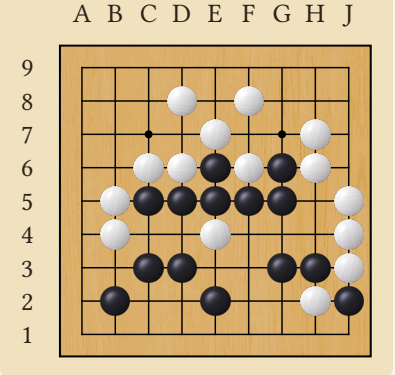
Las piedras de un jugador se dibujan con una imagen mediante `stone.image` (pág. 58), en las opciones de ese jugador. La imagen ocupa el lugar del círculo: no se dibujan el relleno ni el borde. En una rama de imagen, lo que no es un diccionario es el archivo.




```
#gobanist-set(
  background: (fill: (
    image: path("img/pgfgo-board.jpg"))),
  players: (
    black: (stone: (image:
      path("img/pgfgo-shell-black.png"))),
    white: (stone: (image:
      path("img/pgfgo-shell-white-1.png")))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

Las negras se ven bien, pero las blancas son todas iguales, y en un tablero de verdad cada piedra de concha tiene sus vetas. Con varios archivos en `file`, cada piedra usa uno, elegido por su lugar: una piedra en un punto muestra siempre la misma imagen, en todos los diagramas.

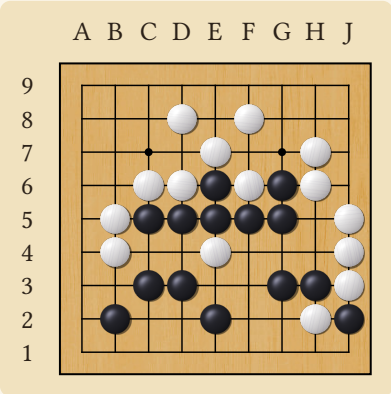



```
#let conchas = range(1, 6).map(n => path(
  "img/pgfgo-shell-white-" + str(n) + ".png"))

#gobanist-set(
  background: (fill: (
    image: path("img/pgfgo-board.jpg"))),
  players: (
    black: (stone: (image:
      path("img/pgfgo-shell-black.png"))),
    white: (stone: (image: conchas))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

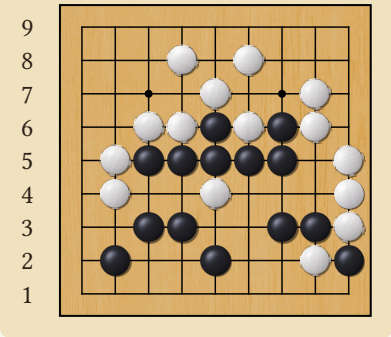
Para que las piedras parezcan apoyadas sobre la madera, falta una sombra. `shadow` la dibuja con la forma de la imagen (si la imagen tiene transparencia, como estos PNG). Como es la misma para los tres jugadores, va en `stone`, en el primer nivel: es `players: (all: (stone: ...))`.




```
#gobanist-set(
  background: (fill: (
    image: path("img/pgfgo-board.jpg"))),
  stone: (shadow: (on: true, offset: 0.75pt,
    angle: -45deg)),
  players: (
    black: (stone: (image:
      path("img/pgfgo-shell-black.png"))),
    white: (stone: (image: conchas))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

Con cinco imágenes, todavía puede verse alguna repetida. `rotate` gira cada piedra un ángulo distinto, elegido también por su lugar. Estas piedras tienen la luz arriba a la izquierda, así que Kāru las gira poco, entre −15 y 15 grados; la sombra gira con la imagen, pero sigue cayendo hacia el mismo lado.




```
#gobanist-set(
  background: (fill: (
    image: path("img/pgfgo-board.jpg"))),
  stone: (shadow: (on: true, offset: 0.75pt,
    angle: -45deg)),
  players: (
    black: (stone: (image:
      path("img/pgfgo-shell-black.png"))),
    white: (stone: (image: (file: conchas,
      rotate: (-15deg, 15deg))))))

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C5, D5, E5, F5, G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, J5, B4, E4, J4, J3, H2"))
```

Kāru va a usar este estilo en todo el cuaderno, y quizás en otros documentos. En lugar de repetir el `gobanist-set`, lo guarda como un perfil (pág. 108): un perfil es un diccionario de opciones, como las de `gobanist-set`, y se usa con la opción `profile`. Lo pone en un archivo propio, `madera.typ`, para importarlo desde cualquier documento.

```
// madera.typ
#let conchas = range(1, 6).map(n => path("img/pgfgo-shell-white-" + str(n) + ".png"))
#let madera = (
  background: (fill: (image: path("img/pgfgo-board.jpg"))),
  stone: (shadow: (on: true, offset: 0.75pt, angle: -45deg)),
  players: (
    black: (stone: (image: path("img/pgfgo-shell-black.png"))),
    white: (stone: (image: (file: conchas, rotate: (-15deg, 15deg)))),
  ),
)

// El cuaderno:
#import "madera.typ": madera
#gobanist-set(profile: madera)
```

Antes de poner el suyo, `profile` vuelve el estilo al inicial, así que el perfil se ve igual en cualquier documento. Un perfil casi igual, con una sombra difusa y sin el borde del tablero, viene con gobanist: `shell` (pág. 109); con `profile: "shell"` no hace falta copiar nada. También está `yunzi`, con piedras mates.

Un perfil vale para todo lo que dibuja gobanist, también para los diagramas de un SGF. Así queda la partida de Lee Sedol y AlphaGo en madera, con un diagrama cada 50 jugadas y la numeración reiniciada en cada uno.

```
#gobanist-set(profile: madera)

#sgf-diagrams(lee, step: 50, restart: true, layout: (columns: 2, fit: true))
```

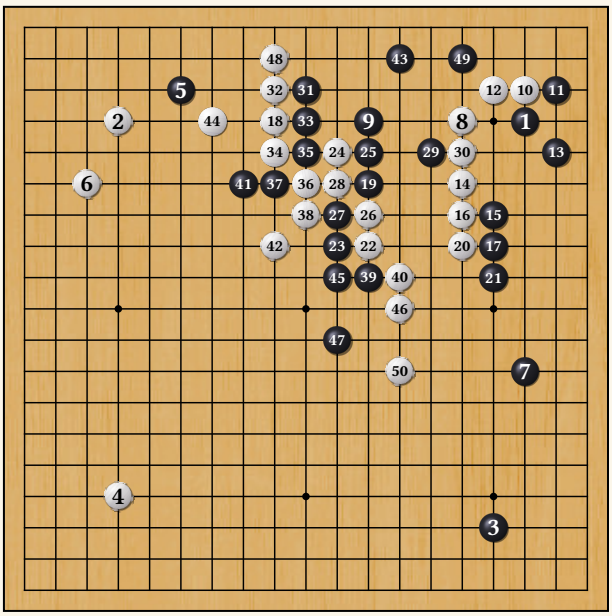


Diagrama 1 (1–50)

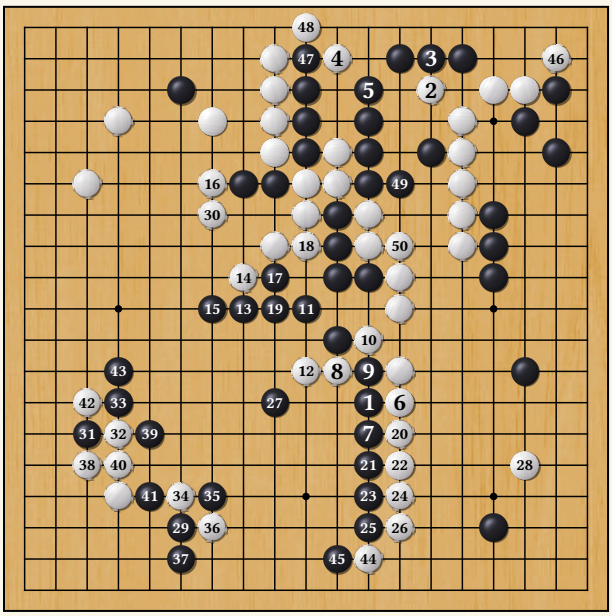


Diagrama 2 (51–100)

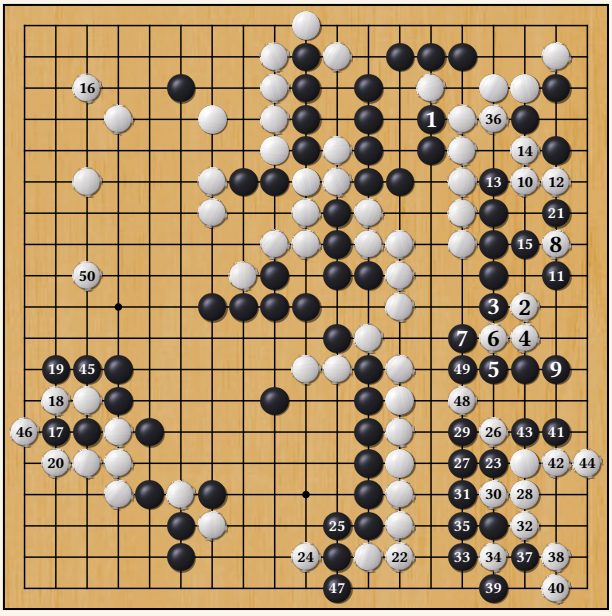


Diagrama 3 (101–150)

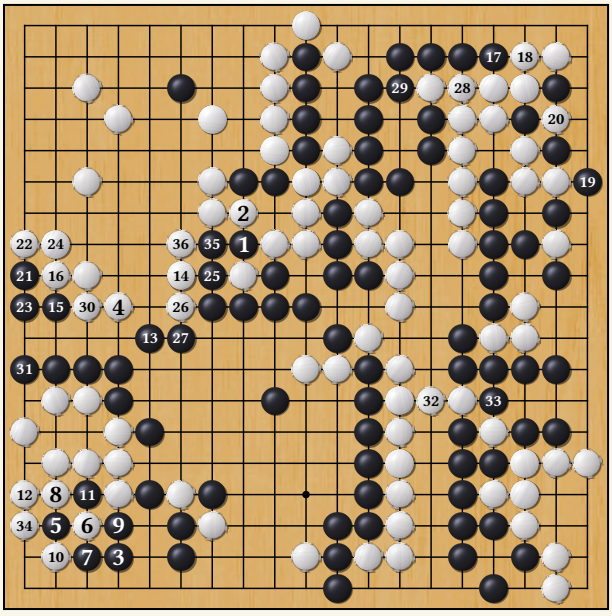


Diagrama 4 (151–186)
Resultado: Gana Blanco por abandono

12 Diapositivas

Kāru también da clases con diapositivas. Typst no hace PDF animados, pero un `goban` funciona en una diapositiva de Touying o de Polylux como en cualquier otro lugar, y `sgf` con `to` permite mostrar la partida jugada por jugada, una diapositiva por jugada, con su comentario al lado.

```
#import "@preview/touying:0.6.1": *
#import themes.simple: *
#import "@preview/gobanist:0.2.0": *
```

```
#show: simple-theme
#let partida = sgf-read(path("partida.sgf"))

#for k in range(1, 11) [
  == Una partida corta
  #grid(columns: (1fr, 1fr), gutter: 1em,
    goban(captures: "remove", scale: 1.5, sgf(partida, to: k)),
    text(size: 0.8em, sgf-comment(partida, k)))
]
```

Para repasar la partida entera, en cambio, le alcanzan los diagramas de Lee Sedol y AlphaGo del artículo (pág. 16), uno por diapositiva. Con `layout: (per-page: 1)`, `sgf-diagrams` pone cada diagrama en su propia página, con su epígrafe (pág. 103).

```
#sgf-diagrams(lee, step: 50, goban: (scale: 0.6), layout: (per-page: 1))
```

Las diapositivas son otro documento, pero Kāru no quiere copiar los tableros del artículo. Los tableros son valores: los guarda en un módulo, `tableros.typ`, y los importa en el artículo y en las diapositivas, donde los dibuja, los muestra con `goban-view` o sigue desde ellos con `resume`.

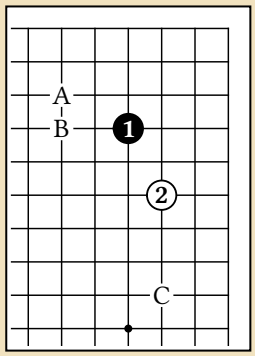
```
// tableros.typ
#import "@preview/gobanist:0.2.0": *
#let pelea = board(stones(black: "C3, D3, E4, C5", white: "D4, D5, E5, F3"),
  marks(triangle: "D5"), marks(custom: "E3", content: [A]))

// En las diapositivas:
#import "tableros.typ": pelea
== Si negras conectan
#goban(pelea, partial: auto, moves("E3"))
```

13 Un diccionario de josekis

Para otra clase, Kāru quiere explicar las respuestas a la aproximación baja al punto 4-4. Las secuencias conocidas de la apertura en una esquina, los *josekis*, suelen guardarse en un SGF con forma de diccionario: un árbol con una rama por cada jugada posible, comentarios en las que importan y letras en las jugadas que pueden seguir. Kāru arma uno pequeño, `josekis.sgf`.

Primero, la posición: las dos primeras jugadas, con las letras de las respuestas. El tablero es de 19×19, pero todo pasa en una esquina; con `partial: auto` (pág. 11), el `goban` muestra solo la parte que usa lo que dibuja, también las letras.




```
#let josekis = sgf-read(path("josekis.sgf"))

#goban(partial: auto, sgf(josekis, to: 2))
```

Para las respuestas, Kāru elige la línea con `variation`. En un diccionario hay tantas ramas que es más cómodo dar las jugadas de la línea, desde la primera, que buscar su número: `variation: "Q16, R14"` (pág. 85). Con `variations`, `sgf-diagrams` dibuja además, cada una en su diagrama, las variantes con comentarios que se apartan de esa línea, y con `from: 3`, solo las que nacen en la jugada 3 o después: las respuestas, y no la

invasión en el 3-3, que es otra jugada 2. Con `partial: auto`, todos los diagramas muestran la misma parte del tablero.

```
#sgf-diagrams(josekis, variation: "Q16, R14",
  variations: (on: true, from: 3),
  partial: auto, comments: true,
  layout: (columns: 3, fit: true))
```

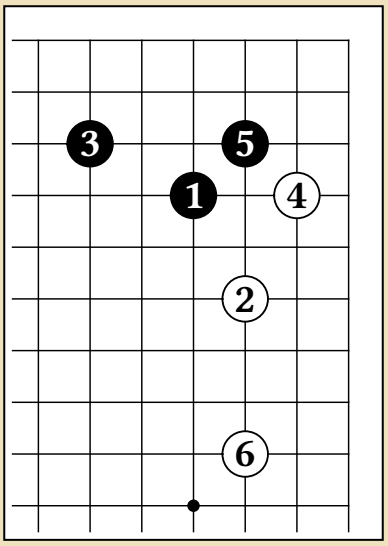


Diagrama 1 (1–6)

2: La aproximación baja.
3: A: el paso de caballo.
4: Blancas se deslizan hacia la esquina.
6: Y se extienden por el lado.

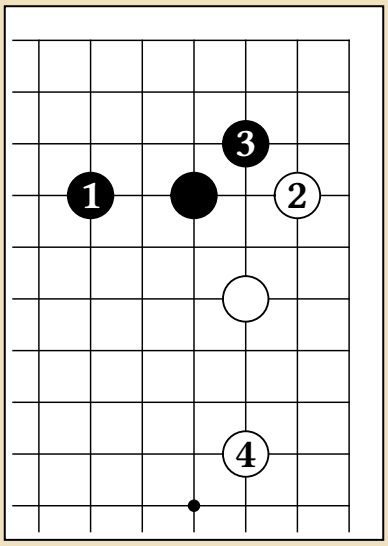


Diagrama 1a (variante en 3)

1: B: el salto de uno.

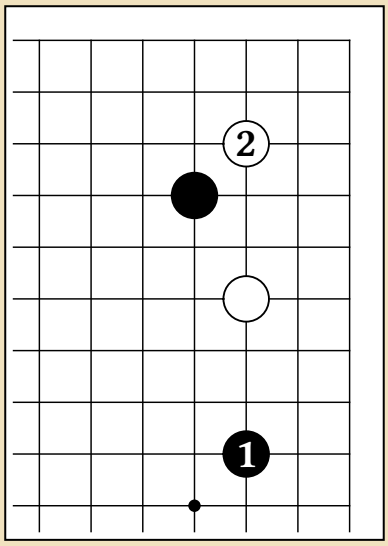


Diagrama 1b (variante en 3)

1: C: la pinza baja.
2: Blancas entran en el 3-3.

El primer diagrama es la línea elegida, hasta el final. Las jugadas de cada variante se numeran desde 1, y el epígrafe dice en qué jugada se aparta. En un diccionario grande, cada jugada trae letras en las que pueden seguir; con `marks: false`, los diagramas no las dibujan.

14 Otros idiomas

Un colega de Kāru quiere traducir el cuaderno al francés. Los textos que escribe gobanist (los epígrafes, el resultado, las notas «N en M» y las anotaciones de los comentarios) siguen el idioma del texto, `text.lang`, también si cambia dentro del documento: con `set text(lang: "fr")` se traducen solos (pág. 31).

```
#set text(lang: "fr")

#sgf-diagrams(partida, step: 5,
  layout: (columns: 2))
#sgf-problem(problema, solutions: false)
```

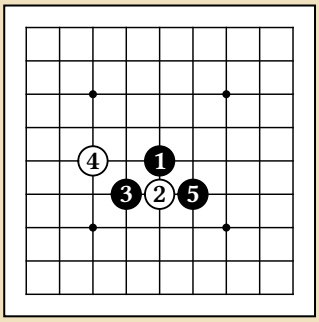


Diagramme 1 (1–5)

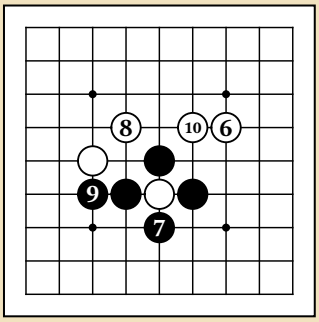
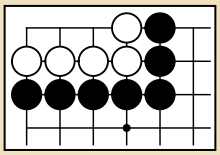


Diagramme 2 (6–10)



Noir joue

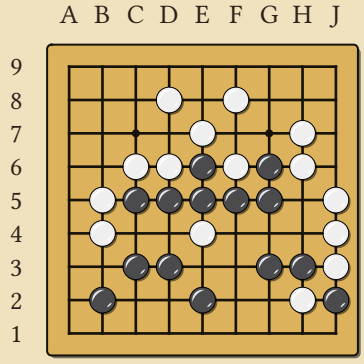
Negras matan.

El comentario del problema, «Negras matan.», no se traduce: está escrito así en el archivo. Cada texto del paquete puede cambiarse, además, con su propia opción, en cualquier idioma: por ejemplo, la palabra del epígrafe de los diagramas, con `diagrams.title`; o los textos de un idioma, con `texts`.

```
#gobanist-set(diagrams: (title: "Figura"))
#gobanist-set(texts: (es: (diagram: "Figura")))
```

15 Escribir menos: la sintaxis de coordenadas

En todos los tutoriales, Kāru escribió las piedras una por una. Cuando son muchas, gobanist permite abreviar las listas de `stones`, `marks` y `prisoners` (pág. 33). Las formas más útiles son los tramos de una fila o de una columna: `C:G5` es la fila 5, de C a G, y `J3:5` es la columna J, de 3 a 5. Con ellas, el primer tablero del artículo queda así:

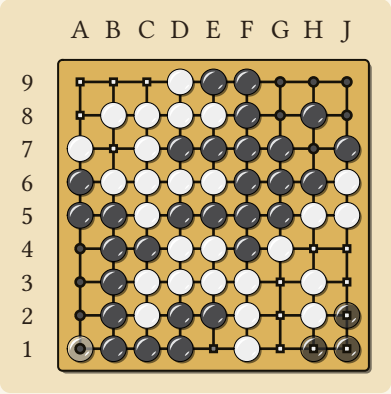



```
#gobanist-set(profile: "pgfgo")

#goban(label: (at: (top, left)),
  stones(
    black: "B2, E2, J2, C3, D3, G3, H3,
      C:G5, E6, G6",
    white: "D8, F8, E7, H7, C6, D6, F6,
      H6, B5, B4, E4, H2, J3:5"))
```

Para grupos más grandes, hay caminos que doblan en ángulo recto. `A6 |- B5` va de A6 hacia abajo hasta la fila 5 y después hacia la derecha hasta B5: son las piedras A6, A5 y B5. `E9 -| F4` va primero hacia la derecha

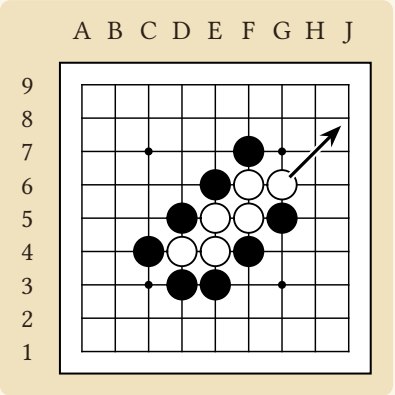
y después hacia abajo. `D5 -- D7` es una línea recta. Los caminos pueden encadenarse, y con ellos la posición final del tutorial de territorio (pág. 19) cabe en dos líneas:




```
#gobanist-set(profile: "pgfgo")

#goban(label: (at: (top, left)),
  stones(
    black: "A6|-B5|-D1|-E2, C4, E9-|F4,
      D5-|G7--D7, H6, H8, J7",
    white: "E8-|C5, B:E6, A7, B8, D9, J6|-H5,
      H2:3, F1|-C3, C2, D:E4, G4"),
  prisoners(black: "H1-|J2", white: "A1"),
  territory(auto, dame: true))
```

Entre dos intersecciones que no están en la misma fila ni en la misma columna, `--` es una línea inclinada: `D5 -- F7` son D5, E6 y F7. Esas piedras se tocan solo por las esquinas; para una cadena conectada, `-|-` rellena cada escalón, primero hacia el costado y después hacia arriba o hacia abajo: `D4 -|- F6` son D4, E4, E5, F5 y F6 (`|-|` empieza por la columna). Con eso, una escalera (*shicho*) se escribe con sus extremos:




```
#goban(label: (at: (top, left)),
  stones(white: "D4 -|- F6 -- G6",
    black: "C4, D3, D5 -- F7, E3 -- G5"),
  // Blancas corren. No por mucho.
  arrows(("G6", "J8"), shorten: 4pt))
```

Hay más: rectángulos llenos (`A1:C3`) o solo su borde (`A1 r C3`), columnas y filas enteras (`D`, `4`), todo el tablero (`*`) y, después de un `;`, piedras que se quitan de la lista. Todo está en la referencia, en la pág. 33.

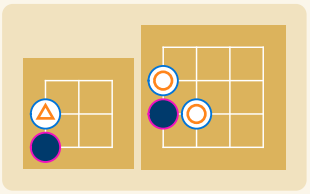
3 Configuración

1 Las opciones

Las opciones de gobanist forman un árbol de diccionarios: cada objeto (el fondo, la cuadrícula, las piedras de cada jugador, los diagramas de una partida…) es una rama, y sus opciones son sus claves. Las de un tablero van en el `goban`, con nombre, junto a lo que se dibuja; las de todo el documento, en `gobanist-set`.

```
gobanist-set(..opciones)
```

Fija opciones para todo lo que sigue en el documento: los tableros, las vistas, los diagramas y los problemas. Recibe las mismas opciones que `goban`, y además las ramas de las partidas (`sgf`, `diagrams`, `problems`), la disposición (`layout`), las vistas (`view`), los textos (`texts`) y las formas de marca (`shapes`). Cada `gobanist-set` se suma a los anteriores. Como `set` de Typst, vale desde donde está hasta el final del bloque que lo contiene.



```
#gobanist-set(
  background: (
    fill: (color: rgb("#dcb35c")),
    line: false),
  grid: (line: (color: white)),
  players: (
    black: (stone: (
      line: (color: fuchsia),
      fill: (color: blue.darken(50%)))),
    white: (
      stone: (line: (color: blue)),
      mark: (line: (color: orange)))))

#goban(size: 3,
  stones(black: "A1"),
  stones(white: "A2", mark: "triangle"))
#goban(size: 4,
  stones(black: "A2"),
  stones(white: "B2, A3", mark: "circle"))
```

Los `goban` que siguen heredan estas opciones, y cada uno puede cambiarlas con las suyas: `goban(size: 9, stone: (radius: 4pt), mark: (line: (color: red)), ...)`. Las opciones se resuelven al dibujar el tablero, con las de `gobanist-set` que hay en ese lugar del documento.

Las opciones se mezclan rama por rama: un diccionario cambia solo las claves que da, y las demás quedan como estaban. `gobanist-set(background: (fill: (color: red)))` cambia el color del relleno, y el borde, la opacidad y la imagen siguen igual. Una clave que no existe es un error. Una longitud sobre una rama de longitudes (`grid.sep`, `background.rounded-corners`) va a todas sus claves: `grid: (sep: 1em)` es `grid: (sep: (x: 1em, y: 1em))`.

Prender y apagar

Muchas ramas prenden o apagan algo: la cuadrícula, el fondo, el borde y el relleno de las piedras, sus sombras y sus brillos, etc. Todas tienen la clave `on`, y siguen la misma regla:

- `true` prende la rama: `shine: true`;
- `false` la apaga: `shine: false`;
- un diccionario cambia sus opciones sin prenderla: `shine: (angle: 45deg)` cambia el ángulo, pero el brillo sigue como estaba. Para prenderlo y cambiarlo a la vez, `shine: (on: true, angle: 45deg)`.

Así, `gobanist-set(stone: (shadow: (opacity: 0.3)))` fija la opacidad de las sombras de todo el documento sin prenderlas, y cada tablero que las quiera las prende con `stone: (shadow: true)`.

Las ramas de imagen (`stone.image`, `background.fill.image`, …) no tienen `on`: las prende el archivo. En ellas, lo que no es un diccionario es el archivo: `stone: (image: path("piedra.png"))` es `stone: (image: (file: path("piedra.png")))`, y `image: none` la quita.

Atajos de los jugadores

Las opciones de las piedras, los prisioneros, las jugadas y las marcas son de cada jugador (pág. 107). En el primer nivel, valen para los tres a la vez:

`stone` opciones de piedra

Es `players: (all: (stone: ...))`: el estilo de las piedras de los tres jugadores (pág. 54).

`prisoner` opciones de piedra

Es `players: (all: (prisoner: ...))`: el estilo de los prisioneros de los tres jugadores (pág. 62).

`move` opciones de jugada

Es `players: (all: (move: ...))`: el estilo de las jugadas de los tres jugadores (pág. 60).

`mark` opciones de marca

Es `players: (all: (mark: ...))`: el estilo de las marcas de los tres jugadores (pág. 64).

`players` `neutral`, `black`, `white` y `all`

Las opciones de cada jugador (pág. 107).

Las demás ramas del primer nivel están en el capítulo de cada objeto: el tablero (`size`, `partial`, `background`, `grid`, `hoshi`, `label`…, pág. 40), las flechas y las líneas (`arrow`, `line`, pág. 69), el juego (`captures`, `notes`, `territory`, `score`, pág. 72), las partidas (`sgf`, `diagrams`, `problems`, pág. 85), la disposición (`layout`, pág. 103), las vistas (`view`, pág. 81), los perfiles (`profile`, pág. 108), los textos (`texts`, pág. 31) y las formas de marca (`shapes`, pág. 68).

2 El kernel

gobanist separa el cálculo del dibujo. Las coordenadas, la numeración de las jugadas y el dibujo están escritos en Typst. Lo que necesita jugar la partida lo hace un *kernel*: un plugin de WebAssembly, `src/kernel.wasm`, compilado del código Rust de la carpeta `kernel/` del repositorio. El kernel:

- juega las jugadas y retira las piedras capturadas (`captures`, pág. 72);
- busca el territorio (`territory(auto)`), arma las regiones (`draw: "region"`) y cuenta el puntaje (pág. 74);
- lee los archivos SGF y sus codificaciones (pág. 85);
- hace la opacidad y las siluetas de las imágenes PNG y JPEG (pág. 58).

Juega cada tablero en una sola llamada, cuando el tablero tiene capturas o territorio. Viene compilado con el paquete: no hace falta instalar nada.

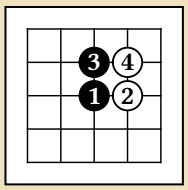
3 Idiomas

Algunos textos los escribe el propio paquete: los epígrafes de `sgf-diagrams` y `sgf-problem`, el resultado de la partida, las anotaciones de los comentarios, las palabras de las notas «N en M» y «N: pasa» y la «y» entre varias vistas. Están en inglés (`en`), español (`es`), francés (`fr`), alemán (`de`), portugués (`pt`), italiano (`it`), esperanto (`eo`), japonés (`ja`), chino (`zh`) y coreano (`ko`). Para el japonés, el chino y el coreano hace falta una fuente con esos caracteres.

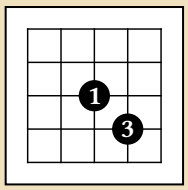
Los textos están en el idioma del texto, `text.lang`, y lo siguen si cambia dentro del documento: `set text(lang: "fr")`. En un idioma sin traducción, están en inglés.

`texts` un diccionario de idiomas

Los textos de cada idioma: `texts: (es: (diagram: "Figura"))`. Un idioma nuevo toma del inglés lo que no tiene. Los textos que tienen partes variables son funciones: `n` es el número de una jugada, `w` el nombre del ganador y `p` la diferencia de puntos.



5 sobre 1



2: transit

```
#gobanist-set(texts: (
  es: (at: "sobre"),
  la: (pass: "transit"),
))

#goban(size: 5, notes: true,
  moves("C3, D3, C4, D4"),
  moves("C3", from: 5))
#goban-notes()

#set text(lang: "la")
#goban(notes: true, sgf-read(
  "(;SZ[5];B[cc];W[];B[dd])"))
#goban-notes()
```

Las claves de cada idioma, con su valor en inglés y en español, son:

Clave	Inglés	Español
`tesuji`	Tesuji	Tesuji
`very-good-tesuji`	Very good tesuji	Muy buen tesuji
`bad-move`	Bad move	Mala jugada
`very-bad-move`	Very bad move	Muy mala jugada
`doubtful`	Doubtful move	Jugada dudosa
`interesting`	Interesting move	Jugada interesante
`good-for-black`	Good for Black	Bueno para Negro
`very-good-for-black`	Very good for Black	Muy bueno para Negro
`good-for-white`	Good for White	Bueno para Blanco
`very-good-for-white`	Very good for White	Muy bueno para Blanco
`even`	Even position	Posición pareja
`unclear`	Unclear position	Posición incierta
`diagram`	Diagram	Diagrama
`variation`	Variation	Variante
`problem`	Problem	Problema
`correct`	correct	correcta
`wrong`	wrong	incorrecta
`variation-at`	variation at 26	variante en 26
`black-to-play`	Black to play	Juega Negro
`white-to-play`	White to play	Juega Blanco
`at`	at	en
`pass`	pass	pasa
`and`	and	y
`result`	Result	Resultado
`black`	Black	Negro
`white`	White	Blanco
`win`	*w* wins	Gana *w*
`win-by-points`	*w* wins by *p* points	Gana *w* por *p* puntos
`win-by-resignation`	*w* wins by resignation	Gana *w* por abandono
`win-on-time`	*w* wins on time	Gana *w* por tiempo
`win-by-forfeit`	*w* wins by forfeit	Gana *w* por incomparecencia
`draw`	Draw (jigo)	Empate (jigo)
`void`	No result	Sin resultado
`unknown-result`	Unknown result	Resultado desconocido

4 Sintaxis de coordenadas

Las intersecciones se escriben como en el go: una letra para la columna, desde la izquierda, y un número para la fila, desde abajo. `"C3"` es la tercera columna y la tercera fila. Las letras van en mayúscula y sin la I, salvo con `use-i` (pág. 41). Después de la Z, en los tableros de más de 25 columnas, las columnas tienen dos letras, ya con la I: `AA`, `AB`, …, `AI`, …, `AZ`, `BA` (`"AA5"`, `"X2:AB2"`).

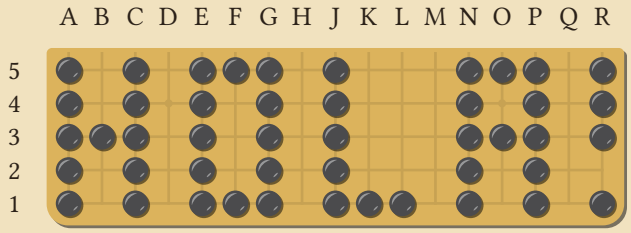
`stones`, `prisoners` y `marks` reciben, para cada jugador o cada forma, una cadena con una o muchas intersecciones, escritas con una sintaxis que permite abreviar:

- varias entradas separadas por comas;
- filas, columnas, tramos, rectángulos y caminos;
- intersecciones que se quitan de la lista, después de un `;`.

La misma sintaxis sirve para `moves` (sin el `;`), para los vértices de `territory` y para `hoshi.at`. Las flechas y las líneas, y `partial`, reciben intersecciones sueltas.

```
#gobanist-set(profile: "pgfgosmooth")

#goban(size: 17, partial: "A1:R5", label: (at: (top, left)),
  stones(black: "*; B, D, H, M, R2, Q, F2:4, K2 r L5, O[1:2, 4]; B3"))
```



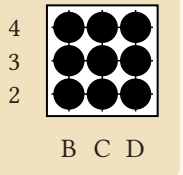
Una coordenada que no se entiende, o que está fuera del tablero, es un error. Los espacios separan los elementos, pero no pueden ir dentro de una intersección: `"B3"` es B3, y `"B 3"` es un error.

1 Formas sueltas

Estas formas van solas: no se encadenan con los operadores de la pág. 35, y se separan de las demás con comas.

Todo el tablero

`*` son todas las intersecciones **visibles** del tablero: con `partial`, las de la parte que se ve.

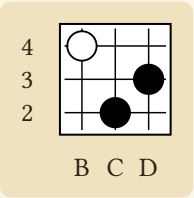



```
#gobanist-set(size: 5, partial: "B2:D4")

#goban(label: (at: (bottom, left)),
  stones(black: "*"))
```

Intersección

Una letra y un número, juntos: `"C2"`.

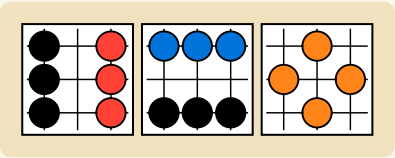



```
#gobanist-set(size: 5, partial: "B2:D4")

#goban(label: (at: (bottom, left)),
  stones(black: "C2, D3", white: "B4"))
```

Columnas y filas enteras

Una letra sola es una columna entera: `D` es la columna D. Un número solo es una fila entera: `4` es la fila 4.

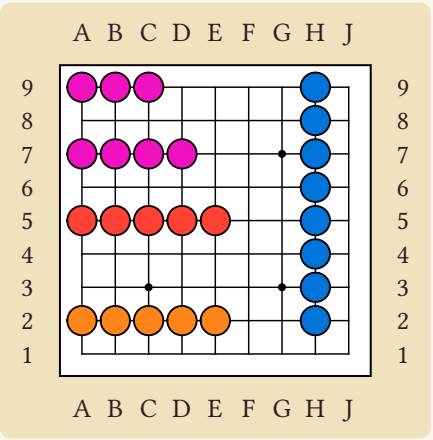


```
#gobanist-set(size: 5, partial: "B2:D4")

#goban(stones(black: "B2, B3, B4"),
  stones("D", fill: (color: red)))
#goban(stones(black: "B2, C2, D2"),
  stones("4", fill: (color: blue)))
// Las líneas C y 3, menos C3.
#goban(stones("C, 3; C3",
  fill: (color: orange)))
```

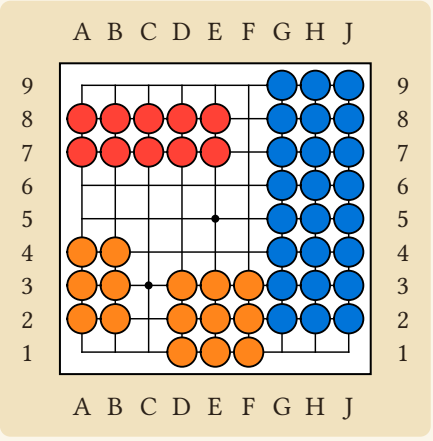
Tramos

Un tramo es una parte de una fila o de una columna: `C:D2` es la fila 2, de C a D; `B2:4` es la columna B, de 2 a 4. Entre la letra y el número puede ir un espacio: `C:D 2`, `B 2:4`. Los tramos van del primer extremo al segundo: `H9:2` baja de H9 a H2, y `E:A2` va de E2 a A2.




```
#goban(label: (at: (top, bottom, left, right)),
  stones("A:E5", fill: (color: red)),
  stones("H9:2", fill: (color: blue)),
  stones("E:A2", fill: (color: orange)),
  stones("A:D7, A:C9", fill: (color: fuchsia)))
```

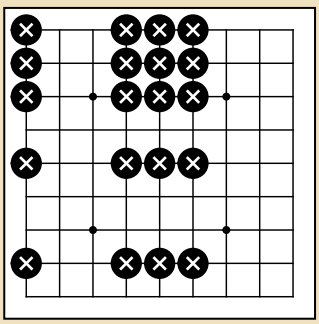
Con dos rangos, un tramo es un rectángulo: `A:E 7:8` son las columnas A a E de las filas 7 y 8. Los operadores de la sección siguiente lo escriben más claro: `A7:E8`.




```
#goban(label: (at: (top, bottom, left, right)),
  stones("A:E 7:8", fill: (color: red)),
  stones("J:G 9:2", fill: (color: blue)),
  stones("A:B 2:4, F:D 1:3",
    fill: (color: orange)))
```

- **Productos**

Una letra seguida de una lista entre corchetes se reparte en cada elemento de la lista: `A[1:3, 5, 7:8]` es `A1:3, A5, A7:8`. Los elementos no tienen por qué ser números: `A[:D 5:7, 1:2]` es `A:D 5:7, A1:2`. Del mismo modo, una lista entre corchetes seguida de un número: `[A, C, E:G]12` es `A12, C12, E:G12`. Y dos listas, una tras otra, se combinan todas con todas: `[A, D:F][2, 5, 7:9]` es `A2, A5, A7:9, D:F2, D:F5, D:F 7:9`. Los corchetes no se anidan.



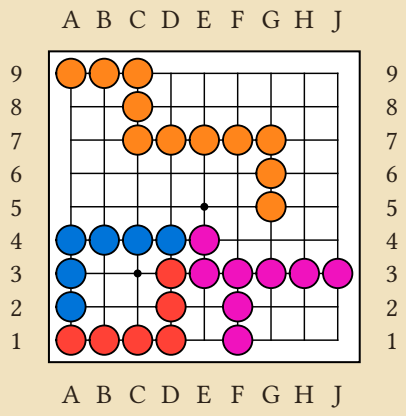
```
#goban(stones(black: "[A, D:F][2, 5, 7:9]"),
  marks(cross: "A2, A5, A7:9, D:F2, D:F5,
    D:F 7:9", player: "black"))
```

2 Caminos

Los operadores unen dos intersecciones con un camino, y se encadenan.

- **En ángulo recto**

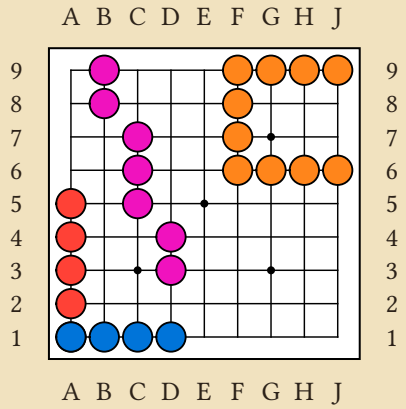
`A1 -| D3` va de A1 hacia el costado hasta la columna D, y después hacia arriba hasta D3. `A1 |- D3` va primero por la columna y después por la fila. Encadenados, forman escaleras o recorridos con la métrica de Manhattan.




```
#goban(label: (at: (top, bottom, left, right)),
  stones("A1 -| D3", fill: (color: red)),
  stones("D4 -| A2", fill: (color: blue)),
  stones("A9 -| C7 -| G5", fill: (color: orange)),
  stones("F1 |- J3 -| E4", fill: (color: fuchsia)))
```

- **En cualquier dirección**

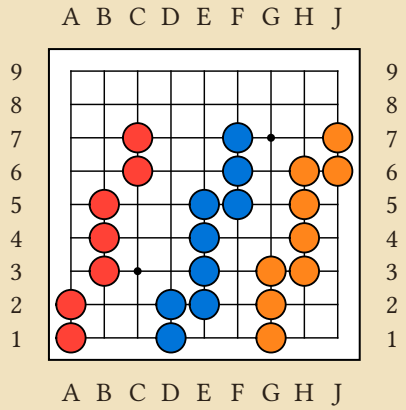
`A1 -- C5` es la línea de piedras más parecida a la recta que une las dos intersecciones: una piedra por fila (o por columna, si la línea avanza más de lo que sube o baja), la más cercana a la recta. Si las dos intersecciones están en una misma fila o columna, es la línea recta entre ellas. Cuando la recta pasa justo entre dos intersecciones, la piedra va del lado del extremo más cercano, y en el medio exacto, a la columna (o la fila) menor: así, `A1 -- C5` y `C5 -- A1` dan las mismas piedras.



```
#goban(label: (at: (top, bottom, left, right)),
  stones("A1 -- A5", fill: (color: red)),
  stones("A1 -- D1", fill: (color: blue)),
  stones("B9 -- D3", fill: (color: fuchsia)),
  stones("J9 -| F6 -- J6", fill: (color: orange)))
```

- **Escaleras**

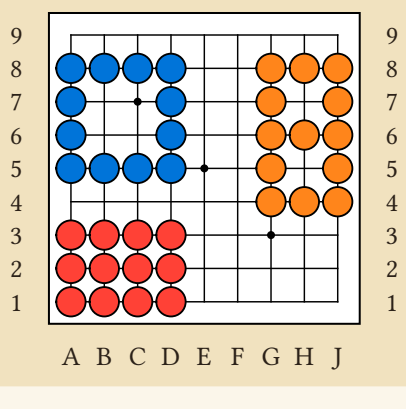
Las piedras de una línea inclinada se tocan solo por las esquinas, así que en go no forman un grupo. `D1 -|- F7` y `G1 |-| J7` rellenan cada escalón con la esquina de `-|` (primero por la fila) o de `|-` (primero por la columna): el resultado es una escalera conectada. Si las dos intersecciones están en una misma fila o columna, dan la misma línea que `--`. En las escaleras, como en `-|` y `|-`, el orden de los extremos importa.



```
#goban(label: (at: (top, bottom, left, right)),
  stones("A1 -- C7", fill: (color: red)),
  stones("D1 -|- F7", fill: (color: blue)),
  stones("G1 |-| J7", fill: (color: orange)))
```

- **Rectángulos**

`A1:D3` rellena el rectángulo que tiene esas dos esquinas opuestas (es la escritura de `partial`); `A5 r D8` es solo su contorno. Un rectángulo de una sola columna o de una sola fila es un tramo: `A5:A9` es `A5:9`.

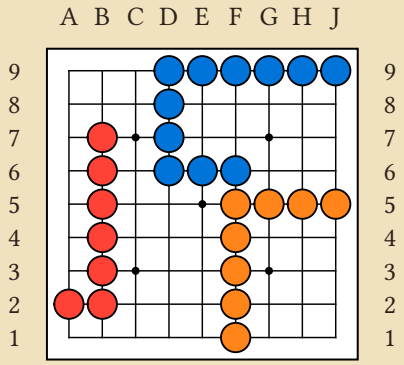


```
#goban(label: (at: (top, bottom, left, right)),
  stones("A1:D3", fill: (color: red)),
  stones("A5 r D8", fill: (color: blue)),
  // ¡Está vivo!
  stones("J8 r G4, H6", fill: (color: orange)))
```

- **Encadenar**

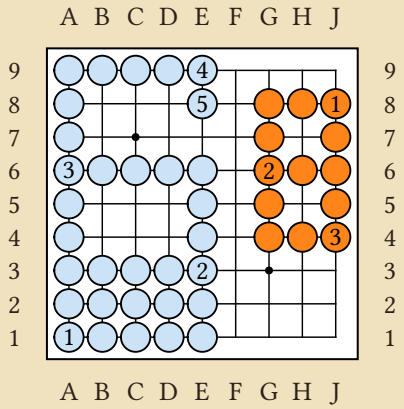
Los operadores (`-|`, `|-`, `--`, `-|-`, `|-|` y `r`) se encadenan: `A1 -| C3 -- E3` es `A1 -| C3` seguido de `C3 -- E3`, y C3 va una sola vez. Entre dos operadores puede ir una intersección, un tramo (`B5:7`, `A:B3`) o un

rectángulo (`A1:C3`). Cada operador va del último punto de lo anterior al primero de lo siguiente: `A2 -| B5:7` es `A2 -| B5` seguido de `B5:7`, y `B7 -| A:B3` es `B7 -| A3 -- B3`.



```
#goban(label: (at: (top, bottom, left, right)),
  stones("A2 -| B5:7", fill: (color: red)),
  stones("J9 -| D:F6", fill: (color: blue)),
  stones("F1:3 |- J5", fill: (color: orange)))
```

Los rectángulos también se encadenan.

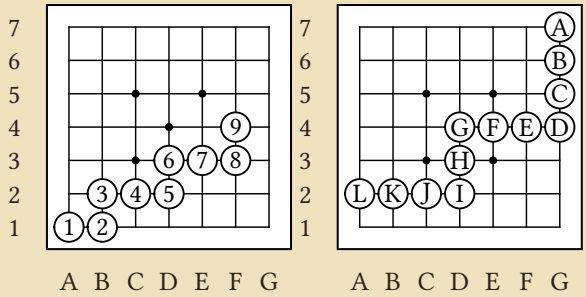


```
#goban(label: (at: (top, bottom, left, right)),
  stones("J8 r G6 r J4", fill: (color: orange)),
  marks(sequence: "J8, G6, J4"),
  stones("A1:E3 r A6 |- E9 -- E8",
    fill: (color: blue.lighten(80%))),
  marks(sequence: "A1, E3, A6, E9, E8"))
```

Las formas sueltas (`*`, una columna o una fila entera, un tramo con dos rangos) no se encadenan.

3 El orden de las intersecciones

Las intersecciones salen en el orden en que se escriben: los tramos van del primer extremo al segundo (`N10:7` baja de N10 a N7), los operadores siguen su camino y los corchetes se reparten en su lugar (`D4, B[3:1]` es `D4, B3:1`). Ese es el orden que numeran `moves` y la marca `sequence`.



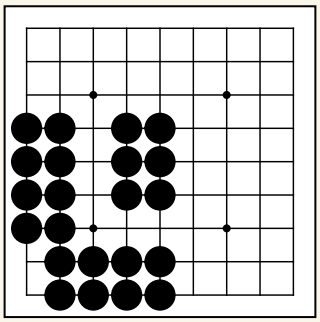
```
#goban(size: 7, label: (at: (bottom, left)),
  stones(white: "A1 -|- F4"),
  marks(sequence: "A1 -|- F4"))
#goban(size: 7, label: (at: (bottom, left)),
  stones(white: "G7:4 -| D2 -- A2"),
  marks(sequence: "G7:4 -| D2 -- A2",
    label: (format: "A")))
```

Algunas formas no tienen un recorrido fijo: los rectángulos (`A1:C3`, `A:C 1:3` y `A1 r C3`), las columnas y las filas enteras y `*`. `moves` y `sequence` las aceptan, y las recorren en un orden que no elige quien escribe.

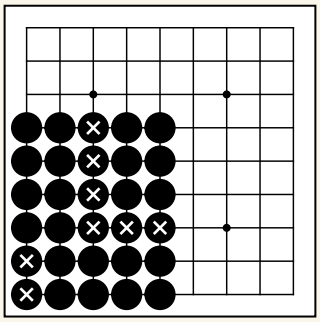
4 Quitar intersecciones

Después de un `;`, las intersecciones se quitan de la lista en lugar de agregarse. Cada `;` cambia de agregar a quitar, o de quitar a agregar, tantas veces como se quiera. `moves` no acepta el `;`: solo agrega jugadas.

Supongamos que queremos dibujar esto:

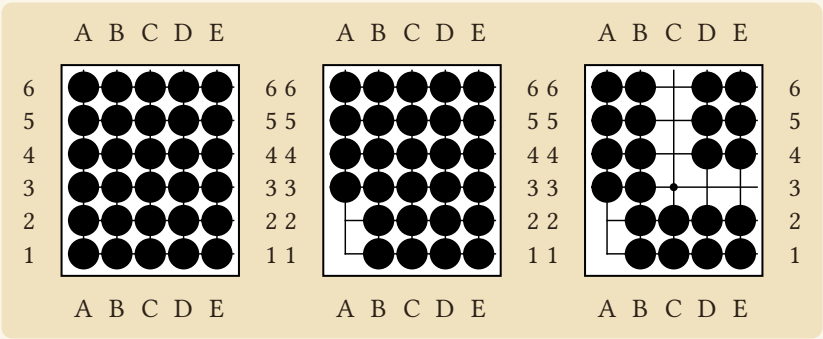


Puede dividirse en partes, o pensarse como un rectángulo completo al que se le quitan las piedras marcadas:



```
#gobanist-set(label: (at: (top, bottom, left, right)),
  partial: "A1:E6")

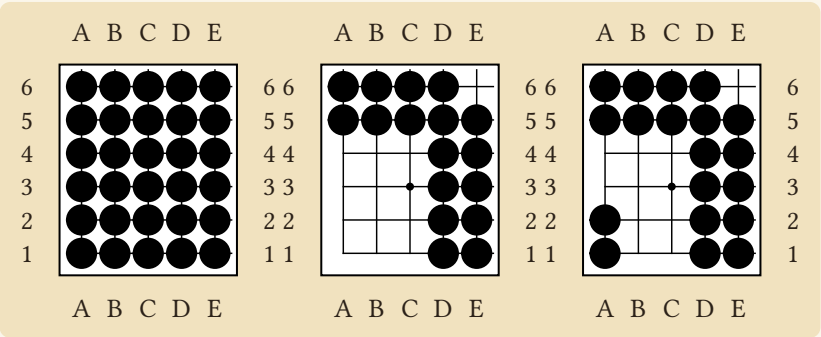
#goban(stones(black: "A1:E6"))
#goban(stones(black: "A1:E6; A1:2"))
#goban(stones(black: "A1:E6; A1:2, C6 |- E3"))
```



Primero, el rectángulo `A1:E6`; después se quita el tramo `A1:2`, y por último el camino `C6 |- E3`. Los grupos se alternan con cada `;`:

```
#gobanist-set(label: (at: (top, bottom, left, right)),
  partial: "A1:E6")

#goban(stones(black: "A1:E6"))
#goban(stones(black: "A1:E6; A1:C4, E6"))
#goban(stones(black: "A1:E6; A1:C4, E6; A1:2"))
```



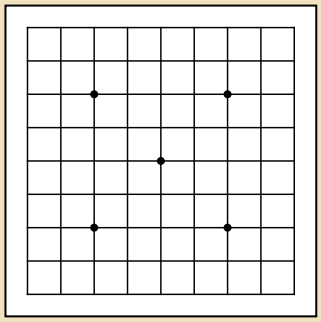
En el último, se agrega el rectángulo, se quitan `A1:C4` y `E6`, y se vuelve a agregar `A1:2`.

5 El tablero

El tablero es lo que dibuja `goban`: la cuadrícula, el fondo, las coordenadas y todo lo que va encima.

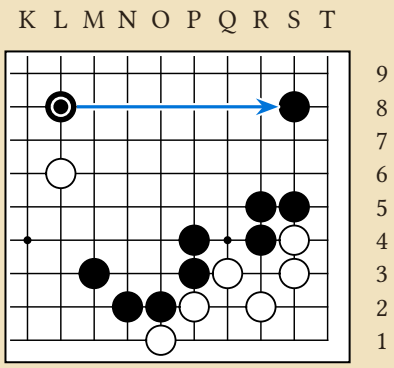
`goban(..opciones, ..elementos)`

Dibuja un tablero. Los elementos son lo que va en él, en orden: piedras (`stones`, `prisoners`), jugadas (`moves`), marcas (`marks`), flechas y líneas (`arrows`, `lines`), territorio (`territory`) y partidas (`sgf-read`, `sgf`). Primero puede ir un tablero hecho con `board`: es ese tablero, con sus opciones y lo que tiene, y las opciones y los elementos del `goban` se le agregan. Las opciones van con nombre, y son las de `gobanist-set` (pág. 30): valen solo para este tablero. Por defecto, el tablero es de 9×9, con un borde visible, fondo blanco y sin coordenadas.



```
#goban()
```

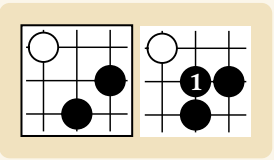
Las opciones del tablero (`size`, `use-i`, `partial`, `scale`, ...) van en el primer nivel; las de sus partes, en sus ramas: `grid`, `hoshi`, `background` y `label`.




```
#goban(size: 19, partial: "K1:T9",
  label: (at: (top, right)),
  stones(black: "L8, S8, R5, S5, P4, R4, M3,
    P3, N2, O2",
    white: "L6, S4, S3, Q3, O1, P2, R2"),
  marks(circle: "L8"),
  arrows(("L8", "S8"), color: blue,
    shorten: 6pt))
```

`board(..opciones, ..elementos)`

Un tablero como valor: recibe lo mismo que `goban`, pero no dibuja nada. Se resuelve al dibujarlo, con las opciones globales del lugar donde se dibuja. Primero puede ir otro tablero hecho con `board`, como en `goban`: `board(base, moves("C3"))` es `base` con una jugada más. Se dibuja con `goban(tablero)`, en `goban-layout` (pág. 103) y con `goban-view` (pág. 81); con `resume`, otro tablero sigue desde él (pág. 80).

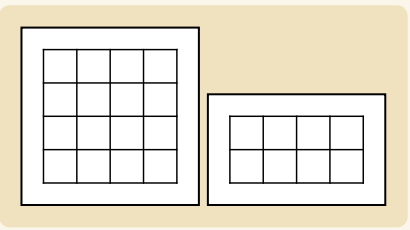


```
#let base = board(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"))

#goban(base)
#goban(base, moves("C3"),
  background: (line: false))
```

`size` un número o `(columnas, filas)` inicial `9`

El tamaño del tablero: las líneas verticales y horizontales. Un número es un tablero cuadrado; `(5, 3)`, uno de 5 columnas y 3 filas. Las columnas van de 2 a 52 (después de la Z, con dos letras: `AA`, `AB`...), y las filas, de 2 a 99. Con una partida (`sgf-read`), el tablero toma el tamaño del archivo (`SZ`), y otro `size` es un error.



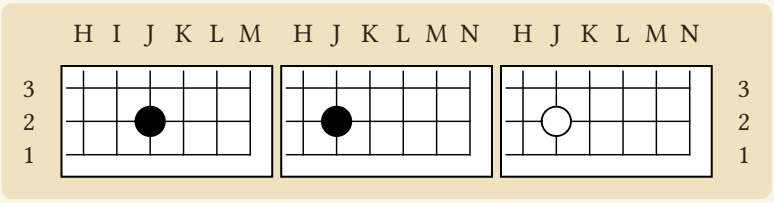
```
#goban(size: 5)
#goban(size: (5, 3))
```

`use-i` `true` o `false` inicial `false`

Usa la letra I en las coordenadas y en las etiquetas. Sin ella, como es costumbre en el go, las columnas pasan de la H a la J, y una I en las coordenadas es la J.

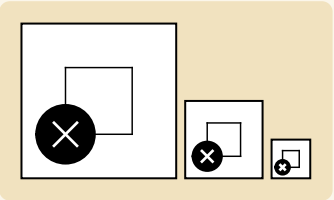
```
#gobanist-set(size: 13, label: (at: top))

#goban(use-i: true, partial: "H1:M3",
  label: (at: (top, left)),
  stones(black: "J2"))
#goban(partial: "H1:N3",
  stones(black: "J2"))
// Sin use-i, I es J.
#goban(partial: "H1:N3",
  label: (at: (top, right)),
  stones(white: "I2"))
```



`scale` un número inicial `1`

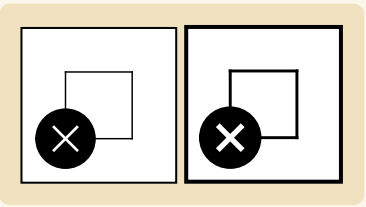
La escala del tablero: sus medidas. El grosor de las líneas no cambia, salvo con `scale-line-width`.



```
#goban(size: 2, scale: 2,
  stones(black: "A1", mark: "cross"))
#goban(size: 2,
  stones(black: "A1", mark: "cross"))
#goban(size: 2, scale: 0.5,
  stones(black: "A1", mark: "cross"))
```

`scale-line-width` `true` o `false` inicial `false`

Con `true`, `scale` multiplica también el grosor de las líneas: la cuadrícula, el borde, las piedras, las marcas, las flechas, las líneas y el territorio. Con `layout.fit`, que elige la escala para llenar la columna, el grosor no cambia (pág. 103).

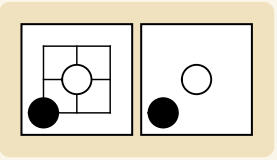


```
#goban(size: 2, scale: 2,
  stones(black: "A1", mark: "cross"))
#goban(size: 2, scale: 2,
  scale-line-width: true,
  stones(black: "A1", mark: "cross"))
```

1 La cuadrícula

`grid` `true`, `false` o un diccionario

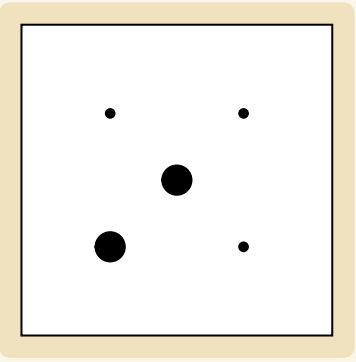
La cuadrícula: las líneas del tablero. Con `false`, sin cuadrícula ni puntos de estrella.



```
#goban(size: 3,
  stones(black: "A1", white: "B2"))
#goban(size: 3, grid: false,
  stones(black: "A1", white: "B2"))
```

`grid.line` `true`, `false` o un diccionario

Las líneas. Con `false`, sin líneas, pero con los puntos de estrella (a diferencia de `grid: false`).



```
#goban(grid: (line: false),
  hoshi: (radius: 2pt),
  stones(black: "C3, E5"))
```

`grid.line.color` un color — inicial `black`

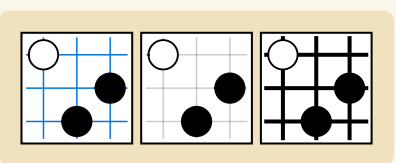
El color de las líneas.

`grid.line.opacity` un número de 0 a 1 — inicial `1`

La opacidad de las líneas.

`grid.line.width` una longitud — inicial `0.55pt`

El grosor de las líneas.

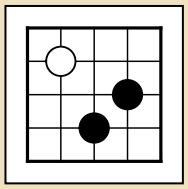


```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, grid: (line: (color: blue)))
#goban(b, grid: (line: (opacity: 0.2)))
#goban(b, grid: (line: (width: 1.5pt)))
```

`grid.edge` un diccionario — inicial `(color: auto, width: auto, opacity: auto)`

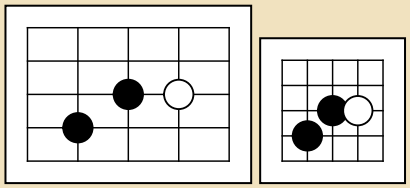
Las cuatro líneas de afuera de la cuadrícula, con otro color, grosor u opacidad que las demás. Con `auto`, el de `grid.line`.



```
#goban(size: 5, grid: (edge: (width: 1.2pt)),
  stones(black: "C2, D3", white: "B4"))
```

`grid.sep` una longitud o `(x: ..., y: ...)` inicial `(x: 12.5pt, y: 12.5pt)`

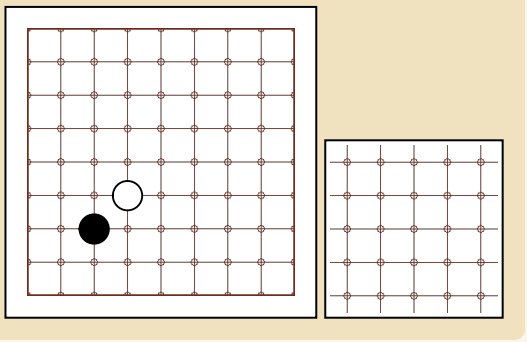
La distancia entre las líneas: `x`, entre las verticales, e `y`, entre las horizontales. Una longitud sola va a las dos.



```
#goban(size: 5, grid: (sep: (x: 1.8em)),
  stones(black: "B2, C3", white: "D3"))
#goban(size: 5, grid: (sep: 0.9em),
  stones(black: "B2, C3", white: "D3"))
```

`grid.image` un archivo o un diccionario

Una imagen en lugar de las líneas, con la opacidad de `grid.line.opacity`. Con `partial` se ve la parte que corresponde. Los puntos de estrella se siguen dibujando (pág. 44). Lo que no es un diccionario es el archivo; `none` la quita.

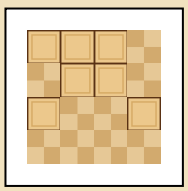


```
#let grilla = path("img/pgfgo-grid.png")

#goban(grid: (image: grilla), hoshi: false,
  stones(black: "C3", white: "D4"))
#goban(grid: (image: grilla), hoshi: false,
  partial: "C3:G7")
```

`grid.image.file` un archivo o un arreglo inicial `none`

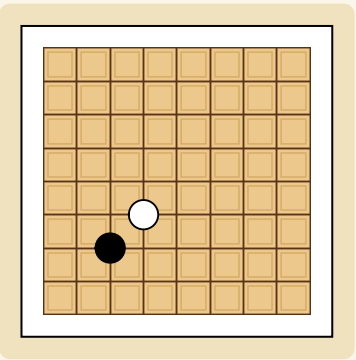
El archivo: un `path` (relativo al documento que lo nombra) o sus `bytes`. Con varios archivos y `fit: "tile"`, cada baldosa usa uno, elegido por su lugar en la malla: siempre el mismo en cada lugar, en todos los diagramas. Con `"stretch"`, se usa el primero.



```
#goban(size: 5, hoshi: false,
  grid: (image: (fit: "tile", file: (
    path("img/pgfgo-cell.png"),
    path("img/pgfgo-tile.png")))))
```

`grid.image.fit` `"stretch"` o `"tile"` inicial `"stretch"`

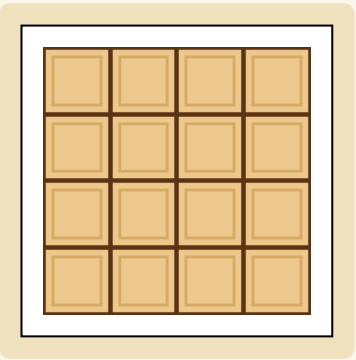
Con `"stretch"`, la imagen se estira desde A1 hasta la esquina opuesta del tablero completo: las líneas exteriores tienen que estar en los bordes de la imagen. Con `"tile"`, se dibuja una vez en cada casilla (entre cuatro intersecciones), estirada al tamaño de la casilla: con la imagen de un cuadrado se arma todo el tablero.



```
#goban(hoshi: false, grid: (image: (
    file: path("img/pgfgo-cell.png"),
    fit: "tile")),
  stones(black: "C3", white: "D4"))
```

`grid.image.tiles` `auto`, un número o `(columnas, filas)` inicial `auto`

Con `fit: "tile"`, la malla en la que se repite la imagen, sobre el área que va de A1 a la esquina opuesta del tablero completo; cada imagen se estira al tamaño de su celda. Un número vale para las dos direcciones. Con `auto`, una imagen por casilla.



```
#goban(hoshi: false, grid: (image: (
  file: path("img/pgfgo-cell.png"),
  fit: "tile", tiles: 4)))
```

`grid.image.rotate` un ángulo, `"random"` o `(mínimo, máximo)` inicial `0deg`

Gira la imagen, como `background.fill.image.rotate`: en cuartos de vuelta, y con `"random"` o un rango, cada baldosa con el suyo.

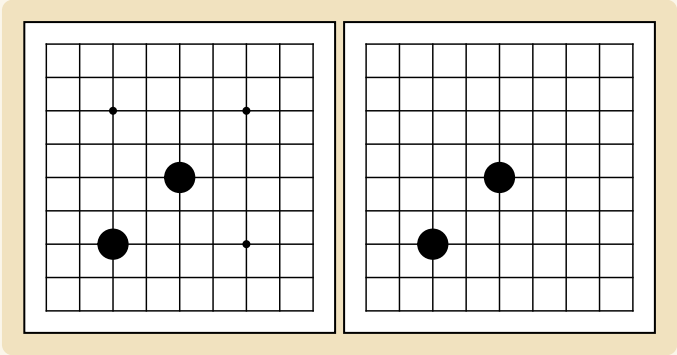
2 Puntos de estrella

Los puntos de estrella (*hoshi*) se dibujan sobre la cuadrícula, con su color. Por defecto son los de siempre: en cada lado, la cuarta línea desde el borde (la tercera en los tableros de menos de 13 líneas); el centro si los dos lados son impares; y, desde 15 líneas, el medio de los lados. En un tablero de 19 hay nueve, y en uno de 13 o de 9, cinco. Los tableros de menos de 7 líneas no tienen. Las piedras los tapan.

`hoshi` `true`, `false` o un diccionario

Los puntos de estrella.

```
#goban(stones(black: "C3, E5"))
#goban(hoshi: false, stones(black: "C3, E5"))
```



`hoshi.radius` una longitud — inicial `1.5pt`

El radio de los puntos.

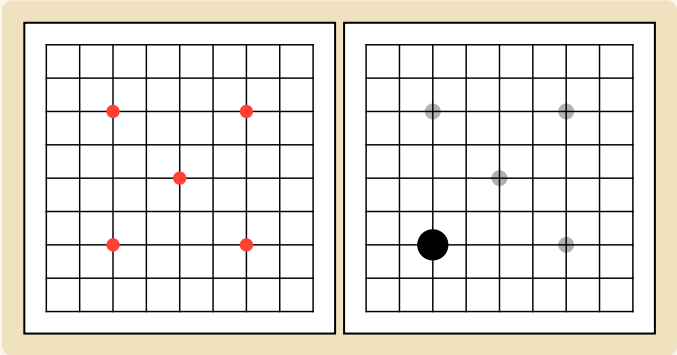
`hoshi.color` `auto` o un color — inicial `auto`

El color de los puntos. Con `auto`, el de la cuadrícula.

`hoshi.opacity` `auto` o un número de 0 a 1 — inicial `auto`

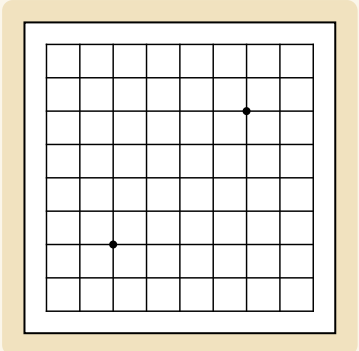
La opacidad de los puntos (también la de su imagen). Con `auto`, la de la cuadrícula.

```
#goban(hoshi: (radius: 2.5pt, color: red))
#goban(hoshi: (radius: 3pt, opacity: 0.3),
  stones(black: "C3"))
```



`hoshi.at` `auto` o coordenadas — inicial `auto`

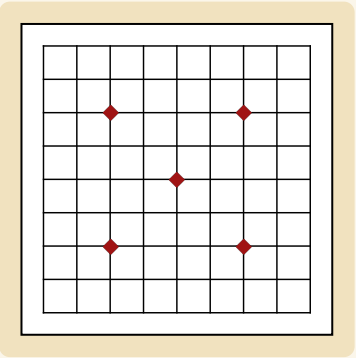
Los puntos, con la sintaxis de `stones` (pág. 33). Con `auto`, los de siempre.



```
#goban(hoshi: (at: "C3, G7"))
```

`hoshi.image` un archivo o un diccionario

Una imagen en cada punto, en lugar del círculo. Lo que no es un diccionario es el archivo; `none` la quita.



```
#goban(hoshi: (radius: 3pt,
  image: path("img/pgfgo-hoshi.png")))
```

`hoshi.image.file` un archivo — inicial `none`

El archivo de la imagen.

`hoshi.image.width` `auto` o una longitud — inicial `auto`

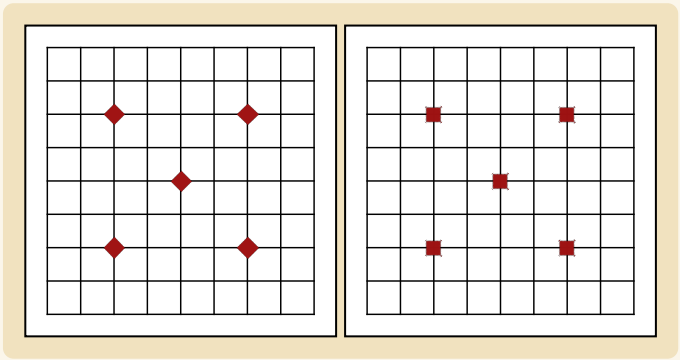
El ancho de la imagen; el alto mantiene la proporción. Con `auto`, el diámetro de los puntos.

`hoshi.image.rotate` un ángulo, `"random"` o `(mínimo, máximo)` — inicial `0deg`

Gira la imagen, en sentido antihorario. Con `"random"` o un rango, cada punto con su ángulo, siempre el mismo (pág. 58).

```
#let hoshi = path("img/pgfgo-hoshi.png")

#goban(hoshi: (image: (file: hoshi,
  width: 8pt)))
#goban(hoshi: (image: (file: hoshi,
  width: 8pt, rotate: 45deg)))
```

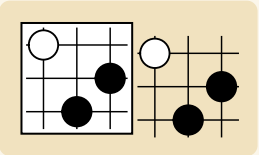


3 El fondo

El fondo del tablero es el rectángulo que va debajo de la cuadrícula, con su relleno y su borde. Por defecto, el relleno es blanco y el borde, negro.

`background` `true`, `false` o un diccionario

El fondo: el relleno y el borde. Con `false`, sin fondo.



```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b)
#goban(b, background: false)
```

`background.fill` `true`, `false` o un diccionario

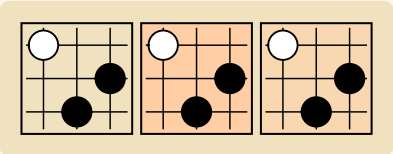
El relleno. Con `false`, el fondo es transparente y queda solo el borde.

`background.fill.color` un color — inicial `white`

El color del relleno.

`background.fill.opacity` un número de 0 a 1 — inicial `1`

La opacidad del relleno, también la de su imagen.

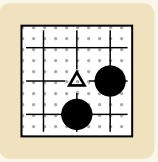


```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, background: (fill: false))
#goban(b, background: (fill: (
  color: orange.lighten(60%))))
#goban(b, background: (fill: (
  color: orange.lighten(60%), opacity: 0.5)))
```

`background.fill.pattern` `none` o un `tiling` — inicial `none`

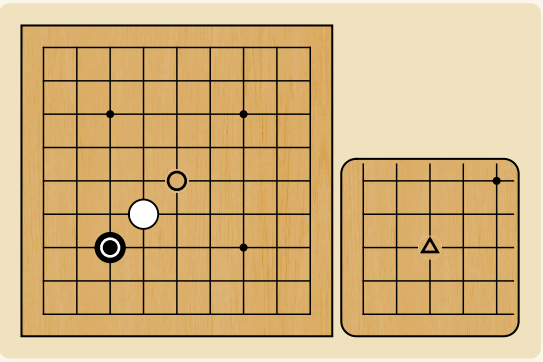
Un patrón encima del color y de la imagen del fondo: un `tiling` de Typst. Las marcas sobre puntos vacíos y la marca `dim`, que tapan la cuadrícula con el fondo, repiten el patrón.



```
#goban(size: 5, partial: "B2:D4",
  background: (fill: (pattern: tiling(
    size: (4pt, 4pt),
    circle(radius: 0.6pt, fill: gray)))),
  stones(black: "C2, D3"),
  marks(triangle: "C3"))
```

`background.fill.image` un archivo o un diccionario

Una imagen de fondo (por ejemplo, la madera de un tablero), sobre el color de relleno y con su misma opacidad. La imagen se ubica según el tablero completo: con `partial` se ve la parte que le corresponde, recortada por el borde y sus esquinas redondeadas. Las marcas sobre puntos vacíos, que tapan la cuadrícula con el fondo, muestran la parte de la imagen que hay debajo, y lo mismo hace la marca `dim`. Lo que no es un diccionario es el archivo; `none` la quita.

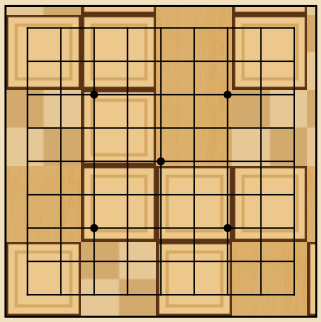


```
#let madera = path("img/pgfgo-board.jpg")

#goban(background: (fill: (image: madera)),
  stones(black: "C3", white: "D4"),
  marks(circle: "E5, C3"))
#goban(partial: "A1:E5", background: (
    fill: (image: madera),
    rounded-corners: 6pt),
  marks(triangle: "C3"))
```

`background.fill.image.file` un archivo o un arreglo — inicial `none`

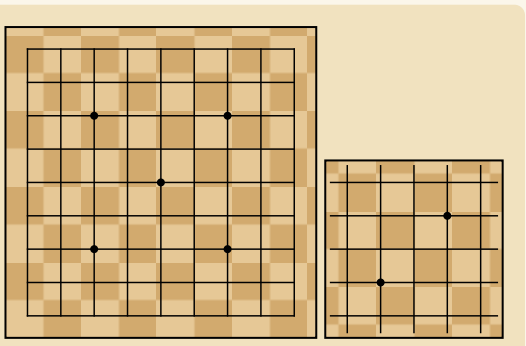
El archivo de la imagen. Con varios archivos y `fit: "tile"`, cada baldosa usa uno, elegido por su lugar: siempre el mismo en cada lugar, también con `partial`. Las baldosas tienen el tamaño de la primera imagen (o el de su celda, con `tiles`). Con `"stretch"` y `"cover"`, se usa el primero.



```
#goban(background: (fill: (image: (
  file: (path("img/pgfgo-tile.png"),
    path("img/pgfgo-cell.png"),
    path("img/pgfgo-board.jpg")),
  fit: "tile", width: 1cm))))
```

`background.fill.image.fit` `"cover"`, `"stretch"` o `"tile"` — inicial `"cover"`

Cómo ocupa la imagen el fondo del tablero completo. `"stretch"` la estira hasta ocupar todo el fondo, sin mantener la proporción. `"cover"` la agranda o la achica, manteniendo la proporción, hasta cubrir el fondo, centrada: lo que sobra no se ve. `"tile"` la repite como baldosas, desde una esquina, un borde o el centro del tablero completo (`anchor`).

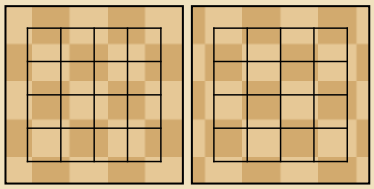


```
#let baldosa = (file: path("img/pgfgo-tile.png"),
  fit: "tile", width: 1cm)

#goban(background: (fill: (image: baldosa)))
#goban(partial: "D4:H8",
  background: (fill: (image: baldosa)))
```

`background.fill.image.anchor` una alineación — inicial `bottom + left`

Con `fit: "tile"`, desde dónde se ponen las baldosas, en el fondo del tablero completo. Con una esquina (`bottom + left`, `top + right`...), en esa esquina queda una baldosa entera. Con un borde (`top`, `bottom`, `left` o `right`), la primera baldosa queda contra ese borde, centrada a lo largo de él. Con `center`, una baldosa queda en el centro del tablero.



```
#let baldosa = (file: path("img/pgfgo-tile.png"),
  fit: "tile", width: 1cm)

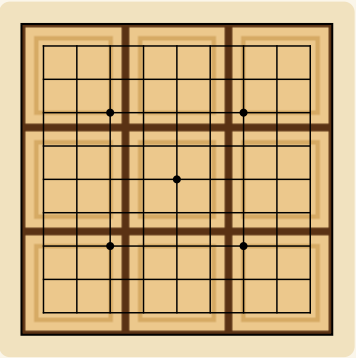
#goban(size: 5, background: (fill: (
  image: baldosa + (anchor: top + right))))
#goban(size: 5, background: (fill: (
  image: baldosa + (anchor: top))))
```

`background.fill.image.width` `auto` o una longitud — inicial `auto`

Con `fit: "tile"`, el ancho de cada baldosa; el alto mantiene la proporción. Con `auto`, el ancho natural de la imagen.

`background.fill.image.tiles` `auto`, un número o `(columnas, filas)` — inicial `auto`

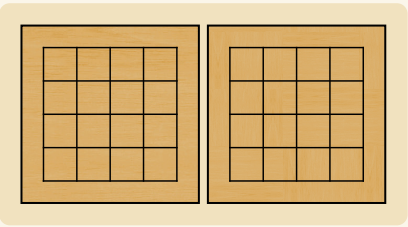
Con `fit: "tile"`, en lugar de baldosas de ancho `width` desde `anchor`, divide el fondo del tablero completo en esa malla y pone la imagen en cada celda, estirada a su tamaño. Un número vale para las dos direcciones.



```
#goban(background: (fill: (image: (
  file: path("img/pgfgo-cell.png"),
  fit: "tile", tiles: (3, 3)))))
```

`background.fill.image.rotate` un ángulo, `"random"` o `(mínimo, máximo)` inicial `0deg`

Gira la imagen en cuartos de vuelta: el ángulo se redondea a 0, 90, 180 o 270 grados, para que la imagen girada siga ocupando su lugar sin dejar huecos. Con 90 o 270, el ancho y el alto se intercambian. Con `"random"` o un rango, cada baldosa tiene su propio giro, siempre el mismo.



```
#let madera = path("img/pgfgo-board.jpg")

#goban(size: 5, background: (fill: (image: (
  file: madera, rotate: 90deg))))
#goban(size: 5, background: (fill: (image: (
  file: madera, fit: "tile", width: 1cm,
  rotate: "random"))))
```

`background.line` `true`, `false` o un diccionario

El borde. Con `false`, sin borde.

`background.line.color` un color inicial `black`

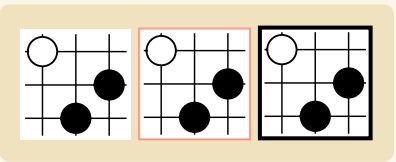
El color del borde.

`background.line.opacity` un número de 0 a 1 inicial `1`

La opacidad del borde.

`background.line.width` una longitud inicial `0.75pt`

El grosor del borde.



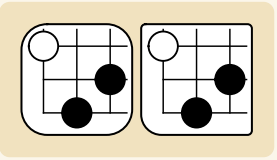
```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, background: (line: false))
#goban(b, background: (line: (
  color: red, opacity: 0.5)))
#goban(b, background: (line: (
  width: 1.5pt)))
```

`background.rounded-corners` una longitud o `(outer: ..., inner: ...)`

inicial `(outer: 0pt, inner: 0pt)`

El radio de las esquinas del fondo: `outer`, el de las esquinas del tablero, e `inner`, el de las que se ven en un tablero parcial. Una longitud sola va a las dos.

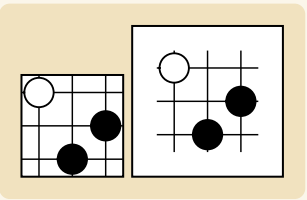


```
#gobanist-set(size: 5, partial: "A1:C3")
#let b = board(stones(black: "B1, C2", white: "A3"))

#goban(b, background: (
  rounded-corners: 10pt))
#goban(b, background: (
  rounded-corners: (outer: 8pt, inner: 2pt)))
```

`background.sep` una longitud — inicial `8.25pt`

La distancia entre la cuadrícula y el borde.

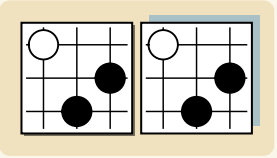


```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, background: (sep: 0.625em))
#goban(b, background: (sep: 1.5em))
```

`background.shadow` `true`, `false` o un diccionario

Una sombra debajo del tablero.



```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, background: (shadow: true))
#goban(b, background: (shadow: (
  on: true, color: blue, opacity: 0.3,
  angle: 45deg, offset: 4pt)))
```

`background.shadow.angle` un ángulo — inicial `-45deg`

Hacia dónde cae la sombra: con `0deg`, a la derecha; con `-45deg`, abajo a la derecha.

`background.shadow.offset` una longitud — inicial `1.5pt`

La distancia entre el tablero y su sombra.

`background.shadow.opacity` un número de 0 a 1 — inicial `0.5`

La opacidad de la sombra.

`background.shadow.color` un color — inicial `black`

El color de la sombra.

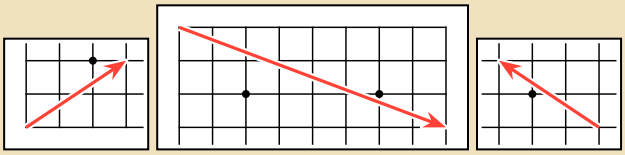
4 Tablero parcial

Muchas veces no hace falta el tablero completo, sino una parte. `partial` la define con dos esquinas opuestas de un rectángulo, en cualquier orden: `"A1:D3"` es lo mismo que `"D3:A1"`, y que `"A3:D1"`.

`partial` `none`, `"A1:D3"` o `auto` — inicial `none`

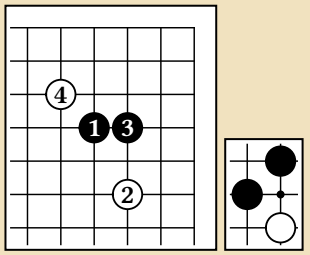
Muestra solo una parte del tablero: el rectángulo que tiene esas dos intersecciones como esquinas opuestas. Lo que cae fuera no se dibuja. Con `auto`, la parte la elige el tablero: la que usan las piedras, jugadas, marcas, flechas, líneas y territorios que se dibujan, con `partial-margin` líneas más. Con `none`, el tablero completo.

```
#goban(partial: "A1:D3",
  arrows(("A1", "D3"), color: red))
#goban(partial: "A9:J6",
  arrows(("A9", "J6"), color: red))
#goban(partial: "E2:B4",
  arrows(("E2", "B4"), color: red))
```



`partial-margin` un número — inicial `1`

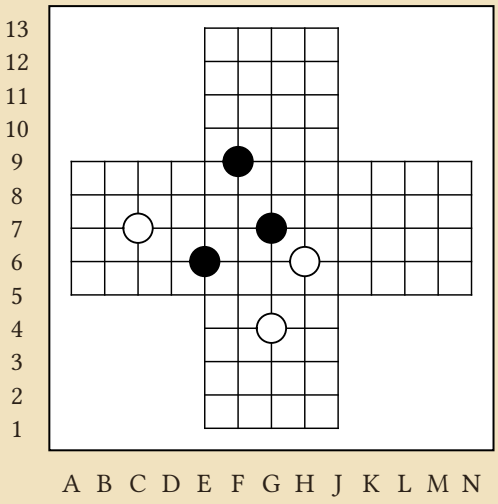
Las líneas que `partial: auto` agrega alrededor de lo dibujado. Un lado que queda a dos líneas o menos del borde llega hasta el borde. Sin nada dibujado, se ve todo el tablero.




```
#goban(size: 19, partial: auto,
  moves("Q16, R14, R16, P17"))
#goban(size: 19, partial: auto,
  partial-margin: 0,
  stones(black: "J10, K11", white: "K9"))
```

`blank` `none` o coordenadas — inicial `none`

Las intersecciones que no son parte del tablero, leídas en el tablero entero (aunque haya `partial`): sin líneas ni puntos de estrella. Una línea une solo dos intersecciones vecinas del tablero: junto a un punto de `blank` termina como en el borde. Las piedras y el fondo no cambian. Con `grid.image`, la imagen va solo en las casillas del tablero.




```
#goban(size: 13, label: (at: (left, bottom)),
  blank: "A:D 1:4, K:N 1:4, A:D 10:13, K:N 10:13",
  stones(black: "G7, F9, E6", white: "H6, G4, C7"))
```

5 Las coordenadas del borde

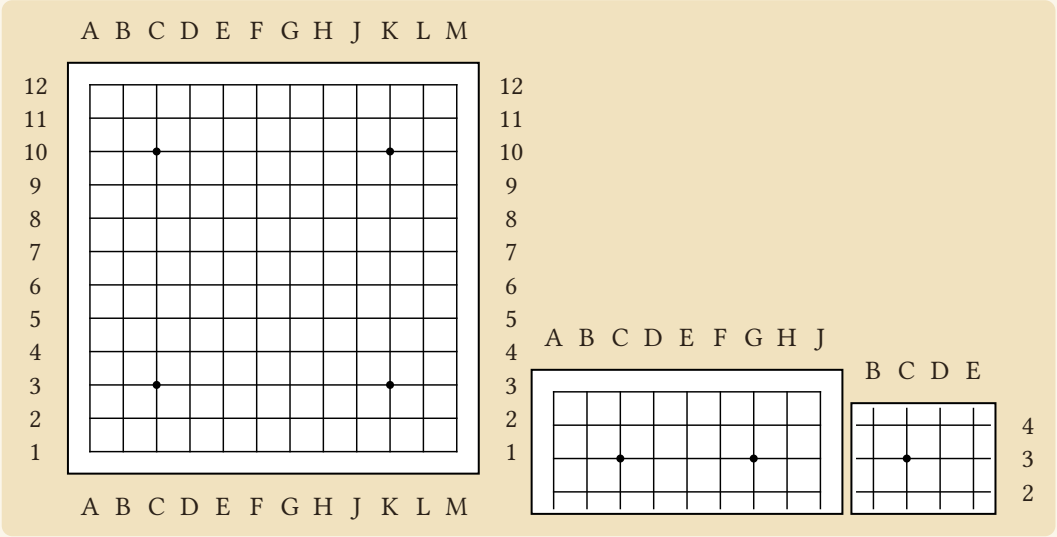
Las etiquetas son las coordenadas que se escriben a los lados del tablero.

`label.at` una alineación o un arreglo

inicial `()`

En qué lados van las etiquetas: `top`, `bottom`, `left` y `right`, o un arreglo de ellos. Arriba y abajo van las columnas; a los costados, las filas. Con `()`, sin etiquetas.

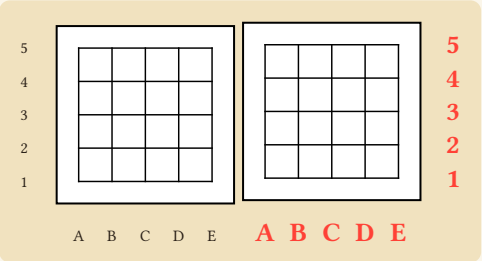
```
#goban(size: 12,
  label: (at: (top, bottom, left, right)))
#goban(partial: "A6:J9", label: (at: top))
#goban(partial: "B2:E4",
  label: (at: (top, right)))
```



`label.text` un diccionario

inicial `(size: 10pt)`

Las opciones de `text` de las etiquetas: la fuente, el tamaño, el color.



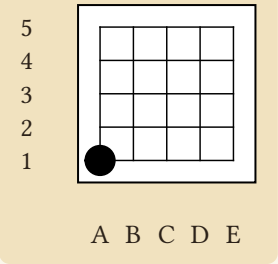

```
#gobanist-set(size: 5)

#goban(label: (at: (bottom, left),
  text: (size: 6pt)))
#goban(label: (at: (bottom, right),
  text: (weight: "bold", fill: red)))
```

`label.sep` una longitud

inicial `0.35cm`

La distancia entre las etiquetas y el borde del tablero.




```
#goban(size: 5, label: (at: (bottom, left),
  sep: 0.6cm), stones(black: "A1"))
```

`label.columns` `"A"`, `"a"` o un patrón de `numbering`

inicial `"A"`

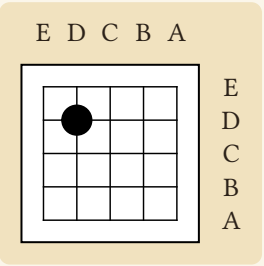
Cómo se escriben las coordenadas de las columnas (arriba y abajo): con `"A"`, las letras del go (sin la I, salvo con `use-i`); con `"a"`, las mismas en minúsculas; con otro patrón de `numbering` de Typst, como `"1"` o `"—"`, números.

`label.rows` `"A"`, `"a"` o un patrón de `numbering` (inicial `"1"`)

Lo mismo para las filas (a los costados). Los kanji (`"一"`) de las filas se alinean del lado del tablero, para que los de dos caracteres no lo pisen.

`label.columns-direction` `ltr` o `rtl` (inicial `ltr`)

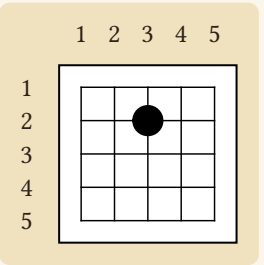
En qué sentido se cuentan las columnas: desde la izquierda o desde la derecha.




```
#goban(size: 5, label: (at: (top, right),
    columns-direction: rtl, rows: "A"),
  stones(black: "B4"))
```

`label.rows-direction` `btt` o `ttb` (inicial `btt`)

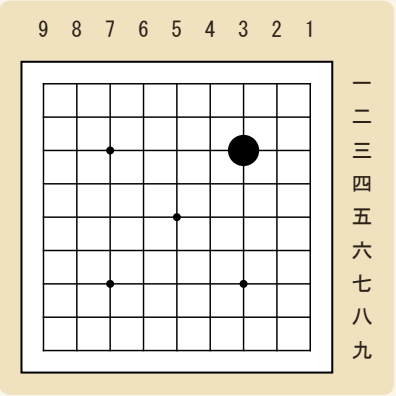
En qué sentido se cuentan las filas: desde abajo o desde arriba. Estas cuatro opciones cambian solo lo que se escribe en el borde: las coordenadas de los elementos (`"C3"`) siguen siendo las mismas. En un tablero parcial se cuenta desde el borde del tablero completo.




```
#goban(size: 5, label: (at: (top, left),
    columns: "1", rows-direction: ttb),
  stones(black: "C4"))
```

`label.style` `"western"`, `"japanese"` o `"chinese"`

Fija las cuatro opciones anteriores según la costumbre de cada lugar. `"western"`: columnas con letras desde la izquierda y filas con números desde abajo, como al empezar. `"japanese"`: columnas con números desde la derecha y filas con kanji desde arriba, como en los libros japoneses: la esquina de arriba a la derecha es 1-一. `"chinese"`: columnas con números desde la izquierda y filas con kanji desde arriba. Los kanji necesitan una fuente que los tenga.




```
#goban(label: (at: (top, right),
    style: "japanese",
    text: (font: "MS Gothic", size: 8pt)),
  stones(black: "G7"))
```

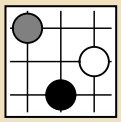
6 Piedras, jugadas y prisioneros

Las piedras se ponen con `stones`; las jugadas, con `moves`, y los prisioneros, con `prisoners`. Su estilo es el de su jugador (pág. 107), y cada elemento puede cambiarlo con sus opciones.

1 Piedras

```
stones(coordenadas, black: ..., white: ..., neutral: ..., ..opciones)
```

Pone piedras en las intersecciones, escritas con la sintaxis de coordenadas (pág. 33). El jugador es la clave: `stones(black: "C2, D3", white: "B4")`. Sin clave, las piedras son neutrales: `stones("C3")`. Las opciones cambian su estilo (las de la rama `stone`, más abajo) y les ponen una marca (`mark`).



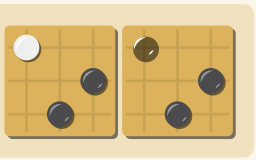
```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2", white: "D3"),
  stones("B4"))
```

Las piedras, marcas y etiquetas que caen fuera de la parte visible de un tablero parcial no se dibujan. Una coordenada que no existe en el tablero (`"E9"` en un tablero de 5×5) es un error. Una piedra en un punto donde ya hay otra la tapa.

El estilo de las piedras de los tres jugadores se cambia con `gobanist-set(stone: ...)`, que es `gobanist-set(players: (all: (stone: ...)))`; el de uno, con `gobanist-set(players: (black: (stone: ...)))`. Las opciones de la rama `stone` son:

`stone.fill` `true`, `false` o un diccionario

El relleno de la piedra. Con `false`, sin relleno: queda lo demás (el borde, la sombra, el brillo).

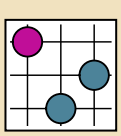


```
#gobanist-set(profile: "pgfgosmooth", size: 5,
  partial: "B2:D4")

#goban(stones(black: "C2, D3", white: "B4"))
#goban(stones(black: "C2, D3"),
  stones(white: "B4", fill: false))
```

`stone.fill.color` un color o `none`

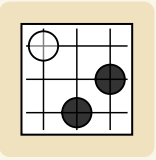
El color del relleno: negro para las negras, blanco para las blancas y gris para las neutrales. Con `none`, sin color: queda el patrón, si lo hay.



```
#goban(size: 5, partial: "B2:D4",
  stones("C2, D3", fill: (color: aqua.darken(40%))),
  stones("B4", fill: (color: fuchsia.darken(20%))))
```

`stone.fill.opacity` un número de 0 a 1 — inicial `1`

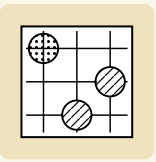
La opacidad del relleno.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", fill: (opacity: 0.8)),
  stones(white: "B4", fill: (opacity: 0.5)))
```

`stone.fill.pattern` `none` o un `tiling` — inicial `none`

Un patrón encima del color del relleno, con su misma opacidad: un `tiling` de Typst. Con `fill.color: none`, la piedra tiene solo el patrón. Las marcas `t-circle` y `t-square` del jugador usan el mismo relleno. El patrón no cambia de tamaño con `scale`.

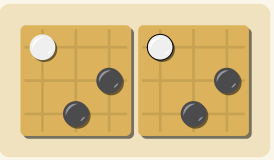


```
#let rayas = tiling(size: (3pt, 3pt),
  line(start: (0pt, 3pt), end: (3pt, 0pt),
    stroke: 0.4pt))

#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3",
    fill: (color: white, pattern: rayas)),
  stones(white: "B4", fill: (color: none,
    pattern: tiling(size: (2pt, 2pt),
      circle(radius: 0.5pt, fill: black)))))
```

`stone.line` `true`, `false` o un diccionario

El borde de la piedra.



```
#gobanist-set(profile: "pgfgosmooth", size: 5,
  partial: "B2:D4")

#goban(stones(black: "C2, D3", white: "B4"))
#goban(stones(black: "C2, D3"),
  stones(white: "B4", line: true))
```

`stone.line.color` un color — inicial `black`

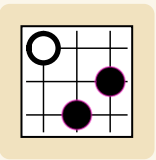
El color del borde.

`stone.line.opacity` un número de 0 a 1 — inicial `1`

La opacidad del borde.

`stone.line.width` una longitud — inicial `0.7pt`

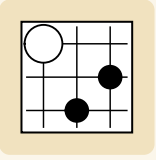
El grosor del borde.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", line: (
    color: fuchsia, opacity: 0.5)),
  stones(white: "B4", line: (width: 2pt)))
```

`stone.radius` una longitud — inicial `5.5pt`

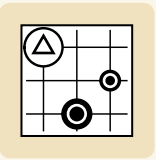
El radio de la piedra.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", radius: 0.15cm),
  stones(white: "B4", radius: 0.25cm))
```

`stone.scale` un número — inicial `1`

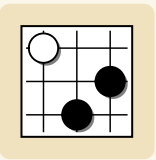
Escala la piedra entera: el radio, el borde y las marcas.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2", mark: "circle"),
  stones(black: "D3", scale: 0.7,
    mark: "circle"),
  stones(white: "B4", scale: 1.3,
    mark: "triangle"))
```

`stone.shadow` `true`, `false` o un diccionario

La sombra de la piedra, dibujada debajo. Tiene la silueta de la piedra: el círculo, o la forma de la imagen si la piedra es una imagen (`image`). Con `shadow.image`, la sombra es una imagen propia. En los dos casos se corre `offset` hacia `angle` y se agranda `scale` veces.



```
#goban(size: 5, partial: "B2:D4",
  stone: (shadow: (on: true, angle: -45deg,
    offset: 1.5pt)),
  stones(black: "C2, D3", white: "B4"))
```

`stone.shadow.angle` un ángulo — inicial `0deg`

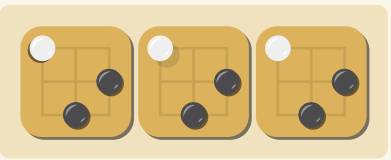
Hacia dónde cae la sombra: con `0deg`, a la derecha; con `-45deg`, abajo a la derecha. No gira la forma de la sombra.

`stone.shadow.offset` una longitud — inicial `1pt`

Cuánto se corre la sombra respecto del centro de la piedra, hacia `angle`.

`stone.shadow.opacity` un número de 0 a 1 — inicial `0.5`

La opacidad de la sombra (también la de la sombra con imagen).



```
#gobanist-set(profile: "pgfgosmooth", size: 3)
#let b = board(stones(black: "B1, C2", white: "A3"))

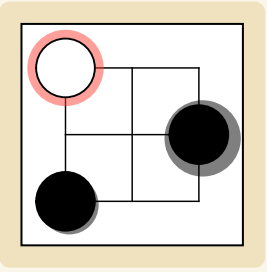
#goban(b, players: (white: (stone: (
  shadow: (angle: 225deg)))))
#goban(b, players: (white: (stone: (
  shadow: (offset: 0.125cm, opacity: 0.1)))))
#goban(b, players: (white: (stone: (
  shadow: false))))
```

`stone.shadow.color` un color — inicial `black`

El color de la sombra. Una sombra con imagen tiene sus propios colores, salvo con `image.silhouette`.

`stone.shadow.scale` un número — inicial `1`

Agranda (o achica) la sombra respecto de la piedra. Una sombra más grande que la piedra la hace parecer más alta. También agranda la sombra con imagen.

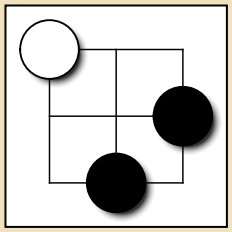


```
#goban(size: 3, scale: 2,
  stones(black: "A1", shadow: (on: true,
    angle: -45deg)),
  stones(black: "C2", shadow: (on: true,
    angle: -45deg, scale: 1.3)),
  stones(white: "A3", shadow: (on: true,
    offset: 0pt, scale: 1.3, color: red)))
```

`stone.shadow.image` un archivo o un diccionario

Una sombra propia: una imagen, centrada en la piedra y dibujada tal cual en lugar de la silueta, por ejemplo una sombra difusa. Se corre y se agranda como la silueta (`angle`, `offset`, `scale`) y usa `opacity`, pero

no `color`. No gira con la imagen de la piedra, así que cae siempre hacia el mismo lado. Lo que no es un diccionario es el archivo; `none` la quita.



```
#goban(size: 3, scale: 2,
  stone: (shadow: (on: true, angle: -45deg,
    offset: 1.5pt, opacity: 0.8,
    image: (file: path("img/pgfgo-shadow.png"),
      width: 13pt))),
  stones(black: "B1, C2", white: "A3"))
```

`stone.shadow.image.file` un archivo — inicial `none`

El archivo de la imagen.

`stone.shadow.image.width` `auto` o una longitud — inicial `auto`

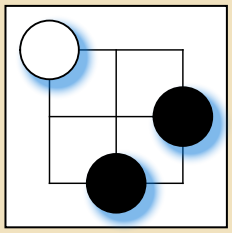
El ancho de la imagen; el alto mantiene la proporción. Con `auto`, el diámetro de la piedra.

`stone.shadow.image.rotate` un ángulo — inicial `0deg`

Gira la imagen de la sombra, igual en todas las piedras.

`stone.shadow.image.silhouette` `true` o `false` — inicial `false`

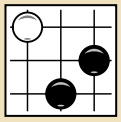
Usa solo la forma de la imagen (su transparencia), pintada con `shadow.color`, en lugar de la imagen tal cual. Solo con PNG y JPEG.



```
#goban(size: 3, scale: 2,
  stone: (shadow: (on: true, angle: -45deg,
    offset: 2pt, color: blue,
    image: (file: path("img/pgfgo-shadow.png"),
      width: 16pt, silhouette: true))),
  stones(black: "B1, C2", white: "A3"))
```

`stone.shine` `true`, `false` o un diccionario

El brillo de la piedra, dibujado encima de ella, sea un círculo o una imagen.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", shine: true),
  stones(white: "B4", shine: (on: true,
    color: luma(50%))))
```

`stone.shine.angle` un ángulo — inicial `0deg`

Gira el brillo dibujado. Con `0deg`, la luz viene de arriba a la izquierda.

`stone.shine.opacity` un número de 0 a 1 — inicial `1`

La opacidad del brillo (también la del brillo con imagen).

`stone.shine.color` un color — inicial `luma(75%)`

El color del brillo dibujado.

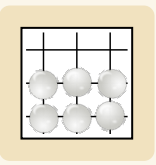


```
#gobanist-set(profile: "pgfgosmooth", size: 3)
#let b = board(stones(black: "B1, C2", white: "A3"))

#goban(b, players: (black: (stone: (
  shine: (angle: 45deg)))))
#goban(b, players: (black: (stone: (
  shine: (opacity: 0.5, color: red)))))
#goban(b, players: (black: (stone: (
  shine: false))))
```

`stone.shine.image` un archivo o un diccionario

Un brillo propio: una imagen, centrada en la piedra, en lugar del brillo dibujado. Si la piedra también es una imagen, el brillo se dibuja encima y no gira con ella: las piedras pueden tener vetas giradas al azar y la luz siempre del mismo lado. Lo que no es un diccionario es el archivo; `none` la quita.



```
#goban(size: 5, partial: "B2:D4",
  stones(white: "B2:D3",
    image: (file: path("img/pgfgo-yunzi-white-1.png"),
      rotate: "random"),
    shine: (on: true,
      image: path("img/pgfgo-shine.png"))))
```

`stone.shine.image.file` un archivo — inicial `none`

El archivo de la imagen.

`stone.shine.image.width` `auto` o una longitud — inicial `auto`

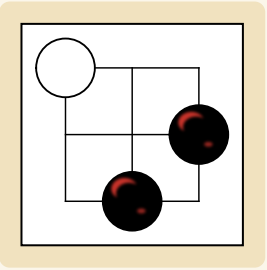
El ancho de la imagen; el alto mantiene la proporción. Con `auto`, el diámetro de la piedra.

`stone.shine.image.rotate` un ángulo — inicial `0deg`

Gira la imagen del brillo, igual en todas las piedras.

`stone.shine.image.silhouette` `true` o `false` — inicial `false`

Usa solo la forma de la imagen (su transparencia), pintada con `shine.color`, en lugar de la imagen tal cual. Solo con PNG y JPEG.



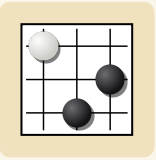
```
#goban(size: 3, scale: 2,
  players: (black: (stone: (shine: (on: true,
    color: red, image: (
      file: path("img/pgfgo-shine.png"),
      silhouette: true))))),
  stones(black: "B1, C2", white: "A3"))
```

Piedras con imagen

`stone.image` un archivo o un diccionario

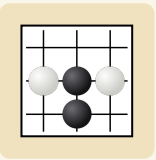
Dibuja la piedra con una imagen en lugar del círculo: no se dibujan el relleno ni el borde, y la imagen no tiene por qué ser redonda. Si el brillo está prendido, se dibuja encima de la imagen. La sombra se dibuja con la forma de la imagen; para eso la imagen tiene que tener transparencia (un PNG con canal alfa): la sombra de un JPEG es un rectángulo. Un prisionero con imagen se ve translúcido, como cualquier prisionero. Lo que no es un diccionario es el archivo; `none` la quita.

Un archivo es un `path`, relativo al documento que lo nombra, o sus `bytes`: PNG, JPEG, GIF, WebP, SVG o PDF. La opacidad y las siluetas (la sombra con la forma de la imagen) solo se pueden con PNG y JPEG: las hace el kernel (pág. 31). Las imágenes de los perfiles `shell`, `yunzi` y `baduktv` están en la carpeta `images` del paquete.



```
#goban(size: 5, partial: "B2:D4",
  stone: (shadow: (on: true, angle: -45deg,
    offset: 1.5pt)),
  stones(black: "C2, D3", image:
    path("img/pgfgo-yunzi-black.png")),
  stones(white: "B4", image:
    path("img/pgfgo-yunzi-white-1.png")))
```

Para todas las piedras de un jugador, la imagen va en sus opciones:

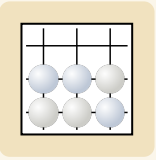


```
#gobanist-set(players: (
  black: (stone: (image:
    path("img/pgfgo-yunzi-black.png"))),
  white: (stone: (image:
    path("img/pgfgo-yunzi-white-1.png")))))

#goban(size: 5, partial: "B2:D4",
  moves("C3, D3, C2, B3", numbers: false))
```

`stone.image.file` un archivo o un arreglo — inicial `none`

El archivo de la imagen. Con varios archivos, cada piedra usa uno, elegido por su lugar: una piedra en un punto muestra siempre la misma imagen, en todos los diagramas.



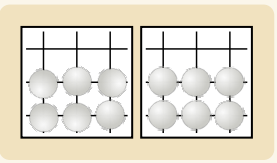
```
#goban(size: 5, partial: "B2:D4",
  stones(white: "B2:D3", image: (
    path("img/pgfgo-yunzi-white-1.png"),
    path("img/pgfgo-yunzi-white-2.png"))))
```

`stone.image.width` `auto` o una longitud — inicial `auto`

El ancho de la imagen; el alto mantiene la proporción. Con `auto`, el diámetro de la piedra: 2 × `radius` × `scale`.

`stone.image.rotate` un ángulo, `"random"` o `(mínimo, máximo)` — inicial `0deg`

Gira la imagen, en sentido antihorario. Con `"random"`, cada piedra se gira un ángulo distinto, elegido por su lugar: una piedra en un punto queda siempre igual, en todos los diagramas. Con un rango `(mínimo, máximo)`, el ángulo se elige igual, entre esos dos (`"random"` es `(0deg, 360deg)`). La sombra gira con la imagen, pero sigue cayendo hacia el mismo lado.

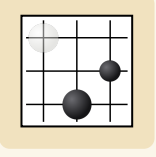


```
#let concha = path("img/pgfgo-yunzi-white-1.png")

#goban(size: 5, partial: "B2:D4",
  stones(white: "B2:D3", image: (file: concha,
    rotate: "random")))
#goban(size: 5, partial: "B2:D4",
  stones(white: "B2:D3", image: (file: concha,
    rotate: (-20deg, 20deg))))
```

`stone.image.opacity` un número de 0 a 1 — inicial `1`

La opacidad de la imagen. Se multiplica por la del relleno: así, un prisionero con imagen es translúcido.



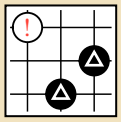
```
#let negra = path("img/pgfgo-yunzi-black.png")

#goban(size: 5, partial: "B2:D4",
  stones(black: "C2", image: negra),
  stones(black: "D3", image: (file: negra,
    width: 8pt)),
  stones(white: "B4", image: (opacity: 0.5,
    file: path("img/pgfgo-yunzi-white-1.png"))))
```

• Las demás opciones de las piedras

`mark` una forma o un diccionario — inicial `none`

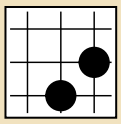
Una marca sobre las piedras, con la forma (`"triangle"`) o, con opciones, un diccionario con la forma en `shape` y las opciones de las marcas (pág. 64): `mark: (shape: "circle", line: (color: red))`. Es una opción de `stones`, `prisoners` y `moves`, no de la rama `stone`.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", mark: "triangle"),
  stones(white: "B4", mark: (shape: "custom",
    content: [!], label: (text: (fill: red)))))
```

`forget` `true` o `false` — inicial `false`

`resume` no trae estas piedras (pág. 80). `goban-view` las dibuja solo con `forgotten`. Es una opción de `stones`, `prisoners` y `moves` (donde también puede ser `"number"`, pág. 60), de `marks`, `arrows`, `lines` y `territory`.

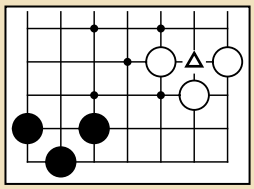


```
#gobanist-set(size: 5, partial: "B2:D4")

#let a = board(stones(black: "C2, D3"),
  stones(white: "B4", forget: true))
#goban(resume: a)
```

`shift` `(columnas, filas)` — inicial `(0, 0)`

Corre las coordenadas esas columnas a la derecha y esas filas hacia arriba, sean cuales sean las etiquetas del borde. Una intersección que queda fuera del tablero es un error. Es una opción de `stones`, `prisoners`, `moves` y `marks`.



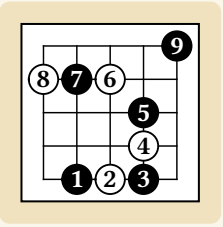
```
#let tigre = "A2, B1, C2"

#goban(size: 7, partial: "A1:G5",
  stones(black: tigre),
  stones(white: tigre, shift: (4, 2)),
  marks(triangle: "B2", shift: (4, 2)))
```

2 Jugadas

```
moves(jugadas, black: ..., white: ..., ..opciones)
```

Juega las intersecciones en el orden en que se escriben, alternando negras y blancas, y las numera. Sin jugador, siguen el turno: `moves("C3, D4")`; con uno, empiezan con él: `moves(white: "D2, B3")`. Recibe una sola secuencia. Una intersección puede repetirse (por ejemplo, para retomar un ko con `captures`). Acepta la sintaxis de `stones` (pág. 33) y juega cada elemento en su orden (pág. 37): un camino (`A1 -- A3 -- C3`) va de su primer punto al último, y un tramo, del primer extremo al segundo. Lo que no tiene un recorrido fijo (rectángulos, `r`, columnas y filas enteras, `*`) se juega igual. El `;` es un error: `moves` solo agrega jugadas.

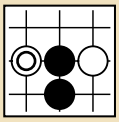


```
#goban(size: 5,
  moves("B1 -| D3, C:A 4, E5"))
```

Las opciones son las del estilo de la piedra (`fill`, `line`, `radius`, `shine`, `image`...), `mark`, `forget`, `shift` y las de esta sección: `numbers`, `label` y `from`.

Dentro de un mismo `goban`, la numeración y el turno siguen de un `moves` al siguiente. Cada `goban` empieza en la jugada 1 con negras; para continuar una partida en otro diagrama, se usan `from` y el jugador, o `resume` (pág. 80).

Las opciones de las piedras sirven, por ejemplo, para marcar la última jugada:

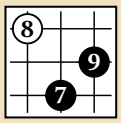


```
#goban(size: 5, partial: "B2:D4",
  moves("C3, D3, C2", numbers: false),
  moves("B3", numbers: false, mark: "circle"))
```

El estilo de las jugadas de los tres jugadores se cambia con `gobanist-set(move: ...)`, que es `gobanist-set(players: (all: (move: ...)))`.

`from` un número

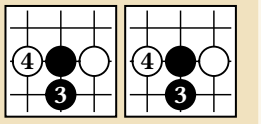
El número de la primera jugada de este `moves`. Las siguientes siguen desde ahí.



```
#goban(size: 5, partial: "B2:D4",
  moves("C2, B4", from: 7),
  moves("D3"))
```

`move.numbers` `true` o `false` inicial `true`

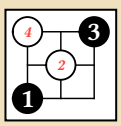
Si las jugadas llevan su número. Sin números, las jugadas igual cuentan: la siguiente sigue la numeración y el turno. En `moves(numbers: false)`, vale para esas jugadas; con `gobanist-set(move: (numbers: false))` o `goban(move: (numbers: false))`, para todas las que siguen.



```
#goban(size: 5, partial: "B2:D4",
  moves("C3, D3", numbers: false),
  moves("C2, B3"))
#goban(size: 5, partial: "B2:D4",
  move: (numbers: false),
  moves("C3, D3"),
  moves("C2, B3", numbers: true))
```

`move.label` un diccionario

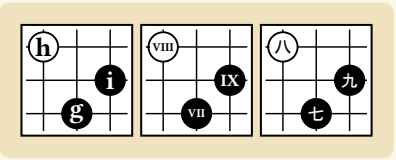
Los números de las jugadas.



```
#goban(size: 3,
  players: (white: (move: (label: (
    text: (fill: red, style: "italic"),
    size: 6pt)))),
  moves("A1, B2, C3, A3"))
```

`move.label.format` un patrón de `numbering` — inicial `"1"`

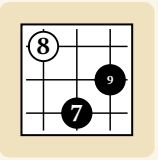
El formato de los números: un patrón de `numbering` de Typst, como `"1"`, `"a"`, `"A"`, `"i"`, `"I"` o `"一"` (los kanji necesitan una fuente que los tenga).



```
#goban(size: 5, partial: "B2:D4",
  move: (label: (format: "a")),
  moves("C2, B4", from: 7), moves("D3"))
#goban(size: 5, partial: "B2:D4",
  move: (label: (format: "I")),
  moves("C2, B4", from: 7), moves("D3"))
#goban(size: 5, partial: "B2:D4",
  move: (label: (format: "一",
    text: (font: "MS Gothic"))),
  moves("C2, B4", from: 7), moves("D3"))
```

`move.label.size` `auto` o una longitud — inicial `auto`

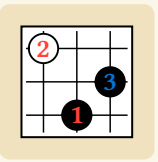
El tamaño de los números. Con `auto`, se ajusta según la cantidad de cifras.



```
#goban(size: 5, partial: "B2:D4",
  moves("C2, B4", from: 7),
  moves("D3", label: (size: 5pt)))
```

`move.label.text` un diccionario — inicial `(weight: "bold", fill: ...)`

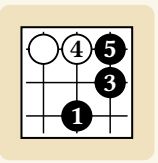
Las opciones de `text` de los números: la fuente, el peso, el color. El color es el contrario al de la piedra: blanco en las negras y negro en las blancas.



```
#goban(size: 5, partial: "B2:D4",
  moves("C2, B4", label: (text: (fill: red))),
  moves("D3", label: (text: (fill: blue))))
```

`forget` `true`, `false` o `"number"` — inicial `false`

En `moves`: con `true`, `resume` no trae estas jugadas, ni la piedra ni el número, y la numeración sigue desde la última jugada que trae. Con `"number"`, la jugada viene sin su número aunque se pida `numbers` (pág. 80); con `goban-view`, el número solo se ve con `forgotten`.



```
#gobanist-set(size: 5, partial: "B2:D4")

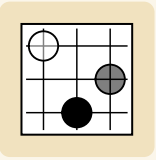
#let a = board(moves("C2"),
  moves("B4", forget: "number"),
  moves("D3"))
#goban(resume: (board: a, numbers: true),
  moves("C:D4"))
```

3 Prisioneros

Los prisioneros son piedras capturadas que siguen en el tablero: por defecto, translúcidas.

```
prisoners(coordenadas, black: ..., white: ..., neutral: ..., ..opciones)
```

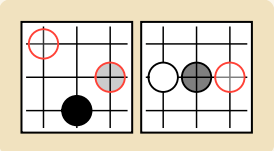
Como `stones`, con el estilo de los prisioneros: el de la piedra del jugador, con la rama `prisoner` encima.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2"),
  prisoners(black: "D3", white: "B4"))
```

`prisoner` opciones de piedra — inicial `(fill: (opacity: 0.5))`

El estilo de los prisioneros: opciones de la rama `stone`, que se ponen encima de las de la piedra del jugador. Con `gobanist-set(prisoner: ...)`, para los tres jugadores; con `gobanist-set(players: (white: (prisoner: ...)))`, para uno. Vale para los de `prisoners` y para las piedras capturadas con `captures: "fade"` (pág. 72). Las demás piedras del jugador no cambian.



```
#gobanist-set(size: 5, partial: "B2:D4")

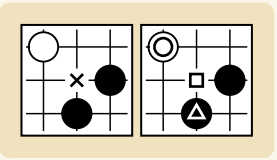
#goban(prisoner: (fill: (opacity: 0.2),
    line: (color: red)),
  stones(black: "C2"),
  prisoners(black: "D3", white: "B4"))
#goban(players: (white: (prisoner: (
    line: (color: red)))),
  stones(white: "B3"),
  prisoners(black: "C3", white: "D3"))
```

7 Marcas, flechas y líneas

Las marcas van sobre una intersección, con o sin piedra: formas (círculo, triángulo, cruz…), números en secuencia o cualquier contenido.

```
marks(coordenadas, forma: ..., ..opciones)
```

Pone marcas en las intersecciones, escritas con la sintaxis de coordenadas (pág. 33). La forma es la clave: `marks(triangle: "C3", circle: "E5")`. Sin forma, una cruz: `marks("C3")`. Un mismo `marks` puede llevar varias formas, y sus opciones valen para todas.



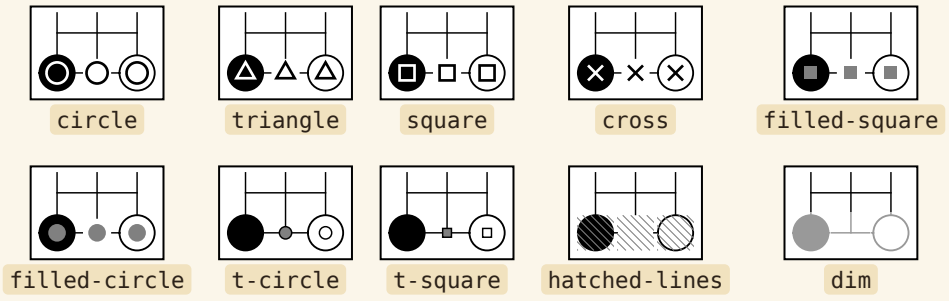
```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

#goban(b, marks("C3"))
#goban(b, marks(triangle: "C2",
  circle: "B4", square: "C3"))
```

Sin `player`, una marca sobre una piedra es del jugador de la piedra, con su estilo, para que se vea; sobre un punto vacío, es neutral. Sobre un punto vacío, la marca tapa la cuadrícula con el fondo del tablero (salvo `t-circle`, `t-square` y `dim`).

1 Las formas

Las formas del paquete son `circle`, `triangle`, `square`, `cross`, `filled-square`, `filled-circle`, `t-circle`, `t-square`, `hatched-lines`, `dim`, `sequence` y `custom`. Las de la opción `shapes` (pág. 68) se usan igual.



`circle`

Un círculo.

`triangle`

Un triángulo.

`square`

Un cuadrado.

`cross`

Una cruz. Es la forma de `marks` sin forma.

`filled-square`

Un cuadrado relleno, con el relleno de la marca (`fill`).

`filled-circle`

Un círculo relleno, con el relleno de la marca.

`t-circle`

Un círculo pequeño, con el relleno de las piedras del jugador y el borde de la cuadrícula. Es la marca del territorio de las negras.

`t-square`

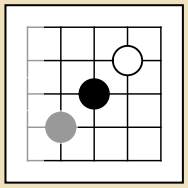
Como `t-circle`, pero cuadrado. Es la marca del territorio de las blancas y de los puntos neutrales.

`hatched-lines`

Un tramado de líneas, del color del relleno de la marca.

`dim`

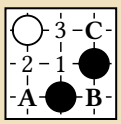
Atenúa el punto: lo cubre con el fondo del tablero, translúcido, sobre la piedra y la cuadrícula. Es lo que dibuja una partida para la propiedad `DD`.



```
#goban(size: 5,
  stones(black: "B2, C3", white: "D4"),
  marks(dim: "A1:5, B2"))
```

`sequence`

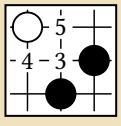
Numera las marcas, en el orden de las coordenadas (pág. 37), desde `from`. El formato, el tamaño y el color de los números son los de `label`, como el contenido de `custom`.




```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(sequence: "C3, B3, C4"),
  marks(sequence: "B2, D2, D4",
    label: (format: "A",
      text: (weight: "bold"))))
```

`from` un número inicial `1`

En `marks`, el número de la primera marca de `sequence`.



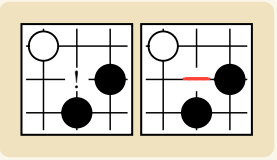

```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(sequence: "C3, B3, C4", from: 3))
```

`custom`

Una marca con cualquier contenido, el de `content`: un texto, un símbolo («!», «?») o un dibujo.

`content` un contenido

En `marks`, lo que dibuja `custom`. Es obligatorio con `custom`. Su tamaño se ajusta a la cantidad de caracteres, salvo con `label.size`.



```
#gobanist-set(size: 5, partial: "B2:D4")
#let b = board(stones(black: "C2, D3", white: "B4"))

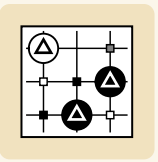
#goban(b, marks(custom: "C3",
  content: [!]))
#goban(b, marks(custom: "C3",
  content: box(stroke: red, radius: 2pt,
    inset: 1pt, text(fill: red)[…])))
```

2 Las opciones de las marcas

El estilo de las marcas es el de su jugador: `gobanist-set(mark: ...)` es `gobanist-set(players: (all: (mark: ...)))`, y las opciones de cada `marks` van encima.

`player` `auto`, `"black"`, `"white"` o `"neutral"` — inicial `auto`

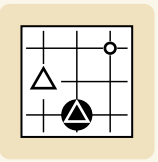
El jugador de las marcas, cuyo estilo toman. Con `auto`, el de la piedra que hay en el punto, o neutral si no hay ninguna.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(t-square: "B2, C3", player: "black"),
  marks(t-square: "D2, B3", player: "white"),
  marks(t-square: "D4", player: "neutral"),
  marks(triangle: "C2, D3, B4"))
```

`mark.scale` un número — inicial `1`

El tamaño de la marca respecto del de siempre: las formas, las imágenes y el texto.



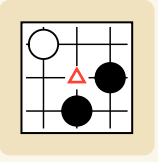
```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2"),
  marks(triangle: "C2, B3", scale: 1.5),
  marks(circle: "D4", scale: 0.6))
```

`mark.line` un diccionario

El trazo de la marca.

`mark.line.color` un color

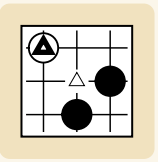
El color del trazo: blanco en las marcas de las negras y negro en las demás.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(triangle: "C3",
    line: (color: red)))
```

`mark.line.width` una longitud — inicial `1pt`

El grosor del trazo.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(triangle: "C3", line: (width: 0.3pt)),
  marks(triangle: "B4", line: (width: 2pt)))
```

`mark.line.opacity` un número de 0 a 1 — inicial `1`

La opacidad del trazo.

`mark.fill` `true`, `false` o un diccionario

El relleno de las marcas rellenas (`filled-square`, `filled-circle` y `hatched-lines`), y el `fill` de las formas propias (pág. 68). Con `false`, las marcas rellenas no se ven.

`mark.fill.color` un color o `none` inicial `luma(50%)`

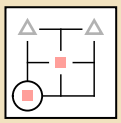
El color del relleno.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(filled-circle: "C3",
    fill: (color: red)))
```

`mark.fill.opacity` un número de 0 a 1 inicial `1`

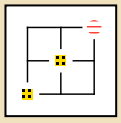
La opacidad del relleno.



```
#goban(size: 3,
  players: (white: (mark: (
    fill: (color: red, opacity: 0.5),
    line: (opacity: 0.3)))),
  stones(white: "A1"),
  marks(filled-square: "A1, B2",
    player: "white"),
  marks(triangle: "C3, A3", player: "white"))
```

`mark.fill.pattern` `none` o un `tiling` inicial `none`

Un patrón encima del color del relleno de `filled-square` y `filled-circle`, como `stone.fill.pattern` (pág. 54). `hatched-lines` y las formas propias usan solo el color.



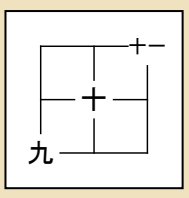
```
#goban(size: 3,
  marks(filled-square: "A1, B2", fill: (
    color: yellow, pattern: tiling(
      size: (2pt, 2pt), square(size: 1pt,
        fill: black)))),
  marks(filled-circle: "C3", fill: (
    color: none, pattern: tiling(
      size: (2pt, 2pt), line(length: 2pt,
        stroke: red)))))
```

`mark.label` un diccionario

El texto de las marcas que lo tienen: `sequence` y `custom`.

`mark.label.format` un patrón de `numbering` inicial `"1"`

El formato de los números de `sequence`: un patrón de `numbering`, como `"1"`, `"a"`, `"A"`, `"i"`, `"I"` o `"一"`. Los kanji necesitan una fuente que los tenga, en `label.text`.



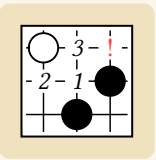

```
#goban(size: 3, scale: 1.6,
  marks(sequence: "A1, B2, C3", from: 9,
    label: (format: "一",
      text: (font: "MS Gothic"))))
```

`mark.label.size` `auto` o una longitud inicial `auto`

El tamaño del texto. Con `auto`, se ajusta según la cantidad de caracteres.

`mark.label.text` un diccionario

Las opciones de `text` del texto: la fuente, el estilo, el color. El color es el de `line.color`.



```
#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(sequence: "C3, B3, C4",
    label: (text: (style: "italic"))),
  marks(custom: "D4", content: [!],
    label: (text: (fill: red))))
```

`shift` `(columnas, filas)` inicial `(0, 0)`

En `marks`: corre las coordenadas, como en las piedras (pág. 60).

`forget` `true` o `false` inicial `false`

En `marks`, `arrows` y `lines`: `resume` no trae estas marcas, flechas o líneas, aunque se pida `marks` (pág. 80).

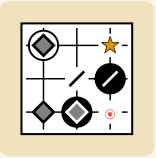
3 Formas propias

`shapes` un diccionario de formas inicial `(:)`

Formas de marca propias, por su nombre: `shapes: (rombo: ..., estrella: ...)`. Se usan como las del paquete: en `marks`, en la opción `mark` de las piedras, en `territory` y en `last-mark`. Una forma con el nombre de otra la reemplaza. Ninguna puede llamarse como una opción de `marks` (`line`, `fill`, `label`, `scale`, `player`, `from`, `content` o `forget`). Una forma es:

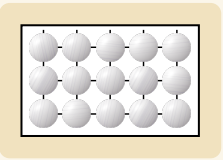
- una función: recibe un diccionario con `radius` (el radio de la piedra del jugador, con `scale`), `stroke` (el trazo de la marca), `fill` (su relleno, `none` si está apagado) y `color` (el color de sus líneas), y lo que devuelve se dibuja con su origen en el punto;
- un contenido, centrado en el punto;
- una imagen, `(image: archivo)` o `(image: (file: archivo, width: ...))`, centrada en el punto, sin los colores de la marca. `width` es su ancho (con `auto`, 1.2 radios de piedra), que cambia con `scale`. Con varios archivos, cada punto usa uno, siempre el mismo.

Sobre un punto vacío, una forma propia tapa la cuadrícula como las demás.



```
#gobanist-set(shapes: (
  rombo: m => polygon(fill: m.fill,
    stroke: m.stroke,
    (0pt, -0.7 * m.radius),
    (0.7 * m.radius, 0pt),
    (0pt, 0.7 * m.radius),
    (-0.7 * m.radius, 0pt)),
  barra: m => line(
    start: (-0.5 * m.radius, 0.5 * m.radius),
    end: (0.5 * m.radius, -0.5 * m.radius),
    stroke: m.stroke),
  estrella: (image: path("img/pgfgo-star.pdf")),
  ojo: text(fill: red, size: 8pt)[◉],
))

#goban(size: 5, partial: "B2:D4",
  stones(black: "C2, D3", white: "B4"),
  marks(rombo: "C2, B4, B2", barra: "D3, C3",
    estrella: "D4", ojo: "D2"))
```



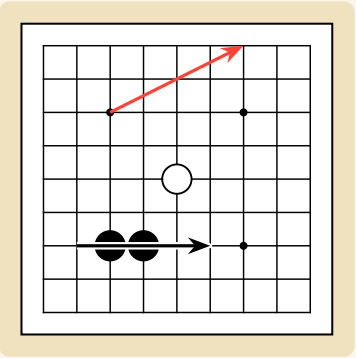
```
#goban(size: 5, partial: "A1:E3",
  shapes: (concha: (image: (width: 1em,
    file: range(1, 6).map(n => path(
      "img/pgfgo-shell-white-" + str(n)
        + ".png"))))),
  marks(concha: "A:E 1:3"))
```

4 Flechas y líneas

Las flechas y las líneas unen dos intersecciones. Se dibujan encima de las piedras, con un halo del color del fondo del tablero que las separa de las piedras y de la cuadrícula. Una partida dibuja las flechas (`AR`) y las líneas (`LN`) del archivo.

`arrows(..pares, ..opciones)`

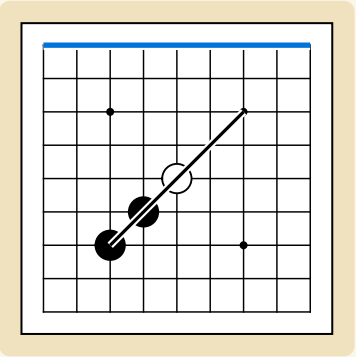
Dibuja flechas, cada una de una intersección a otra: `arrows(("B3", "F3"), ("C7", "G9"))`. Las opciones valen para todas.



```
#goban(stones(black: "C3, D3", white: "E5"),
  arrows(("B3", "F3")),
  arrows(("C7", "G9"), color: red,
    halo: false))
```

`lines(..pares, ..opciones)`

Dibuja líneas, cada una de una intersección a otra. Tiene las opciones de `arrows`, salvo `tip` y `tips`.



```
#goban(stones(black: "C3, D4", white: "E5"),
  lines(("C3", "G7")),
  lines(("A9", "J9"), color: blue, width: 2pt))
```

El estilo de todas las flechas se fija con `gobanist-set(arrow: ...)`, y el de todas las líneas, con `gobanist-set(line: ...)`.

`arrow.color` un color — inicial `black`

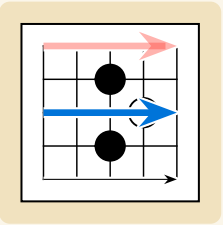
El color de la flecha.

`arrow.width` una longitud — inicial `1.2pt`

El grosor de la flecha. La punta crece con él.

`arrow.opacity` un número de 0 a 1 — inicial `1`

La opacidad de la flecha.



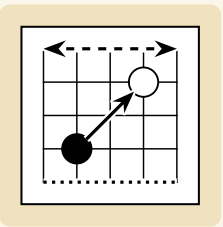
```
#goban(size: 5,
  stones(black: "C2, C4", white: "D3"),
  arrows(("A1", "E1"), width: 0.4pt),
  arrows(("A3", "E3"), color: blue,
    width: 2.5pt),
  arrows(("A5", "E5"), color: red,
    opacity: 0.4, width: 2.5pt))
```

`arrow.dash` `"solid"`, `"dashed"` o `"dotted"` — inicial `"solid"`

El trazo: continuo, con guiones o con puntos. El halo y la punta son siempre continuos.

`arrow.shorten` una longitud — inicial `0pt`

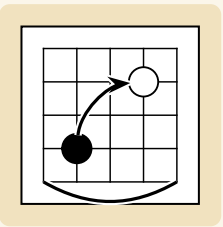
Acorta los dos extremos.



```
#goban(size: 5,
  stones(black: "B2", white: "D4"),
  arrows(("B2", "D4"), shorten: 5pt),
  arrows(("A5", "E5"), dash: "dashed",
    tips: "both"),
  lines(("A1", "E1"), dash: "dotted"))
```

`arrow.bend` un ángulo — inicial `0deg`

Dibuja un arco en lugar de una recta: la flecha sale girada ese ángulo hacia la izquierda y llega girada lo mismo. Con un ángulo negativo, se curva hacia la derecha.



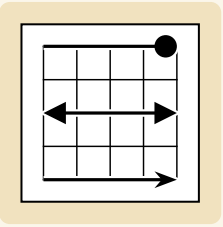
```
#goban(size: 5,
  stones(black: "B2", white: "D4"),
  arrows(("B2", "D4"), bend: 40deg,
    shorten: 5pt),
  lines(("A1", "E1"), bend: -30deg))
```

`arrow.tip` `"stealth"`, `"triangle"` o `"circle"` — inicial `"stealth"`

La punta de la flecha.

`arrow.tips` `"end"` o `"both"` — inicial `"end"`

Dónde va la punta: al final, o en los dos extremos.



```
#goban(size: 5,
  arrows(("A1", "E1")),
  arrows(("A3", "E3"), tip: "triangle",
    tips: "both"),
  arrows(("A5", "E5"), tip: "circle"))
```

`arrow.halo` `true`, `false` o un diccionario

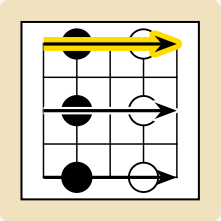
El halo alrededor de la flecha.

`arrow.halo.color` `auto` o un color — inicial `auto`

El color del halo. Con `auto`, el del fondo del tablero.

`arrow.halo.width` una longitud — inicial `0.8pt`

El ancho del halo a cada lado de la flecha.



```
#goban(size: 5,
  stones(black: "B1, B3, B5",
    white: "D1, D3, D5"),
  arrows(("A1", "E1"), halo: false),
  arrows(("A3", "E3")),
  arrows(("A5", "E5"), halo: (color: yellow,
    width: 2pt)))
```

Las líneas tienen las mismas opciones, en la rama `line`, salvo `tip` y `tips`:

`line.color` un color — inicial `black`

Como `arrow.color`.

`line.width` una longitud — inicial `1.2pt`

Como `arrow.width`.

`line.opacity` un número de 0 a 1 — inicial `1`

Como `arrow.opacity`.

`line.dash` `"solid"`, `"dashed"` o `"dotted"` — inicial `"solid"`

Como `arrow.dash`.

`line.shorten` una longitud — inicial `0pt`

Como `arrow.shorten`.

`line.bend` un ángulo — inicial `0deg`

Como `arrow.bend`.

`line.halo` `true`, `false` o un diccionario

Como `arrow.halo`, con `color` y `width`.

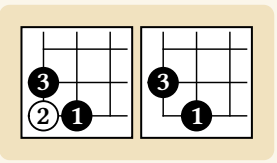
8 El juego: capturas, notas y territorio

Un `goban` puede seguir las reglas del juego: retirar las piedras capturadas, anotar las jugadas que caen en puntos ya jugados y contar el territorio. Lo hace el kernel (pág. 31), que juega el tablero entero al dibujarlo.

1 Capturas

`captures` `none`, `"remove"` o `"fade"` — inicial `none`

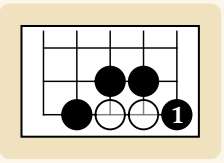
Las capturas. Cuando una jugada deja sin libertades a un grupo rival, ese grupo se retira del tablero; si deja sin libertades a su propio grupo (un suicidio), se retira ese grupo. Las piedras de `stones` también cuentan; las neutrales y los prisioneros, no. Una jugada fuera de un tablero parcial también captura. Una jugada en un punto ocupado es un error, porque en una partida no es posible (salvo en `sgf-diagrams` y `sgf-problem`, pág. 92). Con `none`, no hay capturas: las jugadas se dibujan unas sobre otras.



```
#gobanist-set(size: 5, partial: "A1:C3")

#goban(moves("B1, A1, A2"))
// 3 captura a 2.
#goban(captures: "remove",
  moves("B1, A1, A2"))
```

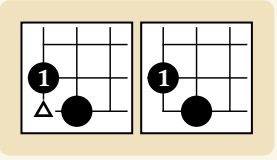
Con `"remove"`, las piedras capturadas desaparecen. Con `"fade"`, quedan en el tablero, dibujadas como prisioneros (con el estilo de `prisoner`, pág. 62).



```
#goban(size: 5, partial: "A1:E3",
  captures: "fade",
  stones(white: "C1, D1",
    black: "B1, C2, D2"),
  moves("E1"))
```

`captured-marks` `"keep"` o `"remove"` — inicial `"keep"`

Qué pasa con las marcas que están sobre una piedra cuando la capturan. Con `"keep"`, quedan sobre el punto vacío. Con `"remove"`, se van con la piedra, también con `captures: "fade"`; las secuencias conservan su numeración. Las marcas sobre puntos vacíos no cambian.

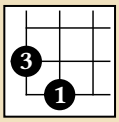


```
#gobanist-set(size: 5, partial: "A1:C3",
  captures: "remove")

#goban(
  stones(white: "A1", black: "B1"),
  marks(triangle: "A1"), moves("A2"))
#goban(captured-marks: "remove",
  stones(white: "A1", black: "B1"),
  marks(triangle: "A1"), moves("A2"))
```

`goban-prisoners(player)`

Cuántos prisioneros tiene el jugador (`"black"` o `"white"`) en el último `goban`: las piedras del rival que capturaron sus jugadas.



Negras: 1. Blancas: 0.

```
#goban(size: 5, partial: "A1:C3",
  captures: "remove",
  moves("B1, A1, A2"))

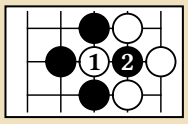
Negras: #goban-prisoners("black").
Blancas: #goban-prisoners("white").
```

2 Jugadas en puntos ya jugados

En los diagramas numerados, cuando una jugada cae en un punto donde ya se jugó (por ejemplo, al retomar un ko), los libros no la dibujan: el punto sigue mostrando la primera piedra, y debajo del diagrama se anota, por ejemplo, «3 en 1».

`notes` `true`, `false` o un diccionario

Una jugada numerada en un punto donde ya hubo una jugada numerada del mismo `goban` no se dibuja: el punto sigue mostrando la primera, aunque haya sido capturada, y se anota «N en M». Solo cuentan las jugadas numeradas que se ven en el tablero. Un pase de una partida que se numeraría se anota «N: pasa», en orden con las demás notas; los pases con los que termina la partida no se anotan.



3 en 1

```
#goban(size: 5, partial: "A2:E4",
  notes: true,
  stones(black: "B3, C2, C4",
    white: "D2, D4, E3"),
  moves(white: "C3"), moves("D3"),
  moves(white: "C3", from: 3))

#goban-notes()
```

`notes.at` `auto` o un texto — inicial `auto`

El texto entre los dos números. Con `auto`, el del idioma (`at` en `texts`, pág. 31).

`notes.separator` un texto o un contenido — inicial `", "`

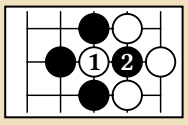
El texto entre las notas.

`notes.text` un diccionario — inicial `(:)`

Las opciones de `text` de las notas, en `goban-notes` y debajo de los diagramas de `sgf-diagrams` y `sgf-problem` (encima de las del epígrafe).

```
goban-notes(at: auto, separator: auto)
```

Las notas del último `goban`, separadas por `separator`. Sin notas, nada. Con `auto`, `at` y `separator` son los de la rama `notes`.



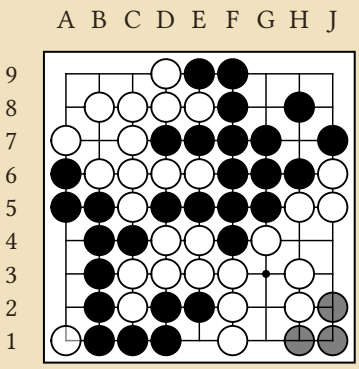
3 sobre 1; 4 sobre 2

```
#goban(size: 5, partial: "A2:E4",
  notes: true, captures: "remove",
  stones(black: "B3, C2, C4",
    white: "D2, D4, E3"),
  moves(white: "C3"), moves("D3"),
  moves(white: "C3", from: 3), moves("D3"))

#goban-notes(at: "sobre", separator: "; ")
```

3 Territorio

Al final de la partida, el territorio puede marcarse a mano con `marks` (pág. 64) o con `territory`, que además lo cuenta. En los ejemplos, `fin` es esta posición, con sus prisioneros:



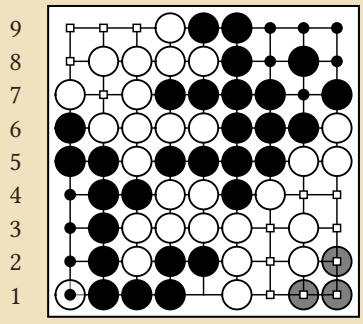

```
#let fin = board(
  stones(black: "A6 |- B5 |- D1 |- E2, C4,
      E9 -| F4, D5 -| G7 -- D7, H6, H8, J7",
    white: "E8 -| C5, B:E6, A7, B8, D9,
      J6 |- H5, H2:3, F1 |- C3, C2, D:E4, G4"),
  prisoners(black: "H1 -| J2", white: "A1"))

#goban(label: (at: (top, left)), resume: fin)
```

`territory(polígono, black: ..., white: ..., neutral: ..., ..opciones)`

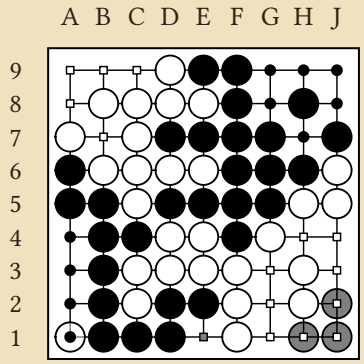
Marca como territorio los puntos del polígono que tiene esos vértices (en orden, con la sintaxis de coordenadas), incluido su borde. No marca los puntos que tienen una piedra viva: los prisioneros cuentan como territorio. Un vértice solo marca un punto, y dos, un tramo. El jugador es la clave: `territory(black: "A1, A5, B5, B1")`; sin jugador, el polígono es neutral.

El territorio se dibuja al terminar el `goban`. Cada punto queda con el último `territory` que lo marcó, con el jugador y las opciones de ese `territory` (así, `territory(neutral: "A1")` después de otro `territory` vuelve neutral a A1), y no se dibuja ni se cuenta si al final tiene una piedra viva.




```
#goban(label: (at: (top, left)),
  resume: fin,
  territory(black: "A1, A5, B5, B1"),
  territory(black: "F9, F7, G7, G6, J6, J9"),
  territory(white: "F1, F4, G4, G5, J5, J1"),
  territory(white: "A9, A6, C6, C8, D8, D9"))
```

Con `auto` en lugar de un polígono, busca solo el territorio: cada región de puntos sin piedras vivas es del color que la rodea. Si la tocan los dos colores (dame, seki) o ninguno, es neutral. Los prisioneros cuentan como piedras muertas. Sin jugador, `territory(auto)` marca el territorio de los dos; con `territory(black: auto)`, solo el de ese jugador.



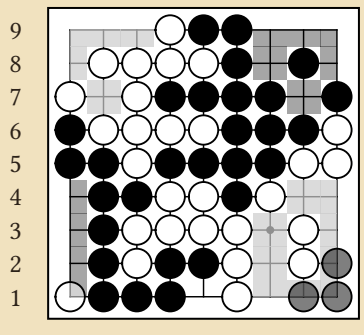

```
#goban(label: (at: (top, left)),
  resume: fin,
  territory(auto, dame: true))
```

`auto` solo mira qué color toca cada región; no sabe leer la posición. Por eso:

- Es para partidas terminadas. En una posición abierta, casi todo el tablero toca los dos colores y es neutral; con piedras de un solo color, todo el tablero es suyo. Si un borde tiene un hueco, el territorio de adentro se junta con el dame y también queda neutral.
- No sabe qué piedras están muertas: hay que ponerlas con `prisoners`. Una piedra muerta puesta con `stones` cuenta como viva, y la región que toca queda neutral.
- Los ojos de un grupo en seki cuentan como territorio (en las reglas japonesas no se cuentan). Pueden volverse neutrales después, con un `territory(neutral: ...)` para cada uno.
- Las piedras neutrales ocupan su punto, pero no son de nadie: una región que solo toca piedras neutrales es neutral.
- Busca las regiones con las piedras que hay donde está el `territory`, así que conviene ponerlo al final del `goban`.
- Con `partial: auto`, el tablero crece hasta mostrar todo el territorio (y el dame, con `dame`).

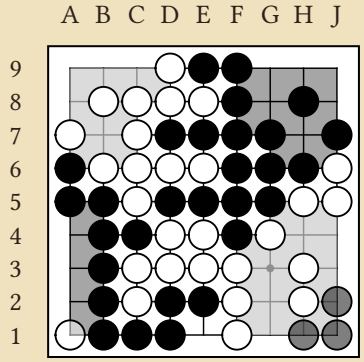
`territory.draw` `"marks"`, `"fill"` o `"region"` — inicial `"marks"`

Cómo se dibuja el territorio. Con `"marks"`, cada punto lleva la marca de territorio del jugador. Con `"fill"`, cada punto se rellena debajo de las piedras.




```
#goban(label: (at: (top, left)),
  resume: fin,
  territory(auto, draw: "fill"))
```

Con `"region"`, el territorio de cada color es un solo polígono: se pinta cada casilla de la cuadrícula que tiene alguna esquina en el territorio (las piedras muertas cuentan como territorio). Sus lados van por las líneas del tablero, sus vértices están siempre en intersecciones, y llega hasta el borde del tablero. Dos zonas que se tocan por una sola esquina quedan separadas. Todos los `territory` de un color en el mismo `goban` (también el territorio `TB` y `TW` de una partida) forman una sola región, que se dibuja con el estilo del último. Los puntos neutrales se rellenan uno por uno, como con `"fill"`. El contorno y sus vértices pueden dibujarse encima, con `line` y `vertices`, y `goban-region` da sus propiedades.




```
#goban(label: (at: (top, left)),
  resume: fin,
  territory(auto, draw: "region"))
```

Es una opción de `territory`, y con `gobanist-set(territory: (draw: ...))`, de todos.

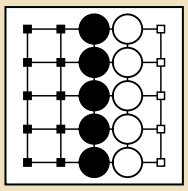
`territory.dame` `true` o `false` — inicial `false`

Con `auto`, marca también los puntos neutrales.

Las demás opciones de `territory` (`mark`, `fill`, `line` y `vertices`) cambian el estilo del territorio del jugador solo para ese `territory`. El estilo de cada jugador está en su rama `territory`:

`players.black.territory.mark` una forma o un diccionario — inicial `"t-circle"`

La marca de cada punto, con `draw: "marks"`: `"t-circle"` para las negras y `"t-square"` para las blancas y los puntos neutrales. Con opciones, un diccionario con la forma en `shape`, como `mark` de las piedras.



```
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    mark: "t-square"),
  territory(white: "E1, E5"))
```

`players.black.territory.fill` `true`, `false` o un diccionario

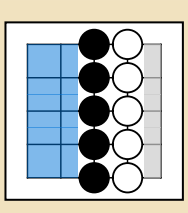
El relleno, con `draw: "fill"` y `draw: "region"`. Con `false`, sin relleno: con `"region"` quedan solo el contorno o los vértices.

`players.black.territory.fill.color` un color o `none` — inicial `black`

El color del relleno: negro para las negras, `luma(80%)` para las blancas y `luma(50%)` para los puntos neutrales. Con `none`, sin color.

`players.black.territory.fill.opacity` un número de 0 a 1 — inicial `0.35`

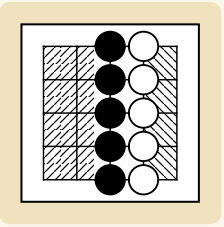
La opacidad del relleno: `0.35` para las negras y los puntos neutrales, `0.7` para las blancas.



```
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "fill",
    fill: (color: blue, opacity: 0.5)),
  territory(white: "E1, E5", draw: "fill"))
```

`players.black.territory.fill.pattern` `none` o un `tiling` — inicial `none`

Un patrón encima del color del relleno, como `stone.fill.pattern` (pág. 54). Con `fill.color: none`, queda solo el patrón.

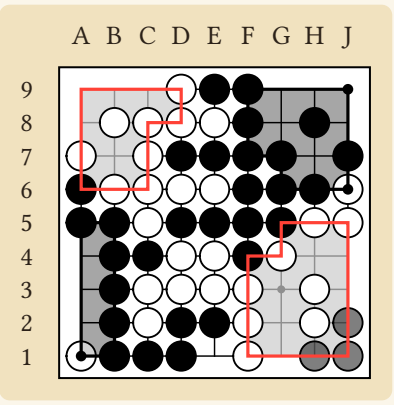


```
#let rayas(a, b) = tiling(size: (3pt, 3pt),
  line(start: (0pt, a), end: (3pt, b),
    stroke: 0.4pt))

#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "fill", fill: (color: none,
      pattern: rayas(3pt, 0pt))),
  territory(white: "E1:5", draw: "region",
    fill: (color: none,
      pattern: rayas(0pt, 3pt))))
```

`players.black.territory.line` `true`, `false` o un diccionario

El contorno de la región, con `draw: "region"`.




```
#goban(label: (at: (top, left)),
  resume: fin,
  territory(black: auto, draw: "region",
    line: true, vertices: true),
  territory(white: auto, draw: "region",
    line: (on: true, color: red)))
```

`players.black.territory.line.color` un color — inicial `black`

El color del contorno: negro para las negras y `luma(40%)` para las blancas.

`players.black.territory.line.width` una longitud — inicial `1.2pt`

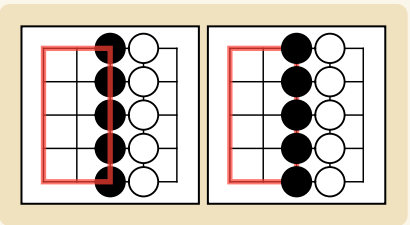
El grosor del contorno.

`players.black.territory.line.opacity` un número de 0 a 1 — inicial `1`

La opacidad del contorno y de los vértices.

`players.black.territory.line.above` `true` o `false` — inicial `true`

Con `true`, el contorno y los vértices se dibujan encima de las piedras y de las marcas; con `false`, debajo de las piedras, junto con el relleno.



```
#let contorno = (on: true, color: red,
  width: 2pt, opacity: 0.7)

#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "region", fill: false,
    line: contorno))
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "region", fill: false,
    line: contorno + (above: false)))
```

`players.black.territory.vertices` `true`, `false` o un diccionario

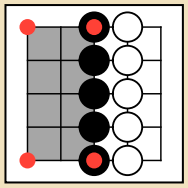
Puntos en los vértices del contorno, con `draw: "region"`, en la misma capa que el contorno.

`players.black.territory.vertices.radius` una longitud — inicial `2pt`

El radio de los puntos.

`players.black.territory.vertices.color` `auto` o un color — inicial `auto`

El color de los puntos. Con `auto`, el del contorno.

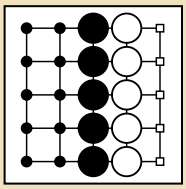


```
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(black: "A1, A5, B5, B1",
    draw: "region", vertices: (on: true,
      radius: 3pt, color: red)))
```

4 Puntaje

`goban-score(player, komi: auto)`

El puntaje del jugador (`"black"` o `"white"`) en el último `goban`, con las reglas japonesas: los puntos de su territorio, más las piedras muertas del rival que hay en él, más las piedras que capturó (con `captures`), más el komi para las blancas. Un punto marcado dos veces cuenta una vez. Con `"neutral"`, la cantidad de puntos neutrales marcados. Con `auto`, el komi es el de `score.komi`.



Negras: 10. Blancas: 11.5.

```
#goban(size: 5,
  stones(black: "C1:5", white: "D1:5"),
  territory(auto))

Negras: #goban-score("black").
Blancas: #goban-score("white", komi: 6.5).
```

`score.komi` un número — inicial `0`

El komi que `goban-score` le suma a las blancas: `gobanist-set(score: (komi: 6.5))`.

`goban-region(player, property, n: none)`

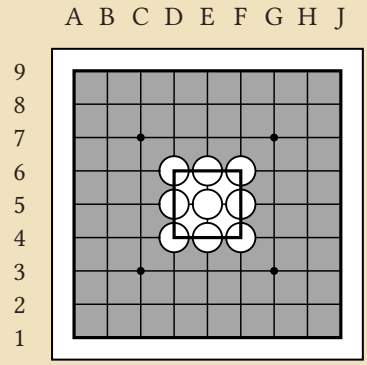
Una propiedad de la región (`draw: "region"`) del jugador (`"black"` o `"white"`) en el último `goban`, como texto. Las propiedades son del contorno completo, aunque el tablero sea parcial. Sin región, los números son 0. Las propiedades son:

- `"polygons"`: la cantidad de contornos, exteriores y huecos.
- `"outlines"`: la cantidad de contornos exteriores, uno por cada zona separada.
- `"holes"`: la cantidad de huecos.
- `"vertices"`: la cantidad de vértices de todos los contornos.
- `"area"`: la cantidad de casillas pintadas.
- `"perimeter"`: el largo de todos los contornos, en casillas.
- `"polygon"`, con `n`: los vértices del contorno `n` (desde 1), como coordenadas separadas por comas (`"A1, D1, D4, A4"`). Vacío si no hay tantos contornos.

Cada contorno empieza en su vértice más bajo y, entre esos, en el de más a la izquierda. Los exteriores van en sentido antihorario y los huecos, en sentido horario. Los contornos están ordenados por ese primer vértice, de abajo hacia arriba.

```
#goban(label: (at: (top, left)),
  stones(white: "D4:F6"),
  territory(black: "B2, B8, H8, H2",
    draw: "region", line: true))

Contornos: #goban-region("black", "polygons")
(#goban-region("black", "holes") hueco).
Casillas: #goban-region("black", "area").
El hueco:
#goban-region("black", "polygon", n: 2).
```



Contornos: 2 (1 hueco). Casillas: 60. El hueco: D4, D6, F6, F4.

`goban-prisoners`, `goban-notes`, `goban-score` y `goban-region` hablan del último tablero dibujado: el de un `goban` o el último diagrama de `sgf-diagrams`, `sgf-problem` o `goban-layout`. Las vistas (`goban-view`) no cuentan. Devuelven contenido: lo que dicen se sabe al dibujar.

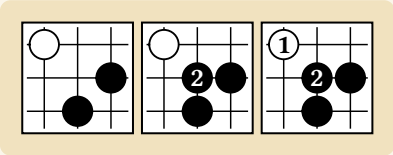
9 Tableros como valores y vistas

1 Seguir desde un tablero

Un tablero puede ser un valor: `board` (pág. 40) recibe lo mismo que `goban` y no dibuja nada. Se guarda con `let`, y se usa de dos maneras:

- primero en un `goban` o en otro `board`: `goban(base, moves("C3"))` es el mismo tablero, con todo lo que tiene (sus opciones, sus marcas, sus números), más lo que se agrega;
- con `resume`: otro tablero empieza con ese, como quedó al terminar, con las capturas resueltas; trae solo lo que se pide (por defecto, las piedras), y la numeración y el turno siguen desde ahí.

`goban-view` muestra una parte en medio del texto (pág. 81).




```
#gobanist-set(size: 5, partial: "B2:D4")

#let tablero1 = board(stones(black: "C2, D3"),
  moves(white: "B4"))

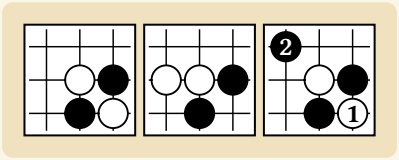
#goban(resume: tablero1)
// Sigue con la 2 (negras).
#goban(resume: tablero1, moves("C3"))
#goban(resume: (board: tablero1,
  numbers: true), moves("C3"))
```

`resume` un tablero o `(board: ..., numbers: ..., marks: ..., territory: ...)`

Empieza el tablero con el tablero `board`, como quedó al terminar: con las capturas resueltas (si ese tablero tenía `captures`, este también, salvo que dé las suyas). La numeración y el turno siguen desde ahí. Un tablero solo, `resume: base`, trae solo las piedras, sin números, marcas ni territorio; en un diccionario, `numbers`, `marks` y `territory` piden el resto.

El tamaño es el del tablero de `resume`, salvo que este dé el suyo (`size`, o una partida); uno puesto con `gobanist-set` no cuenta. Las demás opciones de ese tablero (`partial`, las coordenadas, el estilo) no vienen: se dan en cada tablero.

Un `board` con `resume` tiene lo que trajo, tal como se dibujó, más lo suyo, y el tablero de `resume` no cambia: así se arman variantes.




```
#gobanist-set(size: 5, partial: "B2:D4")

#let base = board(stones(black: "C2, D3",
  white: "C3"))
#let variante1 = board(resume: base,
  moves(white: "D2"))
#let variante2 = board(resume: base,
  moves(white: "B3"))

#goban(resume: variante1)
#goban(resume: variante2)
#goban(resume: (board: variante1,
  numbers: true), moves("B4"))
```

`resume.numbers` `true` o `false` — inicial `false`

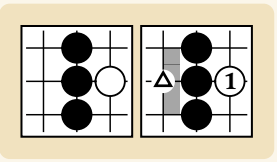
Trae también los números de las jugadas.

`resume.marks` `true` o `false` inicial `false`

Trae también las marcas, las flechas y las líneas.

`resume.territory` `true` o `false` inicial `false`

Trae también el territorio.



```
#gobanist-set(size: 5, partial: "B2:D4")

#let guardado = board(stones(black: "C2:4"),
  moves(white: "D3"), marks(triangle: "B3"),
  territory(black: "B2, B4", draw: "fill"))

#goban(resume: guardado)
#goban(resume: (board: guardado,
  numbers: true, marks: true,
  territory: true))
```

Para que `resume` no traiga una piedra, una jugada, una marca, una flecha, una línea o un territorio, se dibuja con la opción `forget: true` (`stones(white: "C3", forget: true)`, `marks(circle: "A1", forget: true)`, ...); con `forget: "number"`, una jugada viene sin su número aunque se pida `numbers` (pág. 60). Lo olvidado sigue en el tablero: `goban-view` lo muestra con `forgotten`.

Como un tablero es un valor, puede guardarse en un módulo e importarse en otro documento: `#import "tableros.typ": ko`. Así se usa el mismo tablero en un artículo y en sus diapositivas (pág. 25).

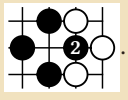
2 Vistas en el texto

`goban-view(tablero, parte, ..opciones)`

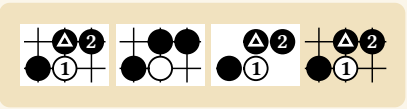
Dibuja la intersección (`"C3"`) o el rectángulo (`"B2:E4"`, escrito como en `partial`) de un tablero (un `board(...)` o una partida), como quedó al terminar, con las capturas resueltas, en medio del texto. Con varias partes separadas por comas (`"F15, G15, C3:E5"`), una vista por parte, unidas con `view.separator` y `view.last`. Por defecto se ve todo lo que dibujó el tablero en esa parte (piedras, números de las jugadas, marcas, flechas, líneas y territorio), con su estilo, a la escala de `view.goban` y centrado a la altura de las minúsculas. Las opciones son las de la rama `view`.

```
#let tablero1 = board(size: 5,
  captures: "remove",
  stones(black: "B3, C2, C4",
    white: "D2, D4, E3"),
  moves(white: "C3, D3"))

La negra en D3 #goban-view(tablero1, "D3")
captura la blanca de C3
#goban-view(tablero1, "C3"). Así quedó el ko:
#goban-view(tablero1, "B2:E4").
```

La negra en D3 ❷ captura la blanca de C3 ┼. Así quedó el ko:  .

La vista se resuelve con las opciones globales del lugar donde está, como un `goban`; encima van las del tablero y, después, las de `view.goban`. Como el tablero es un valor, la vista puede ir antes o después del diagrama que lo dibuja. Una vista no cambia lo que dicen `goban-prisoners`, `goban-score`, `goban-notes` ni `goban-region`.



```
#let tablero2 = board(size: 5,
  stones(black: "B2, C3"),
  moves(white: "C2, D3"),
  marks(triangle: "C3"))

#goban-view(tablero2, "B2:D3")
#goban-view(tablero2, "B2:D3",
  marks: false, numbers: false)
#goban-view(tablero2, "B2:D3",
  goban: (grid: false))
#goban-view(tablero2, "B2:D3",
  goban: (background: false))
```

`view` un diccionario

Las opciones de las vistas: en `goban-view`, para esa vista, y con `gobanist-set(view: ...)`, para todas.

`view.marks` `true` o `false` inicial `true`

Con `false`, sin las marcas.

`view.arrows` `true` o `false` inicial `true`

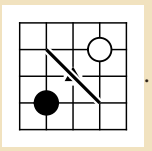
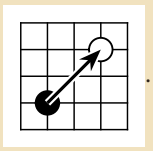
Con `false`, sin las flechas.

`view.lines` `true` o `false` inicial `true`

Con `false`, sin las líneas.

```
#let flechas = board(size: 5,
  stones(black: "B2", white: "D4"),
  arrows(("B2", "D4")), lines(("B4", "D2")),
  marks(triangle: "C3"))

Sin flechas:
#goban-view(flechas, "A1:E5", arrows: false).
Sin líneas ni marcas:
#goban-view(flechas, "A1:E5", lines: false,
  marks: false).
```

Sin flechas:  . Sin líneas ni marcas:  .

`view.numbers` `true` o `false` inicial `true`

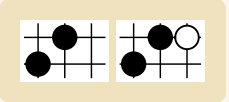
Con `false`, sin los números de las jugadas.

`view.territory` `true` o `false` inicial `true`

Con `false`, sin el territorio.

`view.forgotten` `true` o `false` inicial `false`

Dibuja también lo que tiene `forget`.



```
#let tablero3 = board(size: 5,
  stones(black: "B2, C3"),
  stones(white: "D3", forget: true))

#goban-view(tablero3, "B2:D3")
#goban-view(tablero3, "B2:D3",
  forgotten: true)
```

`view.raise` `auto` o una longitud — inicial `auto`

A qué altura sobre la línea de base del texto queda el centro de la vista. Con `auto`, la mitad del alto de las minúsculas.

```
#let b = board(size: 5, stones(black: "C3"))

Texto #goban-view(b, "C3", raise: 0pt)
texto #goban-view(b, "C3")
texto #goban-view(b, "C3", raise: 0.8em).
```

Texto ● texto ● texto ●.

`view.separator` un texto o un contenido — inicial `", "`

Lo que va entre las vistas de varias partes.

`view.last` `auto`, un texto o un contenido — inicial `auto`

Lo que va antes de la última vista, en lugar de `separator`; con dos partes, va solo esto. Con `auto`, el texto `and` del idioma entre espacios: « y ». Los espacios son parte del texto.

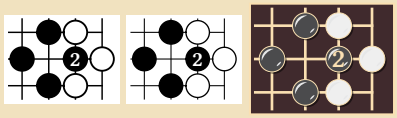
```
#let b = board(size: 5,
  stones(black: "B3, C2, C4", white: "C3"))

Las piedras #goban-view(b, "B3, C2, C4")
rodean a C3.
#goban-view(b, "B3, C2, C4",
  separator: "; ", last: " o ").
```

Las piedras ●, ● y ● rodean a C3. ●; ● o ●.

`view.goban` opciones de tablero — inicial `(scale: 0.8, background: (line: false))`

Las opciones del tablero de la vista, encima de las suyas: el estilo (`profile`, `players`, `stone`, `mark`, `grid`, `background`, `label`, ...) y la escala. Inicialmente, el fondo va sin borde, aunque el tablero lo tenga; un `profile` vuelve el estilo al de su perfil, con su borde. No pueden ser las que cambian la partida o la parte que se ve: `size`, `use-i`, `partial`, `captures`, `captured-marks`, `territory` ni `sgf`.



```
#let ko = board(size: 5, captures: "remove",
  stones(black: "B3, C2, C4",
    white: "D2, D4, E3"),
  moves(white: "C3, D3"))

#goban-view(ko, "B2:E4")
#goban-view(ko, "B2:E4",
  goban: (profile: "simple"))
#goban-view(ko, "B2:E4",
  goban: (profile: "wain", scale: 1))
```

3 Una piedra en el texto

```
goban-stone(jugador, number: none, mark: none, prisoner: false, ..estilo)
```

Dibuja una piedra sola en el texto, sin tablero: de `"black"`, `"white"` o, sin jugador (o con `"neutral"`), neutral. `mark` es una marca encima, como en `stones`; con `number`, la piedra es una jugada con ese número; con `prisoner: true`, un prisionero. Las demás opciones son las del estilo de la piedra (`fill`, `line`, `radius`, ...). Lleva el estilo del lugar donde está y, encima, el de las vistas (`view.goban`,

`view.raise`): queda del tamaño y a la altura de la vista de una intersección, sin grilla ni fondo. No cambia lo que dicen `goban-prisoners`, `goban-score`, `goban-notes` ni `goban-region`.

```
Negra #goban-stone("black"), blanca
#goban-stone("white"), neutral #goban-stone().
#goban-stone("black", mark: "triangle")
#goban-stone("white", number: 12)
#goban-stone("black", prisoner: true)
```

Negra ●, blanca ○, neutral ●. ▲ ⑫ ●

10 Partidas en SGF

SGF es el formato que usan casi todos los programas y servidores de go para guardar partidas. gobanist las lee con el kernel (pág. 31): una partida en un tablero, sus datos y comentarios, la partida entera en diagramas y los problemas con sus soluciones.

En los ejemplos, `partida` es `partida.sgf`, una partida corta de 9×9:

```
(;GM[1]FF[4]SZ[9]PB[Negras]PW[Blancas]C[Una partida corta.]
;B[ee];W[ef];B[df];W[ce];B[ff];W[gd];B[eg]TE[1]C[Negras capturan.];W[dd];B[cf];W[fd])
```

y `variantes` es `variantes.sgf`, una partida de 7×7 con dos bifurcaciones, marcas y `MN[10]` en la cuarta jugada de la línea principal:

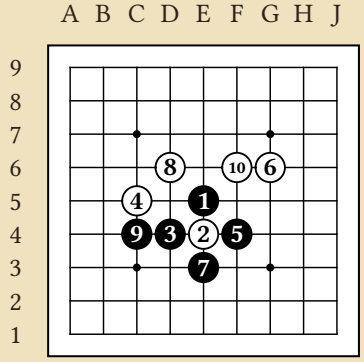
```
(;GM[1]FF[4]SZ[7]PB[Ana]PW[Bruno]RE[B+R]KM[6.5]DT[2026-10-13]
AB[dd]
;B[cc]TR[dd]
(;W[ee]SQ[cc]CR[ee]MA[aa]LB[gg:A][ag:B]
 ;B[ec]
 (;W[ce]MN[10]
  ;B[cf])
 (;W[fe]))
(;W[ce]
 ;B[ec]TR[ec]))
```

1 Leer una partida

```
sgf-read(source, game: 1, charset: auto, origin: top + left, transpose: false, syntax: auto)
```

Lee una partida SGF. `source` es un archivo (`path("partida.sgf")`, relativo al documento que lo nombra), su contenido (`bytes`) o el texto SGF (una cadena). Devuelve la partida como un valor, que se guarda con `let` y se usa en todas partes: en un tablero, en `sgf-info`, `sgf-comment`, `sgf-diagrams`, `sgf-problem`, `goban-layout` y `goban-view`.

En un tablero, la partida es un elemento, como `stones` o `moves`: dibuja la línea principal, y el tablero toma el tamaño de su `SZ` (otro `size` es un error).




```
#let partida = sgf-read(path("partida.sgf"))

#goban(label: (at: (top, left)), partida)
```

`game` un número — inicial `1`

En `sgf-read`: un archivo SGF puede tener varias partidas (una colección); `game` elige cuál se lee. Una que no existe es un error.

```
#let tercera = sgf-read(path("coleccion.sgf"), game: 3)
```

`charset` `auto` o un nombre — inicial `auto`

En `sgf-read`: la codificación del archivo, si no es UTF-8. Con `auto`, la que dice el archivo con `CA` o, si no dice ninguna, Latin-1 (la de SGF por defecto). Se leen Latin-1 (y windows-1252), GBK (GB2312, GB18030), Big5, Shift_JIS, EUC-JP y EUC-KR; los nombres no distinguen mayúsculas ni signos (`"shift-jis"` y `"EUC_KR"` también valen). Un archivo que es UTF-8 válido se lee siempre como UTF-8, aunque `CA` diga otra cosa; uno en UTF-16 (con la marca de orden de bytes al principio) también se lee. Otra codificación es un error: hay que convertir el archivo a UTF-8.

Muchos archivos de servidores asiáticos no dicen su codificación: `sgf-read(path("qipu.sgf"), charset: "GBK")`. Para escribir sus comentarios hace falta una fuente con esos caracteres.

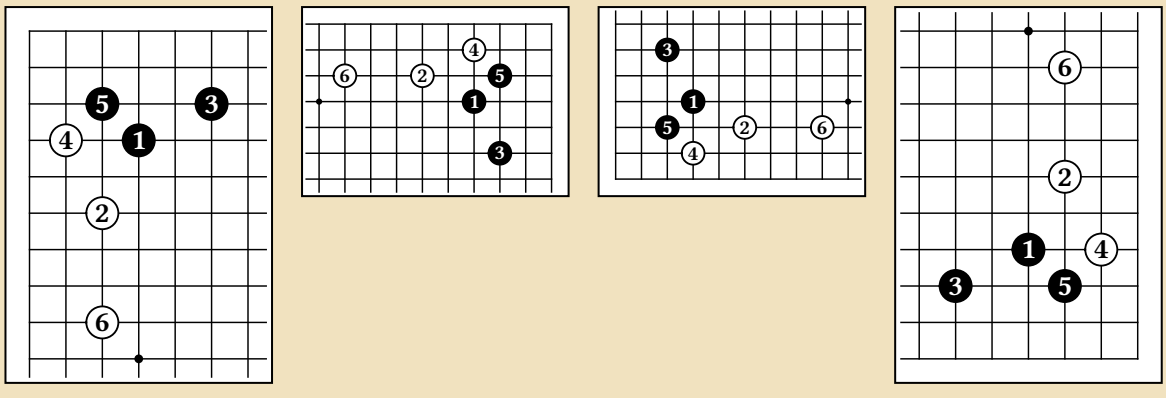
`origin` una esquina — inicial `top + left`

En `sgf-read`: la esquina del tablero donde va el punto `aa` del archivo (`top + left`, `top + right`, `bottom + left` o `bottom + right`). Todo lo que se lee del archivo la sigue, también `partial: auto`; las coordenadas que se escriben en las opciones (`variation: "Q4, R6"`) son las del tablero.

`transpose` `true` o `false` — inicial `false`

En `sgf-read`: la primera letra de cada punto del archivo es la fila, y el tamaño se gira. Con `origin`, las ocho simetrías del tablero.

```
#let archivo = path("josekis.sgf")
#gobanist-set(partial: auto)
#goban-layout(columns: 4, fit: true,
  sgf-read(archivo, origin: top + right),
  sgf-read(archivo, origin: bottom + right,
    transpose: true),
  sgf-read(archivo, transpose: true),
  sgf-read(archivo, origin: bottom + left))
```



`syntax` `auto`, `"sgf"` o `"sensei"` — inicial `auto`

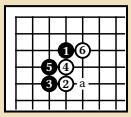
En `sgf-read`: la sintaxis del archivo. `"sensei"` son los diagramas de Sensei's Library (pág. 87); con `auto`, los que se leen así son los archivos sin nodos SGF (`(;`) que tienen líneas que empiezan con `$$`. Todas las sintaxis se leen como una partida SGF: todas las funciones andan igual con cualquiera.

2 Diagramas de Sensei's Library

Sensei's Library y los foros como L19 escriben los diagramas como texto, con líneas que empiezan con `$$`. Un archivo así se lee como una colección: cada bloque de líneas `$$` seguidas es una partida (`game: 2` es el segundo diagrama), y lo que hay entre los bloques no se lee. Las etiquetas `[go]` y `[/go]` de L19 se pueden dejar.

```
#let nadare = sgf-read("$$Bc Nadare
$$  ---------------
$$ | . . . . . . .
$$ | . . . . . . .
$$ | . . . 1 6 . .
$$ | . . 5 4 . . .
$$ | . . 3 2 a . .
$$ | . . . . . . .")

#sgf-diagrams(nadare, goban: (scale: 0.5))
```



Nadare (1–6)

La primera línea es la cabecera: `$$`, `B` o `W` (quién juega la jugada 1; si no está, negras), un tamaño de tablero (`$$B13`; si no está, 19), `m` y el número de la jugada 1 (`$$Wm67`) y el título. La `c` de las coordenadas se acepta pero no se usa: las coordenadas son las de `label`. Cada diagrama se lee así:

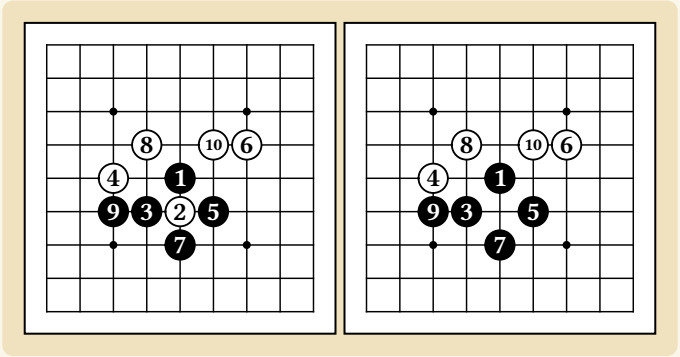
- El título es el nombre de la partida (`GN`) y el de su figura (`FG`): `sgf-diagrams` lo pone en el epígrafe.
- `X` y `O` son piedras iniciales (`AB` y `AW`). `B`, `#`, `Y` y `Z` son piedras negras con círculo, cuadrado, triángulo y cruz; `W`, `@`, `Q` y `P`, blancas. `C`, `S`, `T` y `M` son esas marcas en puntos vacíos, y las minúsculas son letras.
- Los números (`1` a `9`, y `0` es 10) son jugadas, en orden: las impares son del que juega la 1. Si falta un número («5 en 1»), las jugadas siguientes conservan el suyo (`MN`).
- Las marcas, las letras y las líneas `{AR C14 C11}` (flecha) y `{LN 5:6 5:9}` (línea) van en la última jugada. Sus puntos son del tablero (`C14`) o del diagrama (`5:6`, columna y fila desde arriba a la izquierda).
- Los bordes (`---`, `|`, `+`) dicen dónde está el diagrama en el tablero. Sin tamaño, dos bordes opuestos dan ese lado del tablero. Sin bordes, el diagrama va en el medio. Lo dibujado es la vista de la partida (`VW`): `sgf-diagrams` solo muestra esa parte.
- `.` y `,` son puntos vacíos (las estrellas las pone `hoshi`). `_` no es del tablero: es su `blank` (pág. 51). Los enlaces (`[a|página]`) no se leen.

Una línea que no se entiende, un diagrama más grande que el tamaño de la cabecera y una flecha o una línea que sale del tablero son errores.

3 Una partida en un tablero

Una partida en un tablero dibuja las piedras iniciales (`AB` y `AW`), como con `stones`, y las jugadas de la línea principal, como con `moves`. Los pases cuentan para la numeración, pero no se dibujan (con `notes`, se anotan: «N: pasa», pág. 73). Las piedras que agregan (`AB`, `AW`) o quitan (`AE`) los nodos posteriores a la primera jugada se aplican junto con la jugada anterior; quitar piedras necesita `captures`, y sin `captures` es un error. Al terminar, la numeración y el turno quedan listos para seguir la partida con `moves`. La partida lee también `PL` (quién juega después) y `MN` (el número de una jugada).

```
// Sin captures, 2 no se retira.
#goban(partida)
#goban(captures: "remove", partida)
```

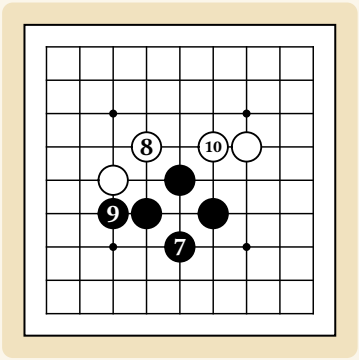


`sgf(partida, ..opciones)`

La misma partida con otras opciones, las de la rama `sgf`, que se suman a las que ya tenía: `sgf(partida, to: 7)`. Además, `forget` (pág. 80). Con `gobanist-set(sgf: ...)`, las opciones valen para todas las partidas.

`sgf.from` un número — inicial `1`

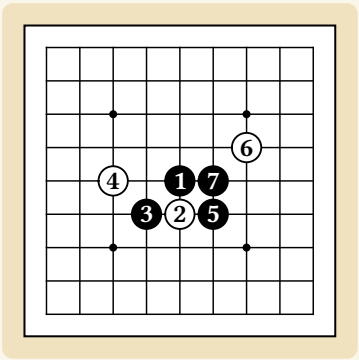
La primera jugada numerada. Las anteriores se dibujan sin número.



```
#goban(captures: "remove",
  sgf(partida, from: 7))
```

`sgf.to` `auto` o un número — inicial `auto`

La última jugada que se juega; las siguientes no se dibujan. Con `auto`, hasta la última.



```
#goban(captures: "remove",
  sgf(partida, to: 6),
  // Sigue con la 7 (negras), en otro lugar.
  moves("F5"))
```

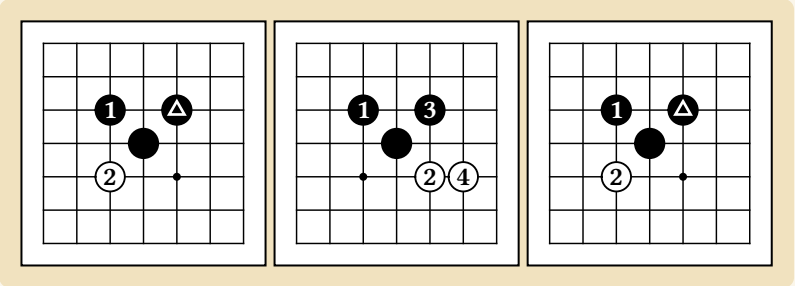
`from` y `to` cuentan las jugadas del archivo. El número que se muestra puede ser otro: `MN` lo cambia (las jugadas siguientes siguen desde ahí), y también `restart`.

`sgf.variation` `none`, números o jugadas — inicial `none`

La línea que se dibuja. Con `none`, la principal. Con números, en cada bifurcación se toma, en orden, la variante indicada (1 es la principal): `(2, 1)` toma la segunda en la primera bifurcación y la primera

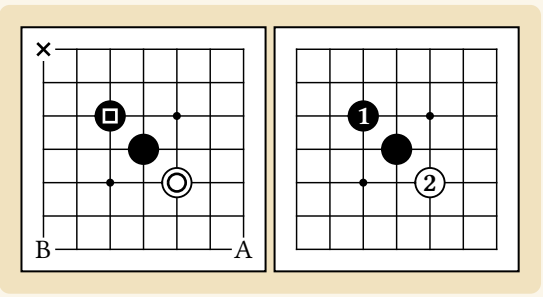
en la segunda. Con un texto, las jugadas de la línea desde la primera, separadas por comas o espacios: `"R16, P17, Q15"`; en cada bifurcación se toma la variante que juega la siguiente, y un pase se escribe `pass`. En los dos casos, cuando la lista se termina, se sigue por la primera variante. Una variante o una jugada que no existe es un error.

```
#goban(sgf(variantes, variation: (2,)))
#goban(sgf(variantes, variation: (1, 2)))
#goban(sgf(variantes, variation: "C5, C3"))
```



`sgf.marks` `true` o `false` — inicial `true`

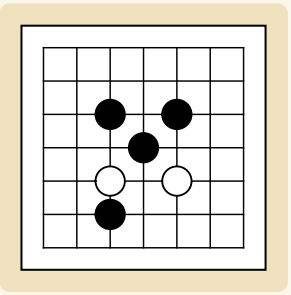
Dibuja las marcas del archivo (`TR`, `SQ`, `CR`, `MA`, `SL` y las etiquetas de `LB`), las flechas (`AR`) y las líneas (`LN`) del último nodo que se muestra, y atenúa los puntos de `DD` (que siguen atenuados hasta el próximo `DD`). Cada marca toma el jugador de la piedra que hay en ese punto, y reemplaza al número de la jugada; las etiquetas no tapan un número, así que sobre una piedra numerada no se dibujan. El territorio del archivo (`TB` y `TW`) se dibuja como el de `territory` (pág. 74) y cuenta para `goban-score`.




```
#goban(sgf(variantes, to: 2))
#goban(sgf(variantes, to: 2, marks: false))
```

`sgf.numbers` `auto`, `true` o `false` — inicial `auto`

Con `false`, las jugadas se dibujan sin número. Con `auto`, lo que pide el archivo con `PM`: con `PM[0]`, no se numeran.



```
#goban(sgf(variantes, numbers: false))
```

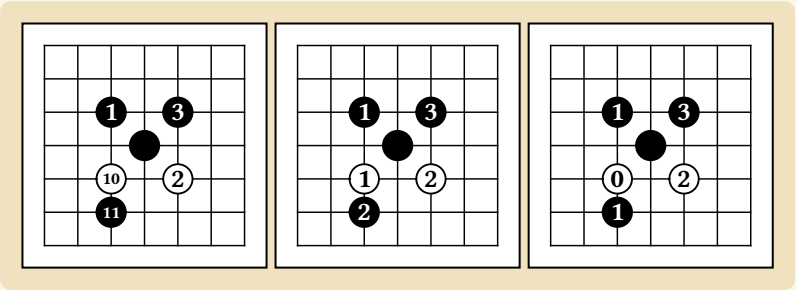
`sgf.step` un número o un arreglo — inicial `100`

El tamaño de los tramos en que `restart` reinicia la numeración. Con un arreglo, cada número es el tamaño de un tramo, y el último se repite: con `(50, 100)`, el primer tramo tiene 50 jugadas y los demás 100.

`sgf.restart` `auto`, `true`, `false` o un diccionario — inicial `auto`

Desde el segundo tramo de `step` jugadas, cada uno numera desde 1 (o desde `restart.from`), como en los libros: con `step: 100, restart: true`, la jugada 101 se muestra como 1. El primer tramo conserva los números de la partida. Con `false`, las jugadas llevan los números de la partida (con `MN`). Con `auto`, lo que pide el archivo con `PM`: con `PM[2]`, la numeración se reinicia cada 100 jugadas, sin importar `step`.

```
#goban(variantes)
#goban(sgf(variantes, step: 3,
  restart: true))
#goban(sgf(variantes, step: 3,
  restart: (on: true, from: 0)))
```

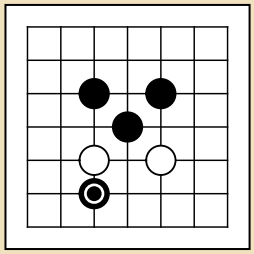


`sgf.restart.from` un número — inicial `1`

El número de la primera jugada de cada tramo, desde el segundo. Con `step: 99` y `restart: (on: true, from: 0)`, la jugada 100 se muestra como 0, la 101 como 1, y así.

`sgf.last-mark` `none`, una forma o un diccionario — inicial `none`

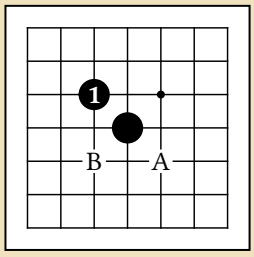
Marca la última jugada que se muestra, con una forma (`"circle"`) o con las opciones de una marca (`(shape: "triangle", line: (color: red))`). Un pase no se marca, y tampoco un punto que el archivo ya marca ni una jugada con número (el número ya dice cuál es la última): en la práctica, sirve con `numbers: false`.



```
#goban(sgf(variantes, numbers: false,
  last-mark: "circle"))
```

`sgf.letters` `none`, `"children"`, `"siblings"` o `auto` — inicial `none`

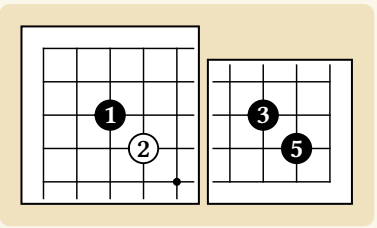
Marca con letras las variantes de la última jugada que se muestra: con `"children"`, las jugadas que pueden seguir; con `"siblings"`, las alternativas a esa jugada (la letra reemplaza a su número). Con `auto`, lo que pide el archivo con `ST`. Solo se marcan si hay más de una.




```
#goban(sgf(variantes, to: 1, marks: false,
  letters: "children"))
```

`sgf.view` `false`, `true` o un número — inicial `false`

Si el archivo tiene vista (`VW`), muestra solo esa parte del tablero, como `partial`. La vista puede cambiar durante la partida: un `VW` vale desde la jugada de su nodo (o desde la siguiente, si el nodo no tiene jugada) hasta el próximo `VW`, y `VW[]` vuelve a mostrar todo el tablero. Con `true`, la vista que abarca las de las jugadas `from` a `to`; con un número, la de esa jugada (con 0, la del principio). Un `partial` del tablero le gana.




```
// vistas: VW[aa:ee] al principio, VW[] en
// la 3 y VW[ff:ii] en la 5.
#goban(sgf(vistas, to: 2, view: true))
#goban(sgf(vistas, view: 5))
```

4 Datos y comentarios

```
sgf-info(partida, property, move: 0, variation: none)
```

Una propiedad de la raíz de la partida, como texto: `"PB"` y `"PW"` (los jugadores), `"RE"` (el resultado), `"KM"` (el komi), `"DT"` (la fecha), etc. Con `move`, la de esa jugada: la de su nodo y la de los nodos sin jugada que lo siguen (con 0, la de la raíz y los nodos anteriores a la primera jugada), en la línea de `variation` (como `sgf.variation`). Si no está, `""`. Si tiene varios valores, van separados por comas.

```
#sgf-info(variantes, "PB") (negras) contra
#sgf-info(variantes, "PW") (blancas),
komi #sgf-info(variantes, "KM").
Resultado: #sgf-info(variantes, "RE").
```

Ana (negras) contra Bruno (blancas), komi 6.5. Resultado: B+R.

El valor es el del archivo, con los caracteres escapados ya resueltos y en la codificación de `CA`: `"RE"` queda como `B+R`, no en palabras (`sgf-diagrams` lo escribe en palabras). Sirve cualquier propiedad. Las de los datos de la partida, según el formato SGF, son:

Propiedad	Qué es	Ejemplo
`PB`, `PW`	El nombre de Negras y el de Blancas.	`Lee Sedol`
`BR`, `WR`	El rango de Negras y el de Blancas.	`9p`
`BT`, `WT`	El equipo de Negras y el de Blancas.	
`RE`	El resultado: `B+` o `W+` y los puntos, `R` (abandono), `T` (tiempo) o `F` (falta); `0` o `Draw`, empate; `Void`, sin resultado; `?`, desconocido.	`W+3.5`
`KM`	El komi.	`6.5`
`HA`	El hándicap: cuántas piedras.	`2`
`DT`	La fecha, `AAAA-MM-DD` (o varias, separadas por comas).	`2016-03-09`
`EV`	El torneo o evento.	`Meijin`
`RO`	La ronda.	`3 (final)`
`PC`	El lugar.	`Tokio`
`GN`	El nombre de la partida.	
`GC`	Un comentario sobre la partida.	
`ON`	La apertura (fuseki).	
`RU`	Las reglas.	`Japanese`
`TM`	El tiempo de cada jugador, en segundos.	`7200`
`OT`	El tiempo extra (byo-yomi).	`5x30 byo-yomi`
`SO`	La fuente: un libro, una revista.	
`AN`	Quién comentó la partida.	
`US`	Quién escribió el archivo.	
`CP`	El copyright.	
`AP`	El programa que escribió el archivo.	`CGoban:3`

Propiedad	Qué es	Ejemplo
`SZ`	El tamaño del tablero (`19` , o `19:13`).	`19`
`CA`	La codificación del archivo.	`UTF-8`

```
sgf-comment(partida, move, variation: none, annotations: true, chat: true)
```

El comentario de una jugada (0: el de antes de la primera jugada), en la línea de `variation`: el nombre del nodo (`N`), en negrita; las anotaciones de la jugada (`TE` , `BM` , `DO` , `IT`) y de la posición (`GB` , `GW` , `DM` , `UC`), con los textos del idioma (pág. 31); y el comentario (`C`). Los valores (`V`) y los puntos calientes (`HO`) no se escriben. `sgf-diagrams` y `sgf-problem` escriben los comentarios de la misma forma.

```
Jugada 7: #sgf-comment(partida, 7)

#sgf-comment(problema, 1, variation: (2,))
```

Jugada 7: Tesuji. Negras capturan.

Mala jugada. Incorrecto: Blancas viven.

`annotations` `true` o `false` inicial `true`

En `sgf-comment`: con `false`, solo el comentario, sin el nombre ni las anotaciones.

```
Con: #sgf-comment(partida, 7) \
Sin: #sgf-comment(partida, 7, annotations: false)
```

Con: Tesuji. Negras capturan.
Sin: Negras capturan.

`chat` `true` o `false` inicial `true`

En `sgf-comment`: con `false`, quita del comentario el chat de los servidores: en OGS, todo lo que sigue a la línea «`-- chat --`»; en KGS, las líneas «`nombre [rango]: ...`» y, si hay alguna, también las «`nombre: ...`» del mismo comentario. Si no queda nada, la jugada no tiene comentario.

5 Diagramas como en los libros

```
sgf-diagrams(partida, ..opciones)
```

Dibuja la partida como en los libros: un diagrama cada `step` jugadas (1–50, 51–100, ...). Cada diagrama parte de la posición real, con las jugadas anteriores sin número y sus capturas; las piedras que capturan las jugadas del diagrama no se quitan (se quitan en el siguiente). Debajo de cada uno van su epígrafe y sus notas «N en M» (pág. 73), y debajo del último, el resultado. Una jugada en un punto ocupado quita la piedra que había, sin prisioneros, para dibujar partidas que no son legales. Va fuera de `goban`.

Las opciones son las de la rama `diagrams`; con `gobanist-set(diagrams: ...)`, valen para todo el documento. Las de `sgf(partida, ...)` valen en cada tablero, salvo las que fija cada diagrama (`from` , `to` , `variation` , `numbers` , `marks` , `last-mark` , `step` , `restart` y `view`), que son un error.

```
#sgf-diagrams(partida, step: 5)
```

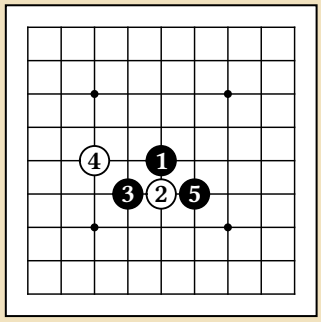


Diagrama 1 (1–5)

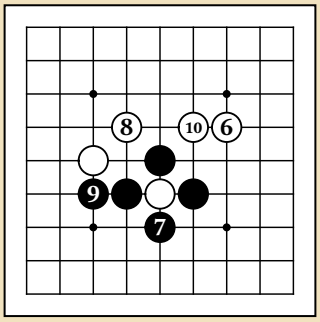


Diagrama 2 (6–10)

`diagrams.step` un número o un arreglo — inicial `50`

La cantidad de jugadas de cada diagrama. Con un arreglo, el primer diagrama tiene tantas jugadas como dice el primer número, el segundo como dice el segundo, y así; el último se repite: con `(5, 10, 3)`, los diagramas tienen 5, 10, 3, 3, 3… jugadas.

`diagrams.from` un número — inicial `1`

La primera jugada. Los diagramas son los mismos que sin `from`, con el mismo número: solo se dibujan los que tienen jugadas desde `from`, y el primero empieza en `from`. Con `step: 50` y `from: 120`, el primero es «Diagrama 3 (120–150)».

`diagrams.to` `auto` o un número — inicial `auto`

La última jugada. Con `auto`, la última de la partida.

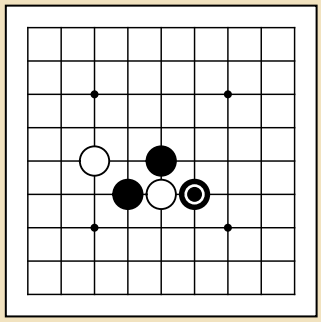
`diagrams.numbers` `true` o `false` — inicial `true`

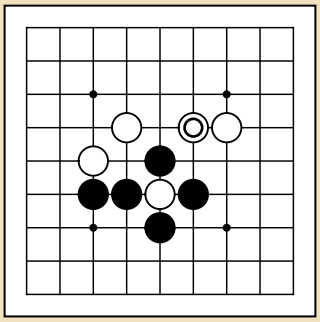
Con `false`, las jugadas de cada diagrama se dibujan sin número (las notas «N en M» siguen usando los números de la partida).

`diagrams.last-mark` `none`, una forma o un diccionario — inicial `none`

Como `sgf.last-mark`, en cada diagrama.

```
#sgf-diagrams(partida, step: 5, to: 10,
  numbers: false, last-mark: "circle")
```


Diagrama 1 (1–5)


Diagrama 2 (6–10)

`diagrams.restart` `auto`, `true`, `false` o un diccionario — inicial `auto`

Como `sgf.restart`, con un diagrama por tramo: desde el segundo diagrama, cada uno numera desde 1 (o desde `restart.from`), como algunos libros. El primer diagrama conserva los números de la partida. Con `auto`, lo que pide `PM[2]`: cada 100 jugadas, sin importar los diagramas.

```
#sgf-diagrams(partida, step: (2, 3), to: 8,
  restart: (on: true, from: 0))
```

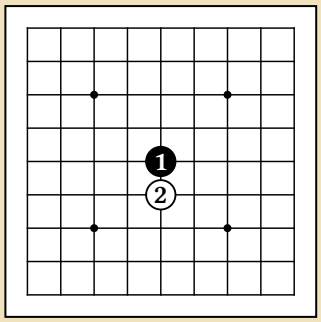


Diagrama 1 (1–2)

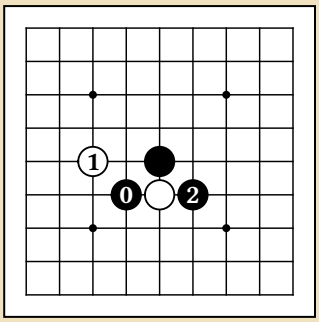


Diagrama 2 (3–5)

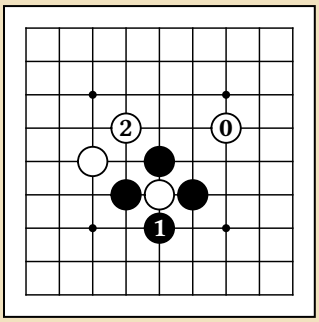


Diagrama 3 (6–8)

`diagrams.restart.from` un número — inicial `1`

El número de la primera jugada de cada diagrama, desde el segundo.

`diagrams.marks` `true` o `false` — inicial `true`

Como `sgf.marks`: cada diagrama muestra las marcas, flechas y líneas de su última jugada.

`diagrams.notes` `true` o `false` — inicial `true`

Escribe debajo de cada diagrama sus notas: «N en M» y los pases, «N: pasa».

`diagrams.title` `auto` o un texto — inicial `auto`

La palabra del epígrafe: «Diagrama 2 (51–100)». Con `auto`, la del idioma (`diagram` en `texts`).

`diagrams.caption` un diccionario

El epígrafe, las notas y el resultado.

`diagrams.caption.text` un diccionario — inicial `(size: 0.9em)`

Las opciones de `text` del epígrafe, las notas y el resultado.

```
#sgf-diagrams(partida, from: 6, title: "Figura",
  caption: (text: (style: "italic")),
  goban: (label: (at: (top, left))))
```

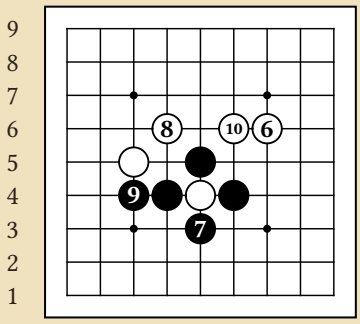


Figura 1 (6–10)

`diagrams.caption.min-width` una longitud — inicial `8em`

Fuera de la grilla, un epígrafe más ancho que su tablero se corta en las líneas que le hacen falta al ancho del tablero o, si el tablero es más angosto, a esta medida; después se angosta hasta donde esas líneas siguen entrando, para que queden parejas. Nunca es más angosto que su palabra más larga. En la grilla, el epígrafe se corta al ancho de la celda.

`diagrams.goban` opciones de tablero — inicial `(:)`

Las opciones de cada tablero, encima de las globales. Los tableros empiezan con `captures: "remove"` y `notes: true`, y tienen el tamaño de la partida.

`diagrams.layout` opciones de disposición — inicial `(:)`

Cómo se disponen los diagramas: las opciones de `layout` (pág. 103), solo para estos diagramas: `layout: (columns: 3)`.

`diagrams.variation` `none`, números o jugadas — inicial `none`

La línea que se dibuja, como `sgf.variation`.

`diagrams.variations` `true`, `false` o un diccionario

Dibuja también las variantes de la partida, cada una en su diagrama, después del diagrama donde nace: «Diagrama 1a», «Diagrama 1b»... Cada variante va entera, con sus jugadas numeradas desde 1; el epígrafe dice qué jugada reemplaza (`variation-title`), y las jugadas anteriores se ven sin número. Sus comentarios y notas se escriben como los de los demás diagramas. Las variantes son las que se apartan de la línea de `variation` (con `variation`, también la principal).

```
#sgf-diagrams(variantes, step: 2, to: 3,
  marks: false,
  variations: (on: true, select: "all"),
  goban: (scale: 0.7))
```

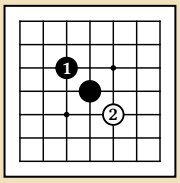


Diagrama 1 (1–2)

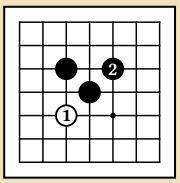


Diagrama 1a
(variante en 2)

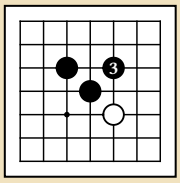


Diagrama 2 (3–3)

`diagrams.variations.select` `"commented"` o `"all"` — inicial `"commented"`

Cuáles se dibujan: las que tienen algún comentario (`C`) en sus jugadas propias, o todas.

`diagrams.variations.depth` un número — inicial `1`

Hasta qué profundidad: con 1, las que se apartan de la línea que se dibuja; con 2, también las variantes de esas, y así. Las de una variante van después de ella, y su epígrafe dice qué jugada de ella reemplazan.

`diagrams.variations.from` un número — inicial `1`

Solo las variantes que nacen en esa jugada o después.

`diagrams.variations.paths` un arreglo de caminos — inicial `()`

Variantes que se dibujan siempre, aunque `variations` esté apagada, a cualquier profundidad: cada camino es como `sgf.variation`, con números (`(2,)`, `(2, 3, 1)`) o con jugadas (`"R16 P17 Q15"`). Los 1 del final no cambian la línea: `(2, 3, 1)` es `(2, 3)`. Un camino que la partida no tiene, o que no se aparta de la línea, es un error.

```
#sgf-diagrams(josekis, variations: (paths: ((2,), "Q16 R14 O16")))
```

`diagrams.variation-title` `auto` o una función — inicial `auto`

Lo que va entre paréntesis en el epígrafe de una variante: «Diagrama 3a (variante en 26)». Una función recibe el número de la jugada que reemplaza: `variation-title: n => [en la #n]`. Con `auto`, el texto del idioma (`variation-at` en `texts`).

`diagrams.figures` `true` o `false` — inicial `true`

Si el archivo marca figuras (`FG`), cada figura es un diagrama, con su nombre en el epígrafe, en lugar de uno cada `step` jugadas.

`diagrams.view` `true` o `false` — inicial `true`

Si el archivo tiene vista (`VW`), cada diagrama muestra solo la que abarca las vistas de sus jugadas, como `sgf.view`; una variante, la de su línea. Un `partial` en `goban` le gana.

`diagrams.partial` `none`, `auto` o `"A1:E5"` — inicial `none`

Con dos esquinas, como en `partial` (`"K10:T19"`), todos los diagramas muestran esa parte, aunque el archivo tenga vista. Con `auto`, donde no hay vista del archivo, todos los diagramas muestran la misma parte del tablero: la que usan las líneas que se dibujan (la de `variation` y las de `variations`), como `partial: auto`.

```
#sgf-diagrams(josekis, partial: auto, variations: true)
```

`diagrams.comments` `true`, `false` o un diccionario

Escribe los comentarios de las jugadas de cada diagrama, como `sgf-comment`, cada uno con el número de la jugada.

```
#sgf-diagrams(partida, step: 5,
  comments: (on: true, root: true),
  layout: (columns: 2))
```

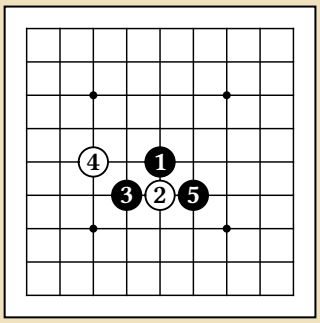


Diagrama 1 (1–5)

Una partida corta.

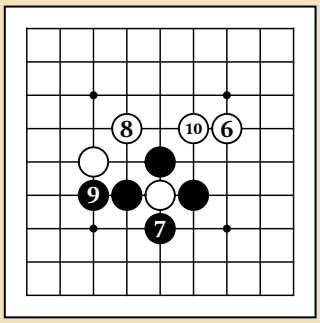


Diagrama 2 (6–10)

7: Tesuji. Negras capturan.

`diagrams.comments.text` un diccionario — inicial `(:)`

Las opciones de `text` de los comentarios, encima de las de `caption.text`.

`diagrams.comments.align` `left`, `center` o `right` — inicial `left`

La alineación de los comentarios en el ancho del diagrama.

`diagrams.comments.justify` `true` o `false` — inicial `false`

Justifica los comentarios. En un diagrama angosto, quedan espacios grandes entre las palabras.

`diagrams.comments.root` `true` o `false` — inicial `false`

Escribe también el comentario de la raíz (el de los nodos anteriores a la primera jugada), sin número, antes de los del diagrama que empieza en la jugada 1.

`diagrams.comments.number` una función — inicial `n => [#strong(str(n)): ]`

Lo que va delante de cada comentario: una función del número de la jugada, el que se ve en el tablero (con `restart`, el reiniciado). Por ejemplo, `n => [Jugada #n. ]`.

`diagrams.comments.separator` un contenido — inicial `parbreak()`

Lo que va entre los comentarios. Con el valor inicial, cada uno va en su párrafo; con `h(1em)`, van seguidos.

`diagrams.comments.position` `bottom`, `left`, `right` o `"after"` — inicial `bottom`

Dónde van los comentarios: debajo del epígrafe; al costado del diagrama, como en los libros de partidas comentadas; o, con `"after"`, todos juntos después de los diagramas, a lo ancho, cada grupo con el epígrafe de su diagrama como encabezado. Al costado empiezan a la altura del borde de arriba del tablero; el epígrafe, las notas y el resultado siguen debajo del tablero.

```
#sgf-diagrams(partida, step: 5,
  comments: (on: true, position: right))
```

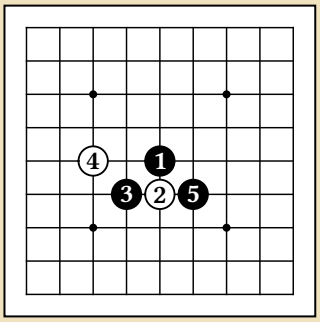


Diagrama 1 (1–5)

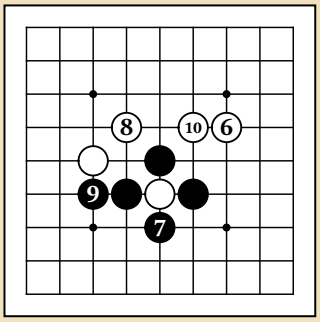


Diagrama 2 (6–10)

7: Tesuji. Negras capturan.

`diagrams.comments.width` `auto` o una longitud — inicial `auto`

El ancho de los comentarios al costado. Con `auto`, lo que queda: fuera de la grilla, el resto de la línea; en la grilla, el resto de la celda, y con `fit`, el tablero ocupa la mitad de la celda y los comentarios la otra mitad.

`diagrams.comments.sep` una longitud — inicial `1em`

El espacio entre el diagrama y los comentarios al costado.

`diagrams.comments.height` `none` o una longitud — inicial `none`

La altura máxima de los comentarios de un diagrama, debajo o al costado. Lo que no entra (se corta entre palabras) va después de los diagramas, como con `"after"`, con el epígrafe del diagrama como encabezado.

```
#sgf-diagrams(lee, step: 30, layout: (columns: 2),
  comments: (on: true, height: 6em))
```

`diagrams.annotations` `true` o `false` — inicial `true`

Con los comentarios, escribe también el nombre y las anotaciones de cada jugada.

`diagrams.chat` `true` o `false` — inicial `true`

Con los comentarios, como `chat` de `sgf-comment`: con `false`, sin el chat de KGS y OGS. Una jugada a la que solo le quedaba chat no aparece.

`diagrams.result` `true`, `false` o un diccionario

Escribe debajo del último diagrama, si llega al final de la partida, el resultado (`RE`) y, si el archivo marca el territorio (`TB`, `TW`), el puntaje de cada jugador: «Resultado: Gana Negro por 0.5 puntos (Negro 6, Blanco 5.5)». El puntaje se cuenta como con `goban-score`, con el komi del archivo (`KM`).

`diagrams.result.title` `auto` o un texto — inicial `auto`

La palabra que precede al resultado. Con `auto`, la del idioma (`result` en `texts`).

`diagrams.result.format` `"text"` o `"raw"` — inicial `"text"`

Cómo se escribe el resultado: con `"text"`, en palabras, en el idioma del texto («Gana Negro por abandono»); con `"raw"`, como está en el archivo («B+R»). Con `"text"` se entienden `0` o `Draw` (empate), `Void` (sin resultado), `?` (desconocido) y `B+` o `W+` seguido de nada, de la diferencia, de `R` o `Resign`, de `T` o `Time`, o de `F` o `Forfeit`. Otro resultado se escribe como está.

`diagrams.result.black` `auto` o un texto — inicial `auto`

El nombre de las negras en el puntaje y en el resultado. Con `auto`, el del idioma.

`diagrams.result.white` `auto` o un texto — inicial `auto`

El nombre de las blancas en el puntaje y en el resultado.

```
#sgf-diagrams(variantes,
  result: (title: "Final", format: "raw"))
#sgf-diagrams(variantes,
  result: (black: "Ana", white: "Bruno"))
```

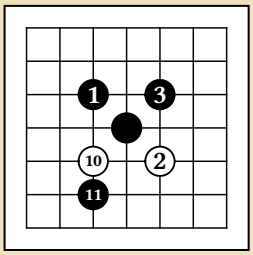


Diagrama 1 (1–11)
Final: B+R

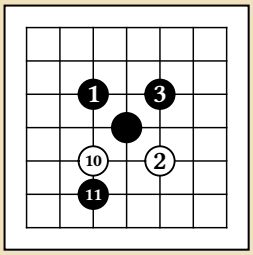


Diagrama 1 (1–11)
Resultado: Gana Ana
por abandono

6 Problemas

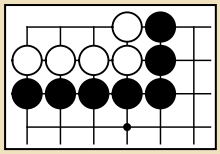
`sgf-problem(..problemas, ..opciones)`

Dibuja un problema (tsumego). Primero, el enunciado: la posición inicial con sus marcas, quién juega (`PL`, o el color de la primera jugada) y el comentario de la raíz. Después, un diagrama por cada línea del árbol del archivo (cada variante de la solución, correcta o no), numerada desde 1, con sus notas «N en M» y los comentarios y anotaciones de sus jugadas, que suelen decir si la variante es correcta. Todos los diagramas muestran la misma parte del tablero: la vista del archivo (`VW`) o, sin vista, la que usa el problema. Va fuera de `goban`.

Con varias partidas (`sgf-problem(p1, p2, p3)`), van primero todos los enunciados, cada uno con su número (el de la partida en su archivo), y después las soluciones de los que las tienen. Así se arma una hoja de problemas.

Las opciones son las de la rama `problems`; con `gobanist-set(problems: ...)`, valen para todo el documento. Las de `sgf(problema, ...)` valen en cada tablero, salvo las que fija el diagrama (`from`, `to`, `variation`, `numbers`, `last-mark`, `step`, `restart` y `view`). En los ejemplos, `problema` es `problema.sgf`, un problema de vida y muerte: la jugada correcta y dos equivocadas, con sus comentarios.

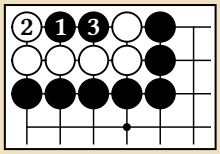
```
#sgf-problem(problema, title: "Problema 1")
```



Problema 1:

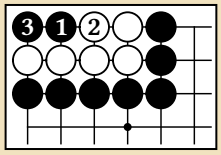
Juega Negro

Negras matan.



Variante 1

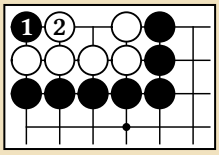
1: Tesuji. Correcto: el punto vital.



Variante 2

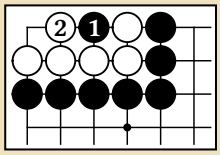
1: Tesuji. Correcto: el punto vital.

3: Blancas tienen un solo ojo.



Variante 3

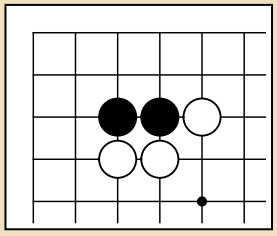
1: Mala jugada. Incorrecto: Blancas viven.



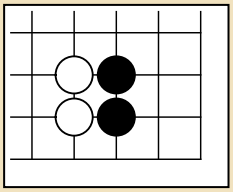
Variante 4

1: Incorrecto.

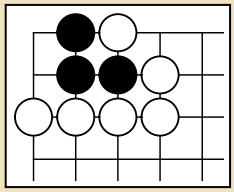
```
// problemas: las tres partidas de problemas.sgf.
#sgf-problem(..problemas, comments: false,
  layout: (columns: 4, fit: true))
```



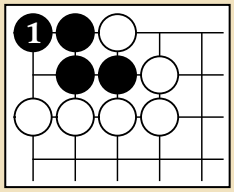
Problema 1



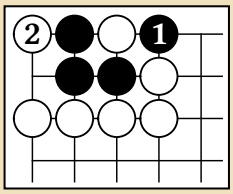
Problema 2: Juega Blanco



Problema 3: Juega Negro



Problema 3: Variante 1



Problema 3: Variante 2

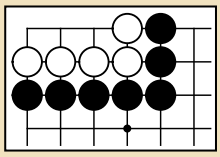
`problems.statement` `true`, `false` o `"alone"` — inicial `true`

Dibuja el enunciado. Con `"alone"`, el enunciado va solo en su fila, centrado, y las variantes debajo.

`problems.solutions` `true` o `false` — inicial `true`

Dibuja las variantes. Con `false`, queda solo el enunciado; las soluciones van en otro lugar, con `statement: false`.

```
#sgf-problem(problema, solutions: false,
  title: "Problema 2")
```



Problema 2:

Juega Negro

Negras matan.

`problems.title` `auto` o un texto — inicial `auto`

El comienzo del epígrafe del enunciado: «Problema 1: Juega Negro». Con varias partidas, va seguido del número de cada una; con `auto`, nada, o con varias partidas, la palabra del idioma (`problem` en `texts`): «Problema 7: Juega Negro». Las soluciones dicen también de qué problema son: «Problema 7: Variante 1».

`problems.to-play` `auto`, `"black"`, `"white"` o `none` — inicial `auto`

Quién juega, para el epígrafe del enunciado. Con `auto`, lo que dice `PL` o, si no está, el color de la primera jugada; si no hay ninguna de las dos, no se dice. Muchas colecciones no lo dicen en el archivo: `to-play: "black"`. Con `none`, el epígrafe no lo dice.

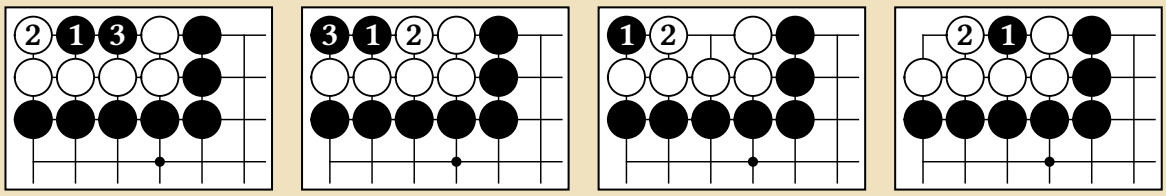
`problems.variation-title` `auto` o una función — inicial `auto`

El epígrafe de cada solución: una función de su número, `n => [Solución #n]`. Con `auto`, «Variante 2», con la palabra del idioma (`variation` en `texts`).

`problems.verdict` `true` o `false` — inicial `false`

El epígrafe de cada solución dice si es correcta o incorrecta («Variante 2 (incorrecta)»), con los textos `correct` y `wrong` del idioma. Se decide con las anotaciones de la línea: el último `GB` o `GW` (la posición es buena para negras o para blancas), según quién juega; si no hay, la primera jugada con `TE` o `BM` (las jugadas impares son de quien juega: un `BM` suyo es un error, uno del rival no); si no, la palabra `RIGHT` en un comentario, como en los archivos de goproblems.com. Si no hay nada de eso, no dice nada.

```
#sgf-problem(problema, verdict: true,
  statement: false, comments: false,
  layout: (columns: 4, fit: true))
```



Variante 1 (correcta) Variante 2 (correcta) Variante 3 (incorrecta) Variante 4

`problems.links` `true` o `false` — inicial `false`

El epígrafe del enunciado es un enlace a su primera solución, y el de cada solución, un enlace al enunciado, también si están en otro `sgf-problem` (con `solutions: false` y `statement: false`). Un enlace cuyo destino no está, o está más de una vez (un problema dibujado dos veces), no se pone.

```
#sgf-problem(problema, links: true, solutions: false)
// …
#sgf-problem(problema, links: true, statement: false)
```

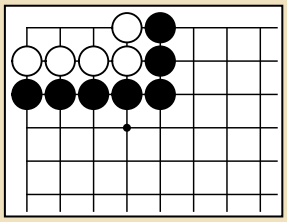
`problems.view` `true` o `false` — inicial `true`

Si el archivo tiene vista (`VW`) al principio, todos los diagramas muestran esa parte.

`problems.partial` `auto`, `none` o `"A1:E5"` — inicial `auto`

Con dos esquinas, como en `partial`, todos los diagramas muestran esa parte, aunque el archivo tenga vista. Con `auto`, si no se usa la vista del archivo, todos los diagramas muestran la parte del tablero donde hay piedras, jugadas, marcas, flechas o líneas en alguna línea del problema, como `partial: auto`. Las líneas que se agregan alrededor se cambian con `goban: (partial-margin: ...)`. Con `none`, se ve todo el tablero.

```
#sgf-problem(problema, solutions: false,
  goban: (partial-margin: 3))
```



Juega Negro
Negras matan.

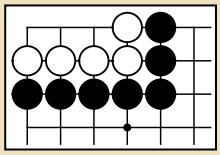
`problems.notes` `true` o `false` — inicial `true`

Escribe las notas «N en M» de cada variante.

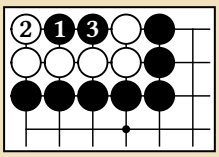
`problems.caption` un diccionario (inicial `(text: (size: 0.9em), min-width: 8em)`)

Los epígrafes, las notas y los comentarios, como `diagrams.caption`.

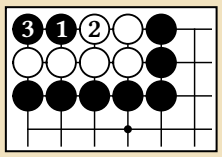
```
#sgf-problem(problema, statement: "alone",
  comments: false, notes: false,
  caption: (text: (style: "italic")))
```



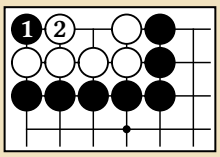
Juega Negro



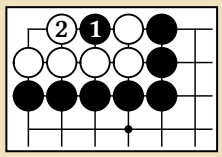
Variante 1



Variante 2



Variante 3



Variante 4

`problems.comments` `true`, `false` o un diccionario

El comentario de la raíz debajo del enunciado y los de cada variante debajo de su diagrama, como `diagrams.comments` (sin `root`). Aquí están prendidos al empezar. `text`, `align` y `justify` valen también para el comentario del enunciado.

`problems.annotations` `true` o `false` (inicial `true`)

Con los comentarios, escribe también el nombre y las anotaciones de cada jugada (`TE`, `BM`…).

`problems.chat` `true` o `false` (inicial `true`)

Con los comentarios, como `chat` de `sgf-comment`.

`problems.goban` opciones de tablero (inicial `(:)`)

Las opciones de cada tablero, como `diagrams.goban`.

`problems.layout` opciones de disposición (inicial `(:)`)

Cómo se disponen el enunciado y las variantes: las opciones de `layout` (pág. 103), solo para estos diagramas.

11 Disposición de los diagramas

Los diagramas de `sgf-diagrams` y `sgf-problem` se disponen según la rama `layout`, y `goban-layout` dispone juntos tableros sueltos y esos diagramas. Sin grilla (`columns: auto`), los diagramas van en el párrafo, uno al lado del otro, y pasan a la línea siguiente cuando no entran. Con un número de columnas, van en una grilla de columnas del mismo ancho: cada fila es una línea aparte (la página puede cortarse entre filas), los tableros de una fila quedan alineados arriba y una fila incompleta deja vacías las últimas columnas. Con `fit`, además, los tableros se escalan para llenar el ancho de su columna.

```
goban-layout(..tableros, ..opciones)
```

Dispone, en el orden en que vienen, tableros hechos con `board(...)`, partidas (`sgf-read`, `sgf`) y los diagramas de `sgf-diagrams(...)` y `sgf-problem(...)`. Las opciones son las de la rama `layout`; las `layout` de los `sgf-diagrams` y `sgf-problem` de adentro no se usan. Con `fit`, cada tablero suelto llena su columna, y los diagramas de un `sgf-diagrams` o un `sgf-problem` usan la escala del primero. Un `goban(...)` ya está dibujado, así que adentro es un error: va `board(...)`.

```
#goban-layout(columns: 4, fit: true,
  board(size: 5, stones(black: "C3")),
  board(size: 5, stones(white: "C3")),
  sgf-diagrams(partida, step: 5))
```

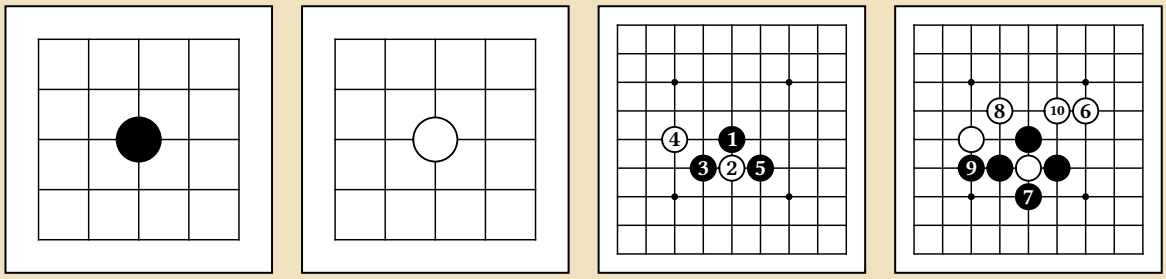


Diagrama 1 (1–5) Diagrama 2 (6–10)

```
#sgf-diagrams(partida, step: 2, to: 6,
  layout: (columns: 3, fit: true))
```

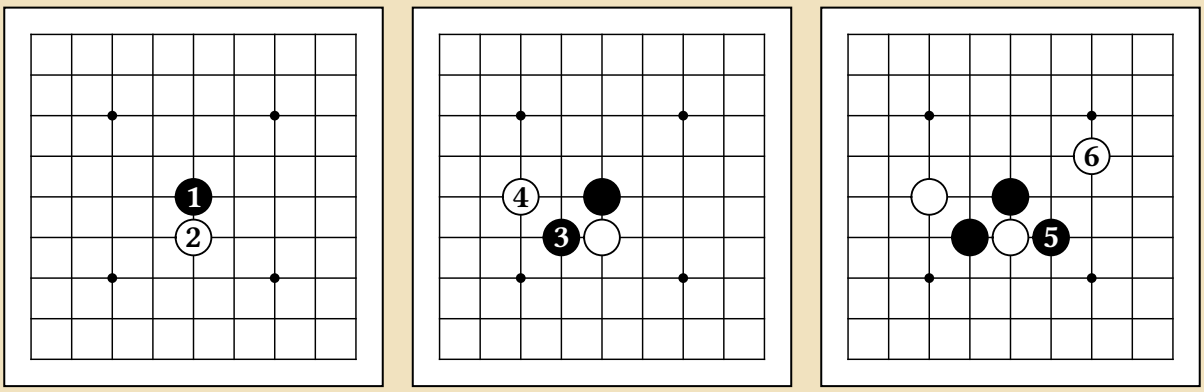


Diagrama 1 (1–2) Diagrama 2 (3–4) Diagrama 3 (5–6)

`layout` un diccionario

Las opciones de la disposición: con `gobanist-set(layout: ...)`, para todo el documento; en `goban-layout`, para esa disposición; con `diagrams.layout` y `problems.layout`, para los de `sgf-diagrams` y `sgf-problem`, encima de las de `layout`.

`layout.columns` `auto` o un número — inicial `auto`

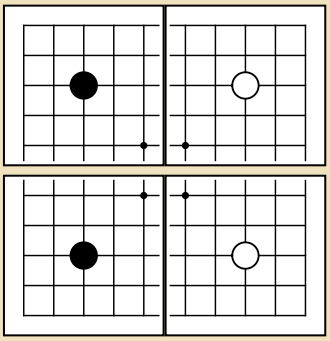
La cantidad de columnas de la grilla. Con `auto`, los diagramas van en el párrafo, sin grilla.

`layout.width` una longitud, una fracción del ancho o `auto` — inicial `100%`

El ancho de la grilla. Si es menor que el de la línea, la grilla se centra. Cada columna mide lo que queda al restarle los `column-sep` a `width`, repartido entre las columnas; un tablero más angosto queda en su columna según `align`, así que entre dos tableros hay más espacio que `column-sep`.

Con `auto`, la grilla se ajusta a los diagramas: las columnas tienen el ancho del diagrama más ancho (con su epígrafe, que se corta como sin grilla, pág. 92) y entre dos tableros del mismo ancho queda exactamente `column-sep`. La grilla mide eso por la cantidad de columnas (o de diagramas, si son menos) y se centra en la línea. `fit` no sirve con `width: auto`: es un error.

```
#goban-layout(columns: 2, width: auto,
  column-sep: 0em, row-sep: 0em,
  board(partial: "A9:E5", scale: 0.9,
    stones(black: "C7")),
  board(partial: "E9:J5", scale: 0.9,
    stones(white: "G7")),
  board(partial: "A1:E5", scale: 0.9,
    stones(black: "C3")),
  board(partial: "E1:J5", scale: 0.9,
    stones(white: "G3")))
```



`layout.column-sep` una longitud — inicial `1em`

El espacio entre las columnas.

`layout.row-sep` una longitud — inicial `1em`

El espacio entre las filas.

`layout.fit` `true` o `false` — inicial `false`

Escala cada tablero para que llene el ancho de su columna; reemplaza el `scale` del tablero. Las líneas no cambian de grosor. En `sgf-diagrams` y `sgf-problem`, todos los tableros usan la escala que llena la columna con el primero, así las piedras tienen el mismo tamaño en todos; si con esa escala un tablero no entra en la columna (un problema más ancho que el primero, por ejemplo), solo ese tablero se achica hasta llenarla. Solo con `columns`, y no con `width: auto`. Sin un ancho disponible (una disposición medida con `measure` sin `width`), la grilla toma el ancho de los diagramas, sin `fit`.

`layout.halign` `left`, `center` o `right` inicial `center`

Dónde va cada diagrama dentro de su columna, con su epígrafe y sus notas. Los comentarios van siempre según `comments.align`.

```
#sgf-diagrams(partida, step: 2, to: 6,
  layout: (columns: 2, width: 60%,
    column-sep: 3em, row-sep: 2em,
    halign: left))
```

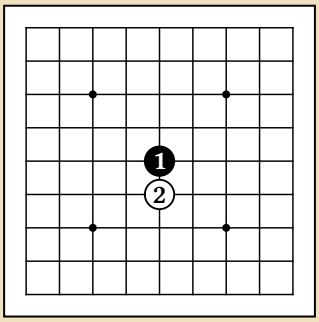


Diagrama 1 (1–2)

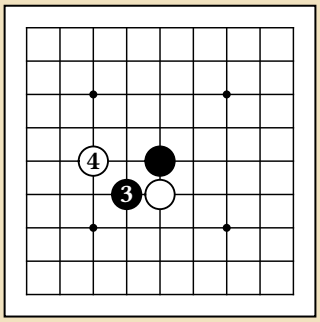


Diagrama 2 (3–4)

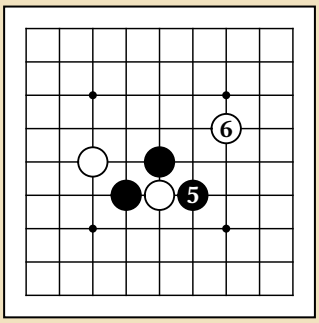
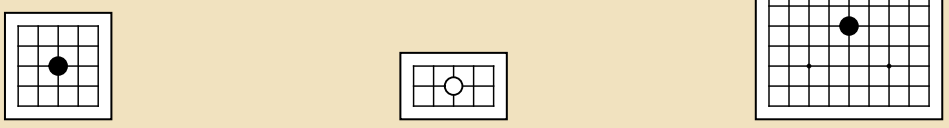


Diagrama 3 (5–6)

`layout.valign` `top` o `bottom` inicial `top`

Cómo se alinean los diagramas de una fila de la grilla cuando sus tableros tienen distinto alto. Con `top`, arriba. Con `bottom`, los tableros terminan a la misma altura, y sus epígrafes y notas empiezan todos en la misma línea, debajo. Con `columns: auto`, los diagramas van en la línea de base del párrafo, como con `bottom`.

```
#goban-layout(columns: 3, valign: bottom,
  board(size: 5, scale: 0.6,
    stones(black: "C3")),
  board(size: (5, 3), scale: 0.6,
    stones(white: "C2")),
  board(size: 9, scale: 0.6,
    stones(black: "E5")))
```



`layout.align-captions` `true` o `false`

inicial `false`

Con `valign: top`, los epígrafes, las notas y los comentarios de los diagramas de una fila empiezan en la misma línea, debajo del tablero más alto; los tableros siguen alineados arriba. Con `valign: bottom` ya están en la misma línea.

```
#sgf-problem(..problemas, solutions: false,
  layout: (columns: 3, align-captions: true))
```

`layout.per-page` `none` o un número

inicial `none`

Cada tantos diagramas, empieza una página nueva: con `per-page: 1`, un diagrama por página. En la grilla, una fila a medias se dibuja antes del corte; conviene que el número sea un múltiplo de `columns`. Con `none`, no hay cortes. En `sgf-problem` con `statement: "alone"`, el enunciado cuenta como un diagrama. Con varias columnas de página (`columns` de Typst, o `columns` en `page`), el corte pasa a la columna siguiente.

```
#sgf-diagrams(lee, step: 50, layout: (columns: 2, fit: true, per-page: 4))
```

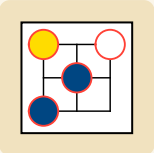
12 Jugadores y perfiles

Cada piedra, jugada y marca pertenece a uno de tres jugadores: el primero (`black`), el segundo (`white`) o el neutral (`neutral`). Su aspecto no está en cada elemento sino en el jugador: la rama de cada uno reúne las opciones de sus piedras, sus prisioneros, sus jugadas, sus marcas y su territorio. Aquí están los jugadores y, después, los perfiles.

1 Jugadores

`players` un diccionario

Las opciones de los jugadores: `neutral`, `black` y `white`, y `all`, que va a los tres.



```
#gobanist-set(players: (
  all: (stone: (line: (color: red))),
  neutral: (stone: (fill: (color: yellow))),
  black: (stone: (fill: (
    color: blue.darken(40%))))))

#goban(size: 3,
  stones(black: "A1, B2", white: "C3"),
  stones("A3"))
```

`players.all` opciones de jugador

Las opciones de los tres jugadores.

`players.neutral` opciones de jugador

Las opciones del jugador neutral: las piedras y las marcas sin jugador.

`players.black` opciones de jugador

Las opciones del primer jugador.

`players.white` opciones de jugador

Las opciones del segundo jugador.

Cada jugador tiene el mismo árbol de opciones. Cada rama se documenta con el objeto al que se aplica, sin el jugador en la ruta: para el jugador blanco, `stone.radius` es `players.white.stone.radius`.

`players.white.stone` opciones de piedra

El estilo de sus piedras (pág. 54). `stone` en el primer nivel es `players.all.stone`.

`players.white.prisoner` opciones de piedra

El estilo de sus prisioneros, encima del de sus piedras (pág. 62). `prisoner` en el primer nivel es `players.all.prisoner`.

`players.white.move` opciones de jugada

El estilo de sus jugadas (pág. 60). `move` en el primer nivel es `players.all.move`.

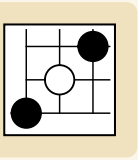
`players.white.mark` opciones de marca

El estilo de sus marcas (pág. 64). La forma no es parte del estilo: va en cada `marks`. `mark` en el primer nivel es `players.all.mark`.

`players.white.territory` opciones de territorio

El estilo de su territorio (pág. 74).

Copiar un jugador en otro es copiar su rama: el árbol es un valor.



```
#gobanist-set(players: (neutral: (stone: (
  fill: (color: white)))))

// Neutral, pero con el aspecto de las blancas.
#goban(partial: "A1:C3",
  stones("B2"), stones(black: "A1, C3"))
```

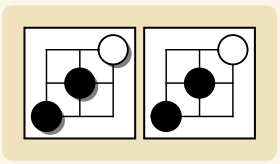
2 Perfiles

Un perfil es un estilo completo, para el tablero y los jugadores, guardado como un diccionario de opciones: las mismas de `gobanist-set`. Los perfiles del paquete se nombran con una cadena (pág. 109); uno propio es un diccionario.

`profile` el nombre de un perfil o un diccionario

Aplica un perfil: `profile: "wain"` usa uno del paquete, y `profile: (background: ..., players: ...)`, uno propio. Va en `gobanist-set`, en las opciones de un tablero y en `view.goban`.

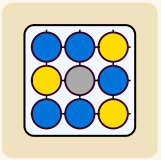
Antes de aplicarlo, el tablero y los jugadores vuelven al estilo inicial (el del perfil `default`): así un perfil no hereda lo que no define, y una sombra o una imagen puesta antes se pierde. Lo que no es estilo queda como estaba: `size`, `use-i`, `partial`, `partial-margin`, `scale`, `scale-line-width`, `label.at`, `label.columns`, `label.rows` y sus direcciones, `hoshi.at`, `move.numbers`, las flechas, las líneas, el juego y las ramas de las partidas, los diagramas y las vistas. Lo que va después del perfil se suma a él.



```
#gobanist-set(stone: (shadow: (on: true,
  angle: -45deg, offset: 2pt)))

#goban(size: 3,
  stones(black: "A1, B2", white: "C3"))
#goban(size: 3, profile: "default",
  stones(black: "A1, B2", white: "C3"))
```

Un `profile` dentro de un perfil es el perfil del que parte: `pgfgosmooth` es `(profile: pgfgo, ...)`.



```
#let ejemplo = (
  background: (fill: (color: blue.lighten(95%))),
  players: (
    all: (stone: (line: (
      color: fuchsia.darken(80%)))),
    neutral: (stone: (fill: (color: gray))),
    white: (stone: (fill: (color: yellow))),
    black: (stone: (fill: (color: blue))),
  ),
)
#let ejemplo2 = (profile: ejemplo,
  background: (rounded-corners: 4pt))

#gobanist-set(profile: ejemplo2)

#goban(partial: "A1:C3",
  moves("A1:3", numbers: false),
  stones(black: "B1, B3, C2", white: "C1, C3"),
  stones("B2"))
```

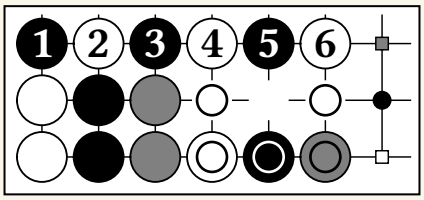
Un perfil propio puede guardarse en un módulo e importarse en cualquier documento (pág. 22).

13 Catálogo de perfiles

Los perfiles que trae gobanist, cada uno con una muestra y con su definición, tal como está en `src/profiles.typ`. Se usan con `profile: "nombre"` (pág. 108). En las definiciones, `xmix(a, p, b)` mezcla `p` de `a` con el resto de `b`, como `a!p!b` de xcolor; `pgf-gray` es `luma(50%)`; y `picture("board.jpg")` es la imagen `images/pgfgo-board.jpg` del paquete.

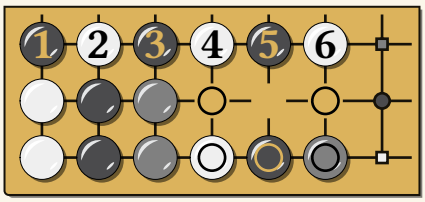
1 default

El estilo inicial del paquete: el que tienen el tablero y los jugadores antes de usar un perfil, y al que vuelven antes de aplicar cualquier otro. Su definición es el diccionario vacío: es el árbol de valores iniciales de `src/options.typ`.



2 pgfgo

Un tablero de madera clara, con sombra y esquinas apenas redondeadas, y piedras con sombra y brillo. Usa los colores de `wain` y la sombra y el brillo de `raised`, definidos aquí.




```
#let pgfgo = (
  background: (
    fill: (color: pgfgo-color, opacity: 1),
    line: (color: pgfgo-dark, opacity: 1),
    shadow: true,
    rounded-corners: (inner: 0pt, outer: 2pt),
  ),
  grid: (line: (color: pgfgo-dark, width: 1pt)),
  players: (
    all: (
      stone: raised + (line: (on: true, color: black, width: 0.3pt, opacity: 1), radius:
0.17cm),
      mark: (line: (color: black, width: 1pt), label: (text: (fill: black))),
    ),
    white: (stone: (fill: (color: wain-white), shine: (color: white)), move: (label:
(text: (fill: black)))),
    black: (stone: (fill: (color: wain-black)), move: (label: (text: (fill: pgfgo-
color))), mark: (line: (color: pgfgo-color))),
```

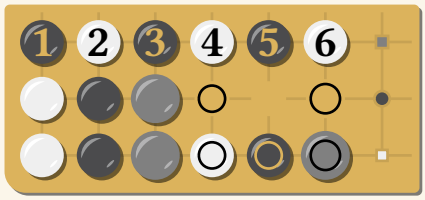
```
  ),
  arrow: (color: xmix(red, 80%, black)),
  line: (color: xmix(red, 80%, black)),
)
```

```
#let pgf-gray = luma(50%)
#let wain-goban = rgb("#402A2D")
#let wain-grid = rgb("#FAE0B2")
#let wain-black = rgb("#4B4B4D")
#let wain-shade = rgb("#E6E7E8")
#let wain-white = rgb("#EFEFEF")
#let pgfgo-color = rgb(220, 179, 92)
#let pgfgo-dark = xmix(pgfgo-color, 10%, black)
#let pgfgosmooth-grid = rgb("#C7A253")
```

```
#let raised = (
  shadow: (on: true, opacity: 0.5, offset: 1pt, angle: -45deg),
  shine: (on: true, angle: 45deg, color: wain-shade),
)
```

3 pgfgosmooth

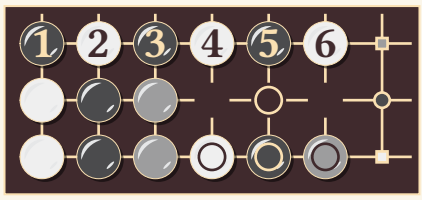
Como `pgfgo`, sin el borde del tablero ni el de las piedras, con las esquinas más redondeadas.




```
#let pgfgosmooth = (
  profile: pgfgo,
  background: (line: false, rounded-corners: (inner: 2pt, outer: 8pt)),
  grid: (line: (color: pgfgosmooth-grid)),
  players: (
    neutral: (stone: (line: false, radius: 0.19cm, shine: (opacity: 0.5))),
    black: (stone: (line: false, shine: (opacity: 0.5))),
    white: (stone: (line: false)),
  ),
)
```

4 wain

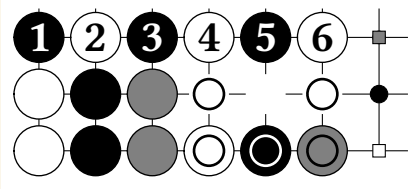
Un tablero oscuro, con líneas claras, y piedras con sombra y brillo. Sus colores y `raised` están en `pgfgo`.



```
#let wain = (
  background: (fill: (color: wain-goban, opacity: 1), line: (color: wain-grid, opacity:
1)),
  grid: (line: (color: wain-grid, width: 1pt)),
  label: (text: (fill: wain-goban)),
  players: (
    all: (
      stone: raised + (line: (on: true, color: wain-grid, width: 0.2pt, opacity: 1),
radius: 0.17cm),
      move: (label: (text: (fill: black))),
      mark: (line: (color: wain-goban, width: 1pt), label: (text: (fill: wain-goban))),
    ),
    neutral: (stone: (fill: (color: xmix(wain-black, 50%, wain-white)))),
    white: (stone: (fill: (color: wain-white)), move: (label: (text: (fill: wain-
goban)))),
    black: (
      stone: (fill: (color: wain-black)),
      move: (label: (text: (fill: wain-grid))),
      mark: (line: (color: wain-grid), label: (text: (fill: wain-grid))),
    ),
  ),
  arrow: (color: wain-grid),
  line: (color: wain-grid),
)
```

5 simple

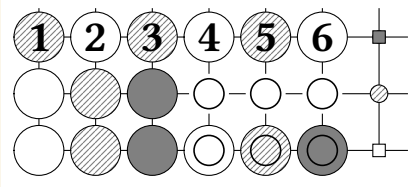
Como el estilo inicial, sin el borde del tablero y con líneas más finas.




```
#let simple = (
  background: (line: false),
  grid: (line: (width: 0.4pt)),
  players: (all: (stone: (line: (width: 0.5pt)))),
  arrow: (width: 0.8pt),
  line: (width: 0.6pt),
)
```

6 draft

Blanco y negro, para imprimir o fotocopiar: donde hay relleno, rayas. Las piedras negras, las marcas rellenas y el territorio usan `fill.pattern`; el territorio, con rayas en un sentido para cada jugador, se ve con `draw: "fill"` o `draw: "region"`. `lines` dibuja las rayas: un `tiling` de líneas a 45 o a −45 grados.

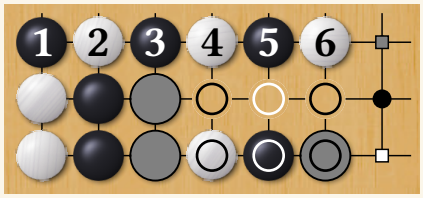



```
#let draft = (
  background: (line: false),
  grid: (line: (width: 0.4pt)),
  arrow: (width: 0.6pt),
  line: (width: 0.4pt),
  players: (
    all: (
      stone: (line: (width: 0.4pt)),
      mark: (line: (width: 0.6pt), fill: (color: white, pattern: lines(45deg, 1.2pt,
0.3pt, black))),
    ),
    black: (
      stone: (fill: (color: white, pattern: lines(45deg, 1.5pt, 0.4pt, pgf-gray))),
      move: (label: (text: (fill: black))),
      mark: (line: (color: black), label: (text: (fill: black))),
      territory: (fill: (color: none, pattern: lines(45deg, 2pt, 0.4pt, pgf-gray))),
    ),
    white: (territory: (fill: (color: none, pattern: lines(-45deg, 2pt, 0.4pt, pgf-
gray)))),
  ),
)
```

```
#let lines(angle, distance, width, color) = {
  let s = distance * calc.sqrt(2)
  tiling(size: (s, s), for k in (-1, 0, 1) {
    let (y1, y2) = if angle > 0deg { (s, 0pt) } else { (0pt, s) }
    place(line(start: (k * s, y1), end: (k * s + s, y2), stroke: (paint: color,
thickness: width)))
  })
}
```

7 shell

Un tablero de madera con piedras de pizarra y de concha, dibujadas con las imágenes de la carpeta `images` del paquete. Cada piedra blanca usa una de cinco imágenes, con un pequeño giro. La sombra es una imagen difusa (`soft-shadow`).



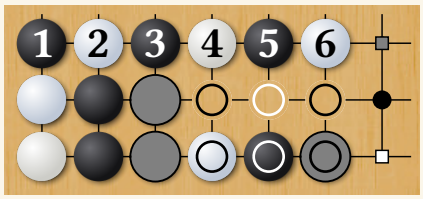

```
#let shell = (
  background: (line: false, fill: (image: picture("board.jpg"))),
  arrow: (halo: false),
```

```
  line: (halo: false),
  players: (
    all: soft-shadow,
    black: (stone: (image: picture("shell-black.png"))),
    white: (stone: (image: (
      file: range(1, 6).map(n => picture("shell-white-" + str(n) + ".png")),
      rotate: (-15deg, 15deg),
    ))),
  ),
)
```

```
#let picture(name) = path("../images/pgfgo-" + name)
#let soft-shadow = (stone: (shadow: (
  on: true, angle: -45deg, offset: 0.75pt, scale: 1.2, opacity: 0.6, image:
picture("shadow.png"),
)))
```

8 yunzi

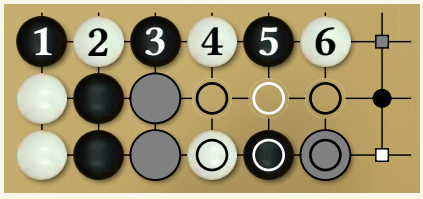
Como `shell`, con piedras mates, sin vetas, como las *yunzi* chinas: dos imágenes para las blancas, sin giro, porque la luz de cada imagen viene de un lado.




```
#let yunzi = (
  profile: shell,
  players: (
    black: (stone: (image: picture("yunzi-black.png"))),
    white: (stone: (image: (file: (picture("yunzi-white-1.png"), picture("yunzi-
white-2.png")), rotate: 0deg))),
  ),
)
```

9 baduktv

Un tablero claro, de madera lisa, con piedras lisas, una imagen para cada color, sin giro; la sombra difusa es la de `shell`.




```
#let baduktv = (
  background: (line: false, fill: (image: picture("baduktv-goban.png"))),
  arrow: (halo: false),
```

```
  line: (halo: false),
  players: (
    all: soft-shadow,
    black: (stone: (image: picture("baduktv-black.png"))),
    white: (stone: (image: picture("baduktv-white.png"))),
  ),
)
```

14 Novedades

- 0.2.0: lo nuevo de pgf-go después de 0.2026.10.23: `goban-stone`, vistas de varias partes, `shift`, `blank`, `grid.edge`, `layout.valign` y `layout.align-captions` (`layout.align` pasa a llamarse `layout.halign`), `origin` y `transpose` en `sgf-read`, tableros de más de 25 columnas y los diagramas de Sensei's Library. Los ejemplos de pgf-go, con una guía en PDF.
- 0.1.0: primera versión. Es un port de pgf-go 0.2026.10.23, con sus mismas ideas y casi todas sus opciones, escritas con los valores de Typst (pág. 5). La lista completa de los cambios está en el archivo `CHANGELOG.md`.

Índice de funciones y opciones

annotations ... 92
arrow.bend ... 70
arrow.color ... 69
arrow.dash ... 70
arrow.halo ... 70
arrow.halo.color ... 70
arrow.halo.width ... 70
arrow.opacity ... 69
arrow.shorten ... 70
arrow.tip ... 70
arrow.tips ... 70
arrow.width ... 69
arrows ... 69
background ... 46
background.fill ... 46
background.fill.color ... 47
background.fill.image ... 47
background.fill.image.anchor ... 48
background.fill.image.file ... 47
background.fill.image.fit ... 48
background.fill.image.rotate ... 49
background.fill.image.tiles ... 48
background.fill.image.width ... 48
background.fill.opacity ... 47
background.fill.pattern ... 47
background.line ... 49
background.line.color ... 49
background.line.opacity ... 49
background.line.width ... 49
background.rounded-corners ... 49
background.sep ... 50
background.shadow ... 50
background.shadow.angle ... 50
background.shadow.color ... 50
background.shadow.offset ... 50
background.shadow.opacity ... 50
blank ... 51
board ... 40
captured-marks ... 72
captures ... 72
charset ... 86
chat ... 92
circle ... 64
content ... 65
cross ... 64
custom ... 65
diagrams.annotations ... 97
diagrams.caption ... 94
diagrams.caption.min-width ... 94
diagrams.caption.text ... 94
diagrams.chat ... 98
diagrams.comments ... 96
diagrams.comments.align ... 96
diagrams.comments.height ... 97
diagrams.comments.justify ... 96
diagrams.comments.number ... 96
diagrams.comments.position ... 97
diagrams.comments.root ... 96
diagrams.comments.sep ... 97
diagrams.comments.separator ... 97
diagrams.comments.text ... 96
diagrams.comments.width ... 97
diagrams.figures ... 96
diagrams.from ... 93
diagrams.goban ... 95
diagrams.last-mark ... 93
diagrams.layout ... 95
diagrams.marks ... 94
diagrams.notes ... 94
diagrams.numbers ... 93
diagrams.partial ... 96
diagrams.restart ... 93
diagrams.restart.from ... 94
diagrams.result ... 98
diagrams.result.black ... 98
diagrams.result.format ... 98
diagrams.result.title ... 98
diagrams.result.white ... 98
diagrams.step ... 93
diagrams.title ... 94
diagrams.to ... 93
diagrams.variation ... 95
diagrams.variation-title ... 95
diagrams.variations ... 95
diagrams.variations.depth ... 95
diagrams.variations.from ... 95
diagrams.variations.paths ... 95
diagrams.variations.select ... 95
diagrams.view ... 96
dim ... 65
filled-circle ... 65
filled-square ... 65
forget ... 60, 62, 68
from ... 61, 65
game ... 86
goban ... 40
goban-layout ... 103
goban-notes ... 73
goban-prisoners ... 72
goban-region ... 78
goban-score ... 78
goban-stone ... 83
goban-view ... 81
gobanist-set ... 30

grid ... 42
grid.edge ... 42
grid.image ... 43
grid.image.file ... 43
grid.image.fit ... 43
grid.image.rotate ... 44
grid.image.tiles ... 44
grid.line ... 42
grid.line.color ... 42
grid.line.opacity ... 42
grid.line.width ... 42
grid.sep ... 43
hatched-lines ... 65
hoshi ... 44
hoshi.at ... 45
hoshi.color ... 45
hoshi.image ... 45
hoshi.image.file ... 46
hoshi.image.rotate ... 46
hoshi.image.width ... 46
hoshi.opacity ... 45
hoshi.radius ... 45
label.at ... 52
label.columns ... 52
label.columns-direction ... 53
label.rows ... 53
label.rows-direction ... 53
label.sep ... 52
label.style ... 53
label.text ... 52
layout ... 104
layout.align-captions ... 106
layout.column-sep ... 104
layout.columns ... 104
layout.fit ... 104
layout.halign ... 105
layout.per-page ... 106
layout.row-sep ... 104
layout.valign ... 105
layout.width ... 104
line.bend ... 71
line.color ... 71
line.dash ... 71
line.halo ... 71
line.opacity ... 71
line.shorten ... 71
line.width ... 71
lines ... 69
mark ... 31, 60
mark.fill ... 67
mark.fill.color ... 67
mark.fill.opacity ... 67
mark.fill.pattern ... 67
mark.label ... 67
mark.label.format ... 67
mark.label.size ... 67
mark.label.text ... 68
mark.line ... 66
mark.line.color ... 66
mark.line.opacity ... 66
mark.line.width ... 66
mark.scale ... 66
marks ... 64
move ... 31
move.label ... 61
move.label.format ... 62
move.label.size ... 62
move.label.text ... 62
move.numbers ... 61
moves ... 60
notes ... 73
notes.at ... 73
notes.separator ... 73
notes.text ... 73
origin ... 86
partial ... 50
partial-margin ... 51
player ... 66
players ... 31, 107
players.all ... 107
players.black ... 107
players.black.territory.fill ... 76
players.black.territory.fill.color ... 76
players.black.territory.fill.opacity ... 76
players.black.territory.fill.pattern ... 76
players.black.territory.line ... 77
players.black.territory.line.above ... 77
players.black.territory.line.color ... 77
players.black.territory.line.opacity ... 77
players.black.territory.line.width ... 77
players.black.territory.mark ... 76
players.black.territory.vertices ... 77
players.black.territory.vertices.color . 78
players.black.territory.vertices.radius 78
players.neutral ... 107
players.white ... 107
players.white.mark ... 107
players.white.move ... 107
players.white.prisoner ... 107
players.white.stone ... 107
players.white.territory ... 108
prisoner ... 31, 63
prisoners ... 62
problems.annotations ... 102
problems.caption ... 102
problems.chat ... 102
problems.comments ... 102
problems.goban ... 102
problems.layout ... 102
problems.links ... 101
problems.notes ... 101
problems.partial ... 101
problems.solutions ... 100
problems.statement ... 100
problems.title ... 100
problems.to-play ... 100
problems.variation-title ... 100
problems.verdict ... 101
problems.view ... 101
profile ... 108

resume ... 80
resume.marks ... 81
resume.numbers ... 80
resume.territory ... 81
scale ... 41
scale-line-width ... 41
score.komi ... 78
sequence ... 65
sgf ... 88
sgf-comment ... 92
sgf-diagrams ... 92
sgf-info ... 91
sgf-problem ... 99
sgf-read ... 85
sgf.from ... 88
sgf.last-mark ... 90
sgf.letters ... 90
sgf.marks ... 89
sgf.numbers ... 89
sgf.restart ... 90
sgf.restart.from ... 90
sgf.step ... 89
sgf.to ... 88
sgf.variation ... 88
sgf.view ... 91
shapes ... 68
shift ... 60, 68
size ... 41
square ... 64
stone ... 31
stone.fill ... 54
stone.fill.color ... 54
stone.fill.opacity ... 54
stone.fill.pattern ... 55
stone.image ... 58
stone.image.file ... 59
stone.image.opacity ... 59
stone.image.rotate ... 59
stone.image.width ... 59
stone.line ... 55
stone.line.color ... 55
stone.line.opacity ... 55
stone.line.width ... 55
stone.radius ... 55
stone.scale ... 55
stone.shadow ... 56
stone.shadow.angle ... 56
stone.shadow.color ... 56
stone.shadow.image ... 56
stone.shadow.image.file ... 57
stone.shadow.image.rotate ... 57
stone.shadow.image.silhouette ... 57
stone.shadow.image.width ... 57
stone.shadow.offset ... 56
stone.shadow.opacity ... 56
stone.shadow.scale ... 56
stone.shine ... 57
stone.shine.angle ... 57
stone.shine.color ... 57
stone.shine.image ... 58
stone.shine.image.file ... 58
stone.shine.image.rotate ... 58
stone.shine.image.silhouette ... 58
stone.shine.image.width ... 58
stone.shine.opacity ... 57
stones ... 54
syntax ... 86
t-circle ... 65
t-square ... 65
territory ... 74
territory.dame ... 76
territory.draw ... 75
texts ... 32
transpose ... 86
triangle ... 64
use-i ... 41
view ... 82
view.arrows ... 82
view.forgotten ... 82
view.goban ... 83
view.last ... 83
view.lines ... 82
view.marks ... 82
view.numbers ... 82
view.raise ... 83
view.separator ... 83
view.territory ... 82